15 直すべき所と直し方（改修の一覧）

この巻は、業務工数システムのプログラム（GitHub の `master`。**2026 年 11 月 9 日のコミット `ec2cedc`**）を 1 ファイルずつ読み直して見つけた「直すべき所」を、**全部** 並べた物です。

前の版は、4 月のコミット `30d45e7` を読んで書きました。その後 `master` は大きく変わりました（共通の確かめ `kosu/permissions.py` が入った・画面のビルドが Vite に変わった・全部のファイルのコメントが書き直された、など）。そこで、全部を読み直し、**直った所は §3 に、残っている所と新しく見つかった所は §5 から** 書き直しました。

どの項目にも、次を書いています。

- **どこ**: ファイルと行番号（`ec2cedc` の行番号）
- **何が困るか**と、手元で確かめた結果
- **直し方**: 直す場所ごとに、「**今**」のコードと「**直した後**」のコードを、全部。直した後のコードは、手元の写しに全部入れて、アプリのテスト（353 個）と画面のテスト（534 個）が全部通ることを確かめた物です（§10）
- **直ったことの確かめ方**

⚠️ **いちばん急ぐこと**: 今の `master` のまま本番に出すと、**ログインした後の保存・削除・ログアウトが、全部 403 で失敗します**（S-1）。GitHub Actions（`master` に入れると本番に出す仕組み）は今、GitHub の画面で手動で止めてあるので（2026 年 11 月 9 日に確かめた）、まだ本番には出ていません。**S-1 を直すまでは、Actions を動かさないでください。**

🔰 **初めての方へ**: 全部をいっぺんに直す必要はありません。§2 の一覧で「🔴 すぐ」の物から、1 つずつ直してください。直した後のコードは、そのまま貼れば動くように書いてあります。コードの枠の右上の「コピー」で写せます。ただし **貼る前に、必ず作業用のブランチを作ってください**（§4）。

📌 **ほかの巻について**: 00〜12 巻と 000 巻には、4 月のコード（`30d45e7`）を元に書いた所（行番号・コードの抜き出し・「API ごとにログインを確かめる」書き方の説明など）が残っています。今のコードと違う所があったら、この巻を正としてください。

この巻で分かること

- 今の `master` で、直さないと困る所（安全・不具合・整理）の全部と、その場所
- 直す場所ごとの「今のコード」と「直した後のコード」（手元で動かして確かめた物）
- 直した後に足すテストと、直す順番

目次

1. この巻の読み方
2. まず、全体を 1 枚で（全部の一覧）
3. 前の版から直った所

4. 直す前の準備
5. 安全の問題（S）
6. 不具合（B）
7. 開発・運用の整理（D）
8. テスト（T）
9. 直す順番（おすすめ）
10. この巻で確かめたこと
11. この巻のまとめ

---

1. この巻の読み方

1.1 印の意味

急ぐ度

印	意味
🔴 すぐ	本番のデータが守られていない、または使えなくなる。ほかの作業より先に直す
🟠 早めに	動かない、または壊れやすい。使う人が困る
🟡 余裕のある時に	整理・改善。直さなくても、すぐには困らない

今の状態（前の版の一覧と比べて）

書き方	意味
新しく見つかった	前の版には無かった（今回のコードの変更で起きた物も含む）
前から残っている	前の版から、まだ直っていない
一部だけ直った	前の版の直し方の一部が入った。残りを書いている
直った	もう直っている。§3 に、どう直ったかを書いた

どうやって分かったか

印	意味
✅ 手元で再現	`master` を手元の別のフォルダに書き出し、メモリの中だけのデータベースで動かして、本当にそうなることを確かめた（本番には一切触っていない。§10）
📖 コードから	コードを読んで分かった。動かしては確かめていない

番号の頭の文字: S ＝ 安全（Security）、B ＝ 不具合（Bug）、D ＝ 開発・運用の整理（Development）、T ＝ テスト（Test）。番号は前の版と同じ物を使っています（直った番号も、欠番にしないで残しています）。

1.2 「今」と「直した後」の読み方

直し方は、次の形で書いています。

- **場所:** ファイルと、`ec2cedc` での行番号。たとえば「`kosu/views/main_views.py` の 196〜222 行目」。
- **今:** その行の、今のコード（コメントも含めて、ファイルのとおり）。
- **直した後:** 「今」の行を、全部これに置き換えます。
- 「この行の前に入れる」と書いてある物は、何も消さずに、その行の前に足します。
- 「新しいファイル」と書いてある物は、そのファイルを新しく作り、中身を全部入れます。

> ⚠️ **行番号は、上から直していくと、ずれます。** 1 つのファイルの中で何か所も直す時は、**下の方の場所から** 直すか、行番号ではなく「今」のコードを、エディターの検索（ `Ctrl` ＋ `F` ）で探して置き換えてください。
>
> ⚠️ **本番のホスト名（住所）は、この資料には書いていません。** コードの中で `<本番のホスト名>` と書いた所は、今のファイルに書いてある名前を、そのまま使ってください。

1.3 この巻で使う言葉

言葉	意味
API（エーピーアイ）	画面（React）がサーバー（Django）に「データをちょうだい」「これを保存して」と頼む窓口。`/api/member_new/` のような URL の 1 つ 1 つ（07 巻）
ログインの確かめ（認証）	「今、だれがログインしているか」を確かめること。このアプリでは、ログインすると **セッション**（サーバーが覚えておく、その人用のメモ）に従業員番号（`login_No`）が入り、`kosu/permissions.py` の `LoginNoAuthentication` がそれを読む
権限の確かめ（認可）	「その人に、その操作をしてよいか」を確かめること。人員の表の「権限（`authority`）」と「管理者（`administrator`）」の 2 つのスイッチを、`permissions.py` の `HasAuthority`・`IsAdministrator` が確かめる
CSRF（シーサーフ）	ログインしたままの人が、別の（悪い）サイトを開いた時に、そのサイトが裏でこのアプリへ「消して」などを送らせる攻撃。防ぐには、このアプリの画面だけが知っている合言葉（**CSRF トークン**）を、書き込みの時に一緒に送らせて確かめる
CORS（コルス）	別のサイトの JavaScript が、このアプリの API の返事を読んでよいかを、サーバーが答える仕組み
Cookie（クッキー）	ブラウザがサイトごとに覚えておく小さなデータ。ログインのセッションの番号（`sessionid`）も、CSRF トークン（`csrftoken`）もここに入る
ステータスコード	API の返事に付く 3 けたの番号。200 ＝ 成功、400 ＝ 送った物がおかしい、401 ＝ ログインしていない、403 ＝ してはいけない、404 ＝ 見つからない、409 ＝ ほかのデータとぶつかる、429 ＝ 呼びすぎ、500 ＝ サーバーの中でエラー（09 巻）
トランザクション	何件もの書き込みを「全部する」か「全部しない」かにまとめる仕組み。途中で失敗したら、それまでの書き込みも取り消す（08 巻）
マイグレーション	データベースの表の形の変更を書いたファイル（`kosu/migrations/`）。`migrate` で本物のデータベースに当てる（08 巻）

2. まず、全体を 1 枚で（全部の一覧）

番号	急ぐ度	今の状態	何が困るか（ひとことで）	主な場所
S-1	🔴	新しく見つかった	ログインした後の保存・削除・ログアウトが、全部 403 になる（CSRF の Cookie を渡していない）	`main_views.py` 148・182〜222 行目
S-2	🟡	一部だけ直った	確かめ漏れはほぼ無くなった。残りは 1 か所	`team_views.py` 769 行目
S-3	🔴	一部だけ直った	他人の工数を見る・日付を変える、他人の問い合わせを書き換える・消す、権限だけの人が管理者を作る、ができる	`kosu_views.py`・`team_views.py`・`inquiry_views.py`・`member_views.py`
S-4	🔴	一部だけ直った	ダウンロードの API が、渡された場所のファイルを、そのまま返して消そうとする（管理者だけになった）	`asynchronous_views.py` 589〜645 行目
S-5	🔴	一部だけ直った	CORS は直った。セッションの Cookie が、別のサイトからの頼みにも付く（`SameSite=None`）	`settings.py` 164〜167 行目
S-6	🔴（要相談）	前から残っている	ログインが従業員番号だけで、パスワードが無い。セッションは 10 年	`main_views.py` 182〜222 行目・`settings.py` 160〜162 行目
S-7	🔴	一部だけ直った	ヘルプの初期化はパスワードを確かめるようになった。総当たりを止めない・途中で失敗すると半分消える・パスワード未設定で 500	`main_views.py` 235〜381 行目
S-8	—	直った	サーバーのログは管理者だけが読める	—
S-9	🔴	一部だけ直った	氏名入りの Excel は消されたが、履歴に残っている。テストで作った Excel がリポジトリに入る	リポジトリ・`media/`
B-1	—	直った	ヘルプの URL の `/` は 1 つになった	—
B-2	🟠	新しく見つかった（前の B-2 が変わった）	バックアップのダウンロードの URL が `undefined/api/…` になり、Excel ではなく画面の HTML が落ちてくる	`AdministratorLoading.tsx` 117〜124 行目
B-3	🟠	前から残っている	定義の Ver 名を変える・消すと、その Ver で入れた工数が開けなくなる	`def_views.py` 293〜361 行目

番号	急ぐ度	今の状態	何が困るか（ひとことで）	主な場所
B-4	🔴	前から残っている	班員に入れた人を人員から消すと、班長の画面が 500	`team_views.py` 421・633 行目
B-5	🔴	前から残っている	人員から消した人の問い合わせが、開けない・直せない・消せない	`inquiry_views.py` 247〜250 行目など
B-6	🔴	前から残っている	同じ従業員番号の人が 2 人いると、その人の画面が 500	`models.py`
B-7	🔴	一部だけ直った	設定データが無いと、班のショップの画面・問い合わせの登録などが 500	`main_views.py`・`team_views.py`・`inquiry_views.py` の 10 か所
B-8	🔴	前から残っている	データのロードが途中で失敗すると、データが半分だけ消える	`tasks.py`
B-9	—	直った	ログインが切れている時は、どの API も 401 になった	—
B-10	🔴	前から残っている	ログイン以外の理由でも 401 を返し、画面がログイン画面へ飛ぶ（29 か所）	各ファイル
B-11	🟡	前から残っている	作業詳細選択肢の「ロード開始」が、定義のロード中に「実行中…」になる	`AdministratorLoading.tsx` 849 行目
B-12	🟡	前から残っている	ファイル選びで CSV・XLS も選べるが、サーバーは xlsx しか読めない	`AdministratorLoading.tsx` 7 か所
B-13	🔴	前から残っている	バックアップ・ロードがサーバーの再起動で止まり、「待ち」のまま残る	`asynchronous_views.py` 115〜123・555〜562 行目
B-14	🟡	前から残っている	月で絞り込むと、月初めの 0〜9 時の記録が前の月に入る	`main_views.py` 1081・1217 行目
B-15	🟡	前から残っている	思っていない値が送られると 500 になる（11 か所）	`kosu_views.py`・`team_views.py` など
B-16	🟡	前から残っている	月間・年間の集計で、最初の 1 件の定義 Ver で全部を数える	`kosu_views.py` 1755〜1760 行目
B-17	🟡	前から残っている	操作履歴を 1 件保存するたびに、全件を数えている（遅くなる）	`models.py` 649〜661 行目
B-18	🟡	前から残っている	問い合わせのお知らせが、別の人の問い合わせを指すことがある	`inquiry_views.py` 178 行目

番号	急ぐ度	今の状態	何が困るか（ひとことで）	主な場所
B-19	🔴	前から残っている	従業員番号を変えると、その人の工数・班・問い合わせが切り離される	`member_views.py` 272〜291 行目
B-20	🔴	前から残っている	`.gitattributes` のせいで、PDF・Excel などのファイルが壊れる（GitHub の PDF が壊れていた）	`.gitattributes`
B-21	🟡	前から残っている	Excel の見出し「ポップアップID6」の中身が ID5	`tasks.py` 398・479・1188・1277 行目
D-1	🔴	前から残っている	GitHub Actions がテストを流さずに本番へ出す作りのまま（今は手動で止めてある）	`.github/workflows/`
D-2	🔴	前から残っている	migrate の決まりが無い。モデルとマイグレーションが 26 か所食い違っている	`kosu/models.py`・`kosu/migrations/`
D-3	🟡	前から残っている	リポジトリの 1,581 ファイルのうち 1,291（212 MB）が、入れなくてよい物	`staticfiles/` など
D-4	🟡	新しく見つかった（前の D-4 が変わった）	手元で `npm start` すると、画面から API に届かない（Vite に中継の設定が無い）	`vite.config.mts`・`axios.ts`
D-5	🟡	前から残っている	住所（ホスト名）がコードに直接書いてある	`settings.py` 56〜62・306〜312 行目
D-6	🟡	前から残っている	HTTPS まわりの設定が足りない	`settings.py`
D-7	🟡	前から残っている	使っていない部品が、サーバー側に 13 個、画面側に 8 個ある	`requirements.txt`・`package.json`
D-8	🔴	一部だけ直った	Django 4.2 と Node 20 はサポートが終わった。画面の部品の版が、記録（lock）と手元で違う	`requirements.txt`・`package.json`
D-9	🟡	前から残っている	`print` の行き先をすり替えている（コマンドの出力まで消える）。ログのファイルの場所がずれる	`main_views.py` 70〜126 行目
D-10	🟡	前から残っている	古い入口 `/BA…` から、全部の API が使える	`hozen_another/urls.py` 34〜36 行目
D-11	🟡	前から残っている	使っていないコードが残っている	各所

番号	急ぐ度	今の状態	何が困るか（ひとことで）	主な場所
D-12	🟡	前から残っている	同じコードの繰り返しが多い	各所
D-13	🟡	前から残っている	URL の名前の入れ違い	`kosu/urls.py` 85〜90・157行目
D-14	🟡	新しく見つかった	アプリのリポジトリの `docs/` が、この資料の古い版。`CLAUDE.md` も古い	`docs/`・`CLAUDE.md`
T-1	🔴	新しく見つかった	CSRF を確かめる状態のテストが 1 つも無い（だから S-1 に気付けない）	`kosu/tests/`
T-2	🟠	前から残っている	直した所を守るテストを足す	`kosu/tests/test_security.py`（新しく作る）
T-3	🟡	新しく見つかった	画面のテストが 3 個落ちる（部品が変わったのに、テストが古いまま）	`frontend/src/tests/`
T-4	🟡	新しく見つかった	テストが `media/` に Excel を作る。直した後の動きに合わせるテストが 4 個	`test_settings.py`・`test_backend.py`

3. 前の版から直った所

前の版（4 月のコード）に書いた所のうち、次の物は、今の `master` で直っていました。✅ は、手元で確かめた物です。

前の番号	直った所	確かめた結果
S-1	`kosu/permissions.py` ができ、`settings.py` 148〜150 行目の `REST_FRAMEWORK` で、全部の API に最初から「ログインが要る」がかかるようになった	✅ ログインせずに書き込みの API を呼ぶと 401（前は 200・201 で、ログインせずに管理者を作れた）
S-2	管理者・権限の API に、`permission_classes = [IsAdministrator]`・`[HasAuthority]` が付いた	✅ 一般の人がログの API・ダウンロードを呼ぶと 403
S-3 の一部	工数の編集（`KosuUpdate.put`）・削除（`KosuDelete`）と、問い合わせの編集画面（`InquirUpdate.get`）で、本人か確かめるようになった	✅ 他人の工数を消すと 404
S-4 の一部	ダウンロード（`download_file`）が、管理者だけになった	✅ 一般の人は 403
S-5 の一部	CORS が、決めた 3 つのオリジン（手元の `localhost:3000`・`localhost:8000`・`127.0.0.1:8000`）だけになった	✅ 別のサイトを名乗ると、許可の見出しが返らない
S-7 の一部	ヘルプの初期化の API（`Help`）が、パスワードを確かめるようになった。画面（`Help.tsx`）もパスワードを送るようになった	✅ パスワード無しは 403
S-8	サーバーのログの API（`WebConsoleLogView`）が、管理者だけになった	✅ 一般の人は 403、ログインしていないと 401
S-9 の一部	リポジトリの直下にあった氏名入りのバックアップの Excel 5 つが、9 月 3 日のコミット `ea674d4` で消された（ただし履歴には残っている。S-9）	📖 `git log`
B-1	ヘルプの初期化の URL が `/api/help/` になった（`Help.tsx` 151 行目）	📖
B-9	ログインが切れている時は、どの API も 401 になった（前は 500 になる API があった）	✅
D-8 の一部	画面のビルドが、開発の終わった Create React App から Vite に変わった	✅ 手元で `vite build` が通った
T-1（前の版）	直すと落ちるテスト 8 個が、直した後の動きに合わせて書き換えられた	✅ アプリのテスト 339 個が全部 OK

> 前の版の S-1 の直し方のうち、**「画面を開いた時に CSRF トークンの Cookie を渡す」（`ensure_csrf_cookie`）だけが入っていません。**そのせいで、書き込みが全部 403 になります（S-1）。前の版の §8 にも「CSRF の確かめだけ入れて、トークンを配らないと、書き込みが全部 403 になる」と書いた組み合わせです。

4. 直す前の準備

4.1 作業用のブランチを作る

コピー

```
cd hozen-kosu-React
git switch master
git pull
git switch -c fix/csrf-and-security
```

直し終わったら、ブランチを GitHub に push し、GitHub の画面で **Pull Request**（「このブランチを master に入れてください」というお願い）を作ります。もう 1 人に見てもらってから master に入れます（02 巻）。

> 📌 **今は、master に入れても本番に出ません。**GitHub Actions のワークフロー「Deploy Django + React to Azure」が、GitHub の画面で手動で止めてあります（Actions のタブで、ワークフローの名前の横に「disabled」と出ます）。最後に動いたのは 2026 年 9 月 3 日のコミット `7544e64` です。止めた理由を前任者に確かめ、動かすのは S-1 と D-1 を直してからにしてください。

4.2 本番のデータを、先にバックアップする

- **Azure のバックアップ**: 本番のデータベース（Azure Database for PostgreSQL）には、Azure が自動で取るバックアップがあります。何日分残っているか、戻す手順を、前任者に確かめておきます（000 巻 §10）。
- **データ管理画面のバックアップ**: 000 巻 §8.2 の 9 種類。本番で今動いているのは 9 月の版なので、今の `master` の不具合（B-2）はまだ出ていないはずですが、ダウンロードしたファイルが Excel で開けるかを、必ず確かめてください。

4.3 今のテストが全部通ることを確かめる

コピー

```
python manage.py test kosu --settings=hozen_another.test_settings
```

2026 年 11 月 10 日に `ec2cedc` の写しで流した時は、**339 個すべて OK** でした（最後に `Ran 339 tests` と `OK` が出ます）。

画面側のテスト（`frontend/src/tests/` の 65 ファイル・535 個）も流します。

コピー

```
cd frontend
npm run test:run
```

今の `master` では、**535 個のうち 3 個が落ちます**（T-3。テストのファイルの中にも「今の部品と合わず失敗する」と書いてあります）。

> 💡 **Node.js 25 以上で流すと、`localStorage.getItem is not a function` で 50 個ほど余計に落ちます。** Node.js 25 から、Node.js 自身が `localStorage` を持つようになり、テスト用のブラウザの再現（jsdom）とぶつかるためです。GitHub Actions と同じ Node.js 22 を使うか、次のように Node.js の `localStorage` を切ってから流してください。
>
> コピー
>
> ```
> $env:NODE_OPTIONS = "--no-experimental-webstorage"
> npm run test:run
> ```

4.4 1 つ直したら、すぐ確かめて、コミットする

- 1 つの番号（例: S-4）を直したら、テストを流し、手元でその画面を動かしてから、コミットします。
- コミットのメッセージに番号を書いておくと、後で探せます（例: `S-4: ダウンロードを media の中のファイルだけにする`）。
- いくつもの番号を 1 つのコミットに混ぜないでください。何かが壊れた時に、どれのせいか分からなくなります。

4.5 直す順番の決まり（先に入れておく物）

いくつかの直しは、ほかの直しで足す関数を使います。次の順に入れてください。

後で入れる物	先に入れておく物	理由
S-3 の④（問い合わせ）、B-5、B-18	B-7 の①（`get_settings` を足す）	設定データを `get_settings()` で取るように直すため
B-15 の①（時刻・年月の関数）	B-7 の①（`get_settings` を足す）	同じ `main_utils.py` の最後に、続けて足すため
B-6 のマイグレーション（`0014`）	D-2 のマイグレーション（`0013`）	`0014` は `0013` の続きとして作ってあるため
T-4 の 4 つのテストの直し	S-4・B-10	直した後の動き（ファイル名だけ返す・404）を確かめるテストのため

S-3 の⑤（人員の直し）には、B-19（従業員番号を変えた時のそろえ）の直しも入っています。

---

5. 安全の問題（S）

5.0 今のしくみ（共通の確かめ）

今の `master` では、`kosu/permissions.py` の部品が、**全部の API の手前で** 次の順に確かめます（`settings.py` 148〜150 行目で、最初から全部の API にかけている）。

1. **だれか**（`LoginNoAuthentication`）：セッションの `login_No` から人員を探す（`permissions.py` 59〜71 行目）。見つかった人の **書き込み**（POST・PUT・DELETE）では、続けて **CSRF トークン** を確かめる（69 行目）。
2. **使ってよいか**（`IsLoggedIn`・`HasAuthority`・`IsAdministrator`）：ログインしていなければ 401、権限が足りなければ 403。

ログインが要らない API（ログイン・ログアウト・ヘルプ）だけ、`permission_classes = [AllowAny]` で外しています。前の版で「API ごとに書いていて、書き忘れが多い」と書いた問題は、これで片付きました（§3）。

残っているのは、(1) CSRF トークンの Cookie を渡していないこと（S-1）、(2)「ログインしている」の先の「**自分のデータか**」の確かめ（S-3）、(3) いくつかの API の作り（S-4・S-7）です。

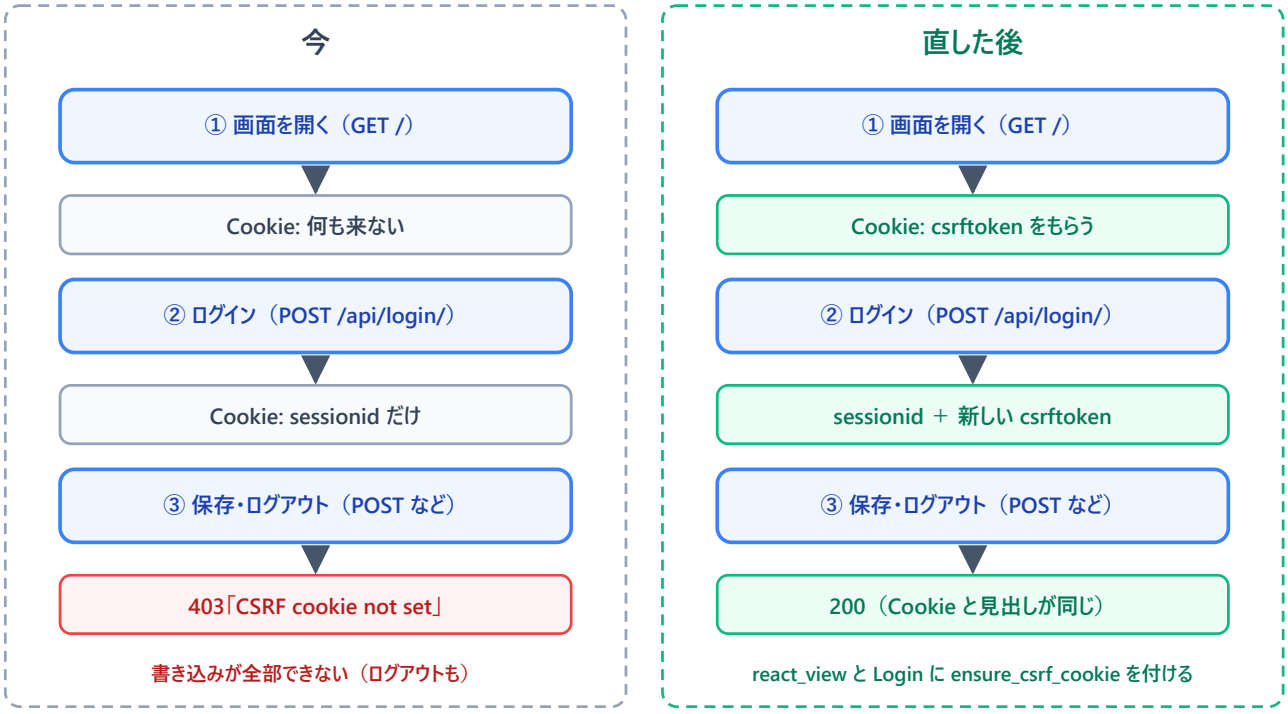


図1: CSRF トークンの Cookie の流れ。今は、画面を開いてもログインしても csrftoken の Cookie が来ないので、ログインした後の書き込みが、共通の確かめ（permissions.py）で全部 403 になる。直した後は、画面を開いた時に Cookie を受け取り、axios がその値を X-CSRFToken の見出しに写して送るので、通る

5.1 S-1 ログインした後の保存・削除・ログアウトが、全部 403 になる

🔴 すぐ ／ 新しく見つかった ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 148 行目（`react_view`）・182〜222 行目（`Login`）、`kosu/permissions.py` 69 行目、`frontend/src/api/axios.ts` 40〜41 行目

何が起きているか:

- CSRF トークンは、ブラウザの Cookie「`csrftoken`」と、画面（axios）が書き込みの時に付ける見出し「`X-CSRFToken`」が **同じか** で確かめます（`axios.ts` 40〜41 行目が、Cookie の値を見出しに写す設定）。
- Django は、**頼まれた時しか `csrftoken` の Cookie を作りません**（`ensure_csrf_cookie` を付けたビューや、`get_token()` を呼んだ時だけ）。今のコードには、そのどちらもありません（`git grep` で確かめた）。
- そのため、ブラウザには `csrftoken` が無いまま、ログインした後の書き込みで `permissions.py` 69 行目の確かめが「CSRF cookie not set（Cookie がありません）」で失敗し、403 になります。

✅ **手元で、ブラウザと同じ流れ（CSRF を確かめる状態）で試した結果**（`master` の写し）:

順番	送った物	返事
1	画面を開く（`GET /`）	200。Cookie は **何も来ない**
2	ログイン（`POST /api/login/`）	200。来た Cookie は `sessionid` だけ
3	メインメニュー（`GET /api/main_menu/`）	200（読むだけの API は CSRF を確かめない）
4	定義の Ver を選ぶ（`POST /api/def_ver/`）	**403**「CSRF Failed: CSRF cookie not set.」
5	ログアウト（`POST /api/logout/`）	**403**（同じ）
6	もう一度ログイン（`POST /api/login/`）	**403**（同じ。セッションに `login_No` が残っているので、ログインの API でも CSRF を確かめる）

つまり、ログインはできても、**工数の保存も、ログアウトも、ログインし直しも、できません。**アプリのテスト（339 個）は全部通りますが、それは Django のテスト用の `Client` が、最初から CSRF の確かめを切っているからです（T-1）。

あわせて、ログインの時にセッションの番号を新しくしていません（206〜207 行目。ログイン前のセッションの番号を知られていると、ログイン後もそのまま使われるおそれがある）。工数区分定義が 1 件も無い時に、エラーを返す前に `login_No` を保存してしまう（207 行目）ことも、ここで一緒に直します。

直し方: `main_views.py` を 4 か所直します。画面側（`axios.ts`）は直さなくて構いません（Cookie があれば、axios が自動で見出しに写します）。

① `import` に、使う部品を足します。この巻のほかの直し（S-7）で使う `hmac`・`transaction`・`ScopedRateThrottle` も、ここで一緒に足しています（使う前に足しておいても、動きは変わりません）。

場所: `kosu/views/main_views.py` の 47〜63 行目（import の並び）

今:

コピー

```
import datetime
import os
import sys
import logging
# environ: .env ファイル（環境変数の設定ファイル）を読み込むライブラリ（django-environ）
import environ
import json
from pathlib import Path
from django.shortcuts import render
from django.http import JsonResponse
from django.conf import settings
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import AllowAny
from rest_framework import status
from ..models import member, Business_Time_graph, kosu_division, team_member, administrator_data, AsyncTask, History
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer, DefSerializer, TaskSerializer, HistorySerializer
```

▶ ここで初めて出てくるコード

- **`AllowAny`**（Django REST framework）:「だれでも使える」という確かめ（いつも通す）。全部の API に「ログインが要る」をかけた後で、ログイン画面のように、ログインしていなくても使う API にだけ `permission_classes = [AllowAny]` と付けて外します。

直した後（上の行を、全部これに置き換える）:

コピー

```
import datetime
import hmac  # 2つの文字列を「かかる時間が中身で変わらない」方法で比べる（ヘルプ用パスワード）
import os
import sys
import logging
import json
from django.db import transaction  # 複数の書き込みを「全部する / 全部しない」にまとめる
from django.shortcuts import render
from django.http import JsonResponse
from django.conf import settings
from django.middleware.csrf import rotate_token  # CSRF トークンを新しくする（ログインの時）
from django.utils.decorators import method_decorator  # 関数用のデコレーターを、クラスのビューに付ける
from django.views.decorators.csrf import ensure_csrf_cookie  # 返事に csrftoken の Cookie を必ず付ける
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import AllowAny
from rest_framework.throttling import ScopedRateThrottle  # 同じ所からの呼び出し回数を制限する
from rest_framework import status
from ..models import member, Business_Time_graph, kosu_division, team_member, administrator_data, AsyncTask, History
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer, DefSerializer, TaskSerializer, HistorySerializer
```

▶ ここで初めて出てくるコード

- **`ScopedRateThrottle`**（Django REST framework）: 同じ所（IP アドレス）からの呼び出しの回数を数えて、上限を超えたら 429（呼びすぎ）を返す部品。ビューに `throttle_classes = [ScopedRateThrottle]` と `throttle_scope = 'help'` を書くと、`settings.py` の `REST_FRAMEWORK` の `DEFAULT_THROTTLE_RATES` の `'help': '5/hour'`（1 時間に 5 回）が上限になります。

② 画面を開いた時に、`csrftoken` の Cookie を渡します。`react_view` は、`/api/` 以外の全部の URL（画面）を返す関数です。

場所: `kosu/views/main_views.py` の 148 行目（react_view）

今:

コピー

```
def react_view(request):
```

直した後（上の行を、全部これに置き換える）:

コピー

```
@ensure_csrf_cookie  # 画面を開いた時に、CSRF トークンの Cookie（csrftoken）を渡しておく
def react_view(request):
```

▶ ここで初めて出てくるコード

- `@ensure_csrf_cookie`（Django）: その画面（ビュー）の返事に、CSRF の合言葉（`csrftoken` というクッキー）を必ず付ける印。React の画面は、このクッキーの値を、書き込みの時に `X-CSRFToken` という見出しに入れて送ります（`frontend/src/api/axios.ts`）。

③ ログインの返事にも、`csrftoken` の Cookie を付けます（画面を開いたまま、新しい版が本番に出た時のため）。

場所: `kosu/views/main_views.py` の 182〜184 行目（Login の頭）

今:

コピー

```
class Login(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
```

直した後（上の行を、全部これに置き換える）:

コピー

```
@method_decorator(ensure_csrf_cookie, name='dispatch')  # ログインの返事にも csrftoken の Cookie を付ける
class Login(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
```

▶ ここで初めて出てくるコード

- `method_decorator()`（Django）：関数用の印（デコレーター）を、クラスの画面（ビュー）のメソッドにも付けられるようにする物。
- `@method_decorator`（Django）：`@method_decorator(ensure_csrf_cookie, name="dispatch")` をクラスの上に付けて、そのクラスの画面に印を付けます。

④ ログインの中身を直します。セッションの番号と CSRF トークンを新しくし（`cycle_key()`・`rotate_token()`。Django 標準の `login()` と同じこと）、従業員番号が数字でない時は 500 ではなく 400 を返します。

場所: `kosu/views/main_views.py` の 196〜222 行目（Login.post）

今:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
  def post(self, request):
    try:
      # 送られてきたデータ確認（リクエスト本文のJSONを辞書に変換し、従業員番号を取り出す）
      data = json.loads(request.body)
      input_number = data.get('employee_no')
    # 本文がJSONとして読めない場合は 400 を返す
    except json.JSONDecodeError:
      return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'},
status=status.HTTP_400_BAD_REQUEST)

    # POSTされた従番が人員データにある場合、セッションに保存&最新工数区分定義取得
    if member.objects.filter(employee_no=input_number).exists():
      request.session['login_No'] = input_number
      # ID が最大（最後に登録された）工数区分定義を「最新」として取得
      def_Ver = kosu_division.objects.order_by("id").last()

      # 工数区分定義が1件も無い場合は工数入力ができないため 400 を返す
      #   ※login_No はすでにセッションに保存された状態のまま返る
      if not def_Ver:
        return JsonResponse({'status': 'error', 'message': '利用可能な工数区分がありません。ERROR052'},
status=status.HTTP_400_BAD_REQUEST)
      # 取得した工数区分定義のVer名をセッションに保存し、ログイン成功を返す
      else:
        request.session['input_def'] = def_Ver.kosu_name
        return JsonResponse({'status': 'success'})

    # 従業員番号が人員データに無い場合は 400 を返す
    else:
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登録がありません。
ERROR048'}, status=status.HTTP_400_BAD_REQUEST)
```

直した後（上の行を、全部これに置き換える）：

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
  def post(self, request):
    try:
      # 送られてきたデータ確認（リクエスト本文のJSONを辞書に変換し、従業員番号を数値で取り出す）
      data = json.loads(request.body)
      input_number = int(data.get('employee_no'))
    # 本文がJSONとして読めない・従業員番号が数字でない場合は 400 を返す（500 にしない）
    except (json.JSONDecodeError, TypeError, ValueError, AttributeError):
      return JsonResponse({'status': 'error', 'message': '従業員番号を数字で入力してください。'},
status=status.HTTP_400_BAD_REQUEST)

    # 従業員番号が人員データに無い場合は 400 を返す
    if not member.objects.filter(employee_no=input_number).exists():
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登録がありません。
ERROR048'}, status=status.HTTP_400_BAD_REQUEST)

    # ID が最大（最後に登録された）工数区分定義を「最新」として取得
    def_Ver = kosu_division.objects.order_by("id").last()
    # 工数区分定義が1件も無い場合は工数入力ができないため、ログインさせずに 400 を返す
    if not def_Ver:
      return JsonResponse({'status': 'error', 'message': '利用可能な工数区分がありません。ERROR052'},
status=status.HTTP_400_BAD_REQUEST)

    # ログインの前に、セッションIDと CSRF トークンを新しくする
    #   （ログイン前のセッションIDやトークンを他人に知られていても、ログイン後には使えないようにする。
    #    Django 標準の login() も同じことをしている）
    request.session.cycle_key()
    rotate_token(request)
    # セッションに従業員番号と工数区分定義Ver名を保存し、ログイン成功を返す
    request.session['login_No'] = input_number
    request.session['input_def'] = def_Ver.kosu_name
    return JsonResponse({'status': 'success'})
```

▶ ここで初めて出てくるコード

- **`.cycle_key()`**（Django）：`request.session.cycle_key()` は、セッションの中身はそのままで、セッションの番号（クッキーの値）だけを新しくするメソッド。ログインの時に呼ぶと、ログインの前の番号を知っている人が、なりすませなくなります。
- **`rotate_token()`**（Django）：CSRF トークンを、新しい物に変える関数（`django.middleware.csrf` の中にある）。ログインの時に呼ぶと、ログイン前のトークンを知られていても、ログインした後は使えなくなります。Django 標準の `login()` の中でも、同じことをしています。
- **`TypeError`**（Python）：値の種類（型）が合わない時のエラー。`int(None)` のように、数にできない種類の値を渡すと出ます。`except (TypeError, ValueError):` と並べると、まとめて捕まえられます。
- **`AttributeError`**（Python）：その値に無い名前（属性・メソッド）を使った時のエラー。`json.loads()` の結果が辞書ではなくリストだった時に、`data.get(...)` と書くと出ます。

直ったかの確かめ方:

- T-1 のテスト（`TestCsrfFlow` の 3 個）が通ること。
- 手元で動かし（D-4）、ブラウザの開発者ツール（`F12`）→「Application」→「Cookies」に、画面を開いた時点で `csrftoken` があること。ログイン → 工数の保存 → ログアウト → ログイン が全部できること。

✅ 直した写しでは、上の表の 4〜6 が、200・200・200 になりました。

5.2 S-2 確かめ漏れはほぼ無くなった（残り 1 か所）

🟡 余裕のある時に ／ 一部だけ直った ／ 📖 コードから

前の版で「ログインしていなくても使える」と書いた 30 個以上の API は、全部「ログインが要る」になりました（§3）。今の、API と確かめの組み合わせは次のとおりです。

確かめ	使っている API
ログイン不要（`AllowAny`）	`login`・`logout`・`help`・`help_pass`
管理者だけ（`IsAdministrator`）	`manager_*`（管理者 MENU の全部）・`def_list`・`def_new`・`def_update`・`def_delete`・`def_detail_*`・バックアップ・ロード・削除の 19 個・`check_backup_status`・`download_backup`・`web_console_log`
権限が要る（`HasAuthority`）	`member_*`（人員の全部）・`team_menu`・`team_new`・`team_list`・`team_detail`・`team_calendar`・`team_calendar_week_jump`・`team_export`・`team_overtime`・`team_view`
ログインだけ（`IsLoggedIn`。何も書いていない所）	工数・定義の選択（`def_ver`・`def_search`）・問い合わせ・各メニュー・**`team_shop_select`**

残っているのは、`team_shop_select`（班の「ショップ単位の工数入力状況」で表示するショップを選ぶ API）だけです。ここは、使う画面（`team_view`）と同じく「権限が要る」にそろえます。セッションに値を入れるだけなので、急ぎではありません。

場所: `kosu/views/team_views.py` の 769 行目（TeamShopSelect の頭）

今:

コピー

```
class TeamShopSelect(APIView):
```

直した後（上の行を、全部これに置き換える）:

コピー

```
class TeamShopSelect(APIView):
  # 班員の画面（TeamView）と同じく、「権限」がある人だけが使える
  permission_classes = [HasAuthority]
```

▶ ここで初めて出てくるコード

- `HasAuthority`（Django REST framework）：`kosu/permissions.py` にある、権限の確かめの部品。ログインしていて、人員の「権限（`authority`）」が ON の人だけを通します（足りない時は 403）。ビューに `permission_classes = [HasAuthority]` と書いて使います。

5.3 S-3 他人のデータを見たり、変えたり、消したりできる所

🔴 すぐ ／ 一部だけ直った ／ ✅ 手元で再現

何が困るか: 「ログインしている」「権限がある」までは確かめていますが、その先の「**そのデータは、その人が触ってよい物か**」を確かめていない所が残っています。

✅ **手元で試した結果**（`master` の写し。A・B ＝ 一般の人、L ＝「権限」はあるが管理者ではない人）：

番号	試したこと	今の `master`
①	A が、B の工数を、工数の編集画面の API（`GET /api/kosu_update/<id>/`）で開く	200（B の工数が全部見えた）
②	A が、B の工数の就業日を変える（`PUT /api/day_update/`）	200（B の工数の日付が変わった）
③	L が、自分の班でない人の工数を開く（`GET /api/team_detail/<id>/`）	200（見えた）
④	A が、B の問い合わせを書き換える・回答を書く（`PUT /api/inquir_update/<id>/`）	200（書き換わった）
④	A が、B の問い合わせを消す（`DELETE /api/inquir_update/<id>/`）	204（消えた）
⑤	L が、「管理者」付きの人員を登録する（`POST /api/member_new/`）	201（管理者ができた）
⑤	L が、自分を「管理者」にする（`PUT /api/member_update/1501/`）	200（管理者になった）
⑤	L が、管理者の人員を消す（`DELETE /api/member_delete/<番号>/`）	204（消えた）

工数の編集（PUT）・削除と、問い合わせの編集画面（GET）は、すでに本人か確かめていて、404・403 になりました。

直し方: ①〜⑤ を順に直します。どれも「本人（または、その人を扱ってよい人）でなければ断る」を足すだけです。他人のデータの時は、「**無い」時と同じ 404** を返します（他人のデータがあるかどうかも分からないようにするため）。

① 工数の編集画面（`KosuUpdate.get`）

`PUT` には本人の確かめがあるのに、`GET` にはありません。`PUT` と同じ確かめを足します。

場所: `kosu/views/kosu_views.py` の 897〜902 行目（KosuUpdate.get の最初）

今:

コピー

```
# 工数データ取得
kosu_instance = self.get_object(pk)

# 該当する工数データがない場合は 404 を返す
if not kosu_instance:
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
# 工数データ取得
kosu_instance = self.get_object(pk)

# 該当する工数データがない場合、または他人の工数データの場合は 404 を返す
#    （他人の工数データがあるかどうかも分からないよう、「無い」時と同じ返事にする）
if not kosu_instance or kosu_instance.employee_no3 != request.user.employee_no:
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

▶ ここで初めて出てくるコード

- `.user`（Django）: Django の `request.user` は、今ログインしている人（していなければ「だれでもない人」）です。`request.user.is_authenticated` で、ログインしているかが分かります。

② 就業日の変更（`DayUpdate.put`）

今は、送られてきた ID の工数を、だれの物かを見ずに `update_or_create` で書き換えています。`update_or_create` は、その ID が無ければ **新しく作る** ので、無い ID を送ると、だれの物でもない工数データができます。本人の工数を探してから変えるように、`put` を全部置き換えます（日付として読めない値も、500 ではなく 400 にします）。

場所: `kosu/views/kosu_views.py` の 1107〜1135 行目（DayUpdate.put 全体）

今:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
  def put(self, request):
    # 変更後の就業日を取得
    day = request.data.get('work_day2')

    # ログイン者の人員データ
    member_data = request.user

    # 就業日が空なら、セッションの就業日も空にして 400 を返す
    if not day:
      request.session['day'] = ""
      return Response({'status': 'error', 'message': '就業日が未指定で
す。'},status=status.HTTP_400_BAD_REQUEST)
    # 変更後の就業日をセッションに保存する（以降の画面表示をその日付にするため）
    else:
      request.session['day'] = str(day)

    # 工数データ確認（変更先の日付に、ログイン者の工数データがあるか）
    obj_filter = Business_Time_graph.objects.filter(employee_no3=member_data.employee_no, work_day2=day)
    # 変更日に工数データがある場合、エラー（1日1件のルールを守るため）
    if obj_filter.exists():
      return Response({'status': 'error', 'message': '変更日に既に工数データがあります。変更先の工数データ
を削除してから実行してください。'}, status=status.HTTP_400_BAD_REQUEST)

    # 工数データ日付更新
    #   送信された ID の工数データの就業日だけを変更する
    #   （update_or_create は該当IDが無い場合は新規作成しようとする）
    Business_Time_graph.objects.update_or_create(
      id=request.data.get('id'),
      defaults = {'work_day2': day}
      )
    return Response({'status': 'success', 'message': '日付が変更されました。'})
```

▶ ここで初めて出てくるコード

- **`.update_or_create()`**（Django）：条件に合う行があれば書きかえ、無ければ作るメソッド。（物，新しく作ったか）の組が返ります。

直した後（上の行を、全部これに置き換える）：

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
  def put(self, request):
    # 変更後の就業日を取得
    day = request.data.get('work_day2')

    # ログイン者の人員データ
    member_data = request.user

    # 就業日が空、または日付として読めない場合は、セッションの就業日も空にして 400 を返す
    try:
      datetime.date.fromisoformat(str(day))
    except ValueError:
      request.session['day'] = ""
      return Response({'status': 'error', 'message': '就業日が未指定で
す。'},status=status.HTTP_400_BAD_REQUEST)
    # 変更後の就業日をセッションに保存する（以降の画面表示をその日付にするため）
    request.session['day'] = str(day)

    # 就業日を変える工数データを取得する（ログイン者本人の工数データだけが対象）
    #   他人の工数データ・無いIDの場合は 404（update_or_create のように新しく作ることもしない）
    try:
      target = Business_Time_graph.objects.filter(id=int(request.data.get('id')),
employee_no3=member_data.employee_no).first()
    except (TypeError, ValueError):
      target = None
    if target is None:
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'},
status=status.HTTP_404_NOT_FOUND)

    # 工数データ確認（変更先の日付に、ログイン者の工数データがあるか）
    obj_filter = Business_Time_graph.objects.filter(employee_no3=member_data.employee_no, work_day2=day)
    # 変更日に工数データがある場合、エラー（1日1件のルールを守るため）
    if obj_filter.exists():
      return Response({'status': 'error', 'message': '変更日に既に工数データがあります。変更先の工数データ
を削除してから実行してください。'}, status=status.HTTP_400_BAD_REQUEST)

    # 工数データ日付更新（見つけた工数データの就業日だけを変更する）
    target.work_day2 = day
    target.save()
    return Response({'status': 'success', 'message': '日付が変更されました。'})
```

③ 班員の工数の詳細（`TeamDetail.get`）

その工数の人が、ログインした人の班員か（または、ログインした人が管理者か）を確かめます。班員の番号を集める関数 `team_member_numbers` を `team_utils.py` に足して使います（D-12 の繰り返しをまとめる時にも使えます）。

場所: `kosu/utils/team_utils.py` の 22 行目（import）

今:

コピー

```
from ..models import member, Business_Time_graph, kosu_division
```

直した後（上の行を、全部これに置き換える）:

コピー

```
from ..models import member, Business_Time_graph, kosu_division, team_member
```

場所: `kosu/utils/team_utils.py` の 183 行目（ファイルの最後）

今:

コピー

```
  return wb
```

直した後（上の行を、全部これに置き換える）:

コピー

```
  return wb



# 班を管理する人の、班員の従業員番号を返す（TeamDetail で「自分の班員の工数データか」を確かめるのに使う）。
#
# 引数:
#   leader_no: 班を管理する人（班員登録をした人）の従業員番号
# 戻り値:
#   set[int]: 班員1〜15 の従業員番号（空欄は除く。班員データが無ければ空）
def team_member_numbers(leader_no):
  team = team_member.objects.filter(employee_no5=leader_no).first()
  if team is None:
    return set()
  # member1〜member15 は文字列の項目なので、数字だけのものを数値にしてから集める
  numbers = [getattr(team, f'member{i}') for i in range(1, 16)]
  return {int(n) for n in numbers if n and str(n).strip().isdigit()}
```

▶ ここで初めて出てくるコード

- **`.isdigit()`**（Python）: 文字列が、全部数字の文字だけでできていれば `True` を返すメソッド。`str(a).strip().isdigit()` で、「従業員番号として読める文字か」を確かめています。
- **`set()`**（Python）: 集合（同じ値は 1 つしか入らない入れ物）を作る関数。`seen = set()` で空の集合を作り、`seen.add(x)`・`x in seen` で使います。重複を見つける時に便利です。

場所: `kosu/views/team_views.py` の 44 行目（import）

今:

コピー

```
from ..utils.team_utils import excel_function
```

直した後（上の行を、全部これに置き換える）:

コピー

```
from ..utils.team_utils import excel_function, team_member_numbers
```

場所: `kosu/views/team_views.py` の 301〜305 行目（TeamDetail.get の最初）

今:

コピー

```
    # URLで指定されたIDの工数データを取得
    kosu_instance = self.get_object(pk)
    # 該当するデータがない場合は 404（見つからない）を返す
    if not kosu_instance:
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # URLで指定されたIDの工数データを取得
    kosu_instance = self.get_object(pk)
    # 該当するデータがない場合、またはログイン者の班員の工数データでない場合は 404（見つからない）を返す
    #    （管理者はどの工数データでも見られる）
    if not kosu_instance or (not request.user.administrator and
                             kosu_instance.employee_no3 not in team_member_numbers(request.user.employee_no)):
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

④ 問い合わせの編集・削除（`InquirUpdate.put`・`delete`）

> 先に B-7 の①（`get_settings` を足す）を入れてください。

`GET` だけにある「本人か、問い合わせ担当者か」の確かめを関数 `inquiry_permission` にして、`PUT`・`DELETE` でも使います。あわせて、

- 回答（`answer`）を書けるのは、問い合わせ担当者だけにします（画面でも、回答欄は担当者にだけ出しています。`inquiry_views.py` 499 行目のコメント）。
- 問い合わせた人（`employee_no2`・`name`）は、編集で書き換えられないようにします（`InquirSerializer`）。
- 人員から消された人の問い合わせでも、直す・消すができるようにします（B-5）。

場所: `kosu/views/inquiry_views.py` の 34〜39 行目（import と、新しい関数）

今:

コピー

```
from ..models import member, administrator_data, inquiry_data
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import MemberSerializer, InquirSerializer
from ..utils.main_utils import CustomPagination
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from ..models import member, administrator_data, inquiry_data
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import MemberSerializer, InquirSerializer
from ..utils.main_utils import CustomPagination, get_settings



# 問い合わせを見たり直したり消したりしてよい人かを判定する（InquirUpdate の GET / PUT / DELETE で使う）。
#
# 引数:
#   member_data    : ログイン者の人員データ
#   inquir_instance: 問い合わせデータ
#   admin_data     : 管理者設定（問い合わせ担当者の従業員番号1〜3を持つ）
# 戻り値:
#   (bool, bool): (使ってよいか＝本人か担当者, 問い合わせ担当者か)
def inquiry_permission(member_data, inquir_instance, admin_data):
  staff = set()
  for a in [admin_data.administrator_employee_no1, admin_data.administrator_employee_no2, admin_data.administrator_employee_no3]:
    if a and str(a).strip().isdigit():
      staff.add(int(a))
  is_staff = int(member_data.employee_no) in staff
  is_owner = int(member_data.employee_no) == int(inquir_instance.employee_no2)
  return (is_owner or is_staff), is_staff
```

場所: `kosu/views/inquiry_views.py` の 399〜421 行目（InquirUpdate.get の確かめ）

今:

コピー

```
# 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の判定に使う）
#   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
try:
  admin_data = administrator_data.objects.order_by("id").last()
except administrator_data.DoesNotExist:
  return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

# 問い合わせ担当者の従業員番号1〜3（文字列項目。空欄の場合もある）
admin_list_raw = [
  admin_data.administrator_employee_no1,
  admin_data.administrator_employee_no2,
  admin_data.administrator_employee_no3
]
# 空欄や数字以外を除き、数値に変換した担当者従業員番号のリストを作る
#   例: ["123", "", None] → [123]
admin_list = []
for a in admin_list_raw:
  if a and str(a).strip().isdigit():
    admin_list.append(int(a))

# ログイン者が問い合わせ者本人でも問い合わせ担当者でもない場合は 403（権限なし）を返す
if int(member_data.employee_no) != int(inquir_instance.employee_no2) and int(member_data.employee_no) not in admin_list:
  return Response({'status': 'error', 'message': '権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の判定に使う。無ければ初期値で作る）
admin_data = get_settings()

# ログイン者が問い合わせ者本人でも問い合わせ担当者でもない場合は 403（権限なし）を返す
allowed, is_staff = inquiry_permission(member_data, inquir_instance, admin_data)
if not allowed:
  return Response({'status': 'error', 'message': '権限がありません。'}, status=status.HTTP_403_FORBIDDEN)
```

場所: `kosu/views/inquiry_views.py` の 449〜475 行目（InquirUpdate.put の最初）

今:

コピー

```
def put(self, request, pk):
  # ログイン者の人員データ（この処理内では未使用）
  member_data = request.user

  # 更新対象の問い合わせを取得し、なければ 404 を返す
  inquir_instance = self.get_object(pk)
  if not inquir_instance:
    return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

  # 既存データと送信データをシリアライザに渡し、入力チェックの準備をする
  data = request.data
  serializer = InquirSerializer(inquir_instance, data=data)

  # 最新（IDが最大）の管理者設定を取得（担当者向けお知らせの書き込み先）
  #   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
  try:
    admin_data = administrator_data.objects.order_by("id").last()
  except administrator_data.DoesNotExist:
    return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

  # 問い合わせ者の人員データを取得（本人向けお知らせの書き込み先）。なければ 401 を返す
  try:
    inquir_member = member.objects.get(employee_no=inquir_instance.employee_no2)
  except member.DoesNotExist:
    return Response({'status': 'error', 'message': '質問者の人員情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

  if serializer.is_valid():
```

直した後（上の行を、全部これに置き換える）：

コピー

```
def put(self, request, pk):
  # ログイン者の人員データ
  member_data = request.user

  # 更新対象の問い合わせを取得し、なければ 404 を返す
  inquir_instance = self.get_object(pk)
  if not inquir_instance:
    return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

  # 最新（IDが最大）の管理者設定を取得（担当者の判定と、担当者向けお知らせの書き込み先。無ければ初期値で作る）
  admin_data = get_settings()

  # 問い合わせ者本人と問い合わせ担当者だけが直せる（GET と同じ確かめ）
  allowed, is_staff = inquiry_permission(member_data, inquir_instance, admin_data)
  if not allowed:
    return Response({'status': 'error', 'message': '権限がありません。'}, status=status.HTTP_403_FORBIDDEN)

  # 既存データと送信データをシリアライザに渡し、入力チェックの準備をする
  data = request.data
  serializer = InquirSerializer(inquir_instance, data=data)

  # 問い合わせ者の人員データを取得（本人向けお知らせの書き込み先）。人員から消されていれば None
  inquir_member = member.objects.filter(employee_no=inquir_instance.employee_no2).first()

  if serializer.is_valid():
    # 回答を書けるのは問い合わせ担当者だけ（画面でも回答欄は担当者にだけ出している）
    new_answer = serializer.validated_data.get('answer', inquir_instance.answer)
    if not is_staff and (new_answer or '') != (inquir_instance.answer or ''):
      return Response({'status': 'error', 'message': '回答は問い合わせ担当者だけが書けます。'}, status=status.HTTP_403_FORBIDDEN)
```

▶ ここで初めて出てくるコード

- **`.validated_data`**（Django REST framework）: DRF の `serializer.validated_data` は、確かめ（`is_valid()`）を通った後の、安全な値です。

場所: `kosu/views/inquiry_views.py` の 500 行目（回答のお知らせ）

今:

コピー

```
        if 'answer' in changed_fields:
```

直した後（上の行を、全部これに置き換える）:

コピー

```
        if 'answer' in changed_fields and inquir_member is not None:
```

場所: `kosu/views/inquiry_views.py` の 529〜571 行目（InquirUpdate.delete の前半）

今:

コピー

```
def delete(self, request, pk):
  # 削除対象の問い合わせを取得し、なければ 404 を返す
  inquir_instance = self.get_object(pk)
  if inquir_instance is None:
    return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

  # 問い合わせ者の人員データを取得（本人向けお知らせの削除先）。なければ 401 を返す
  try:
    inquir_member = member.objects.get(employee_no=inquir_instance.employee_no2)
  except member.DoesNotExist:
    return Response({'status': 'error', 'message': '質問者の人員情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

  # 最新（IDが最大）の管理者設定を取得（担当者向けお知らせの削除先）
  #   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
  try:
    admin_data = administrator_data.objects.order_by("id").last()
  except administrator_data.DoesNotExist:
    return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)


  # 本人向けお知らせの削除: この問い合わせID と一致する枠をすべて空にする（まだ保存しない）
  for i in range(1, 6):
    pop_up_id_attr = f'pop_up_id{i}'
    pop_up_attr = f'pop_up{i}'
    if getattr(inquir_member, pop_up_id_attr) == str(pk):
      setattr(inquir_member, pop_up_id_attr, '')
      setattr(inquir_member, pop_up_attr, '')

  # 本人向けお知らせの前詰め: 文言が空の枠があれば次の枠の内容を移し、次の枠を空にする（まだ保存しない）
  for i in range(1, 5):
    pop_up_attr = f'pop_up{i}'
    pop_up_id_attr = f'pop_up_id{i}'
    next_pop_up_attr = f'pop_up{i + 1}'
    next_pop_up_id_attr = f'pop_up_id{i + 1}'

    if getattr(inquir_member, pop_up_attr) in ['', None]:
      setattr(inquir_member, pop_up_attr, getattr(inquir_member, next_pop_up_attr))
      setattr(inquir_member, pop_up_id_attr, getattr(inquir_member, next_pop_up_id_attr))
      setattr(inquir_member, next_pop_up_attr, '')
      setattr(inquir_member, next_pop_up_id_attr, '')

  # 本人向けお知らせの変更を最後に一度だけ保存
  inquir_member.save()
```

▶ ここで初めて出てくるコード

- **`setattr()`**（Python）：`setattr(物, '名前', 値)` で、名前を文字で指定して、その物の属性に値を入れる関数。`setattr(inquir_member, 'pop_up1', '')` は `inquir_member.pop_up1 = ''` と同じです。名前を `f'pop_up{i}'` のように作れるので、`pop_up1` 〜 `pop_up5` のような似た名前の項目を、ループで扱えます。

直した後（上の行を、全部これに置き換える）：

コピー

```
def delete(self, request, pk):
  # 削除対象の問い合わせを取得し、なければ 404 を返す
  inquir_instance = self.get_object(pk)
  if inquir_instance is None:
    return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

  # 最新（IDが最大）の管理者設定を取得（担当者の判定と、担当者向けお知らせの削除先。無ければ初期値で作る）
  admin_data = get_settings()

  # 問い合わせ者本人と問い合わせ担当者だけが消せる（GET と同じ確かめ）
  allowed, is_staff = inquiry_permission(request.user, inquir_instance, admin_data)
  if not allowed:
    return Response({'status': 'error', 'message': '権限がありません。'}, status=status.HTTP_403_FORBIDDEN)

  # 問い合わせ者の人員データを取得（本人向けお知らせの削除先）。人員から消されていれば、本人向けの片付けは飛ばす
  inquir_member = member.objects.filter(employee_no=inquir_instance.employee_no2).first()
  if inquir_member is not None:
    # 本人向けお知らせの削除: この問い合わせID と一致する枠をすべて空にする（まだ保存しない）
    for i in range(1, 6):
      pop_up_id_attr = f'pop_up_id{i}'
      pop_up_attr = f'pop_up{i}'
      if getattr(inquir_member, pop_up_id_attr) == str(pk):
        setattr(inquir_member, pop_up_id_attr, '')
        setattr(inquir_member, pop_up_attr, '')

    # 本人向けお知らせの前詰め: 文言が空の枠があれば次の枠の内容を移し、次の枠を空にする（まだ保存しない）
    for i in range(1, 5):
      pop_up_attr = f'pop_up{i}'
      pop_up_id_attr = f'pop_up_id{i}'
      next_pop_up_attr = f'pop_up{i + 1}'
      next_pop_up_id_attr = f'pop_up_id{i + 1}'

      if getattr(inquir_member, pop_up_attr) in ['', None]:
        setattr(inquir_member, pop_up_attr, getattr(inquir_member, next_pop_up_attr))
        setattr(inquir_member, pop_up_id_attr, getattr(inquir_member, next_pop_up_id_attr))
        setattr(inquir_member, next_pop_up_attr, '')
        setattr(inquir_member, next_pop_up_id_attr, '')

    # 本人向けお知らせの変更を最後に一度だけ保存
    inquir_member.save()
```

場所: `kosu/views/serializers.py` の 117〜121 行目（InquirSerializer）

今:

コピー

```
class InquirSerializer(serializers.ModelSerializer):
  # 変換対象のモデルと項目の指定
  class Meta:
    model = inquiry_data  # 対象モデル: 問い合わせ
    fields = '__all__'  # 全項目を対象にする
```

直した後（上の行を、全部これに置き換える）：

コピー

```
class InquirSerializer(serializers.ModelSerializer):
  # 変換対象のモデルと項目の指定
  class Meta:
    model = inquiry_data  # 対象モデル: 問い合わせ
    fields = '__all__'  # 全項目を対象にする
    # 問い合わせた人（従業員番号・氏名）は、編集の時に書き換えさせない（返すだけ）
    read_only_fields = ['employee_no2', 'name']
```

⑤ 人員の登録・編集・削除（`MemberNew`・`MemberUpdate`・`MemberDelete`）

人員の画面は「権限」がある人なら使えますが、「**管理者**」を付けたり外したり、管理者の人員を直したり消したりするのは、管理者だけにします。`MemberUpdate` の直しには、従業員番号を変えた時に、その人の工数・問い合わせ・班のデータも新しい番号にそろえる直し（B-19）も入っています。

場所: `kosu/views/member_views.py` の 21〜28 行目（import）

今:

コピー

```
from django.db.models import Case, When, Value, IntegerField
from ..models import member
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import MemberSerializer
from ..utils.main_utils import CustomPagination
from ..permissions import HasAuthority, IsAdministrator
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from django.db import transaction  # 複数の書き込みを「全部する / 全部しない」にまとめる
from django.db.models import Case, When, Value, IntegerField
from ..models import member, Business_Time_graph, inquiry_data, team_member
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .serializers import MemberSerializer
from ..utils.main_utils import CustomPagination
from ..permissions import HasAuthority, IsAdministrator
```

場所: `kosu/views/member_views.py` の 139〜141 行目（MemberNew.post の確かめ）

今:

コピー

```
    # 同じ従業員番号の人員がすでに存在する場合は登録不可
    if member.objects.filter(employee_no=data.get('employee_no')).exists():
      return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されています。'}, status=status.HTTP_400_BAD_REQUEST)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 同じ従業員番号の人員がすでに存在する場合は登録不可
    if member.objects.filter(employee_no=data.get('employee_no')).exists():
      return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されています。'}, status=status.HTTP_400_BAD_REQUEST)

    # 「管理者」を付けて登録できるのは管理者だけ（「権限」だけの人が管理者を作れないようにする）
    if data.get('administrator') and not request.user.administrator:
      return Response({'status': 'error', 'message': '管理者は、管理者だけが登録できます。'}, status=status.HTTP_403_FORBIDDEN)
```

場所: `kosu/views/member_views.py` の 279〜289 行目（MemberUpdate.put の中ほど）

今:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →

コピー

```
    # 既存データと送信データをシリアライザに渡し、入力チェックの準備をする
    data = request.data
    serializer = MemberSerializer(member_instance, data=data)
    if serializer.is_valid():
      # 従業員番号の一意性確認:
      #   送信された従業員番号が変更前（pk）と異なり、かつ他の人員がその番号を使っている場合は更新不可
      if data.get('employee_no') != pk and
member.objects.filter(employee_no=data.get('employee_no')).exists():
        return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されていま
す。'},status=status.HTTP_400_BAD_REQUEST)
      # 問題なければデータベースに保存し、更新後の内容を 200（成功）で返す
      serializer.save()
      return Response(serializer.data, status=status.HTTP_200_OK)
```

直した後（上の行を、全部これに置き換える）:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
    # 管理者の人員データを直せるのは管理者だけ
    if member_instance.administrator and not request.user.administrator:
      return Response({'status': 'error', 'message': '管理者の人員データは、管理者だけが直せます。'},
status=status.HTTP_403_FORBIDDEN)

    # 既存データと送信データをシリアライザに渡し、入力チェックの準備をする
    data = request.data
    serializer = MemberSerializer(member_instance, data=data)
    if serializer.is_valid():
      # 「管理者」を付けられるのは管理者だけ（自分を管理者にする、などを防ぐ）
      if serializer.validated_data.get('administrator') and not request.user.administrator:
        return Response({'status': 'error', 'message': '管理者は、管理者だけが付けられます。'},
status=status.HTTP_403_FORBIDDEN)
      # 従業員番号の一意性確認:
      #   送信された従業員番号が変更前（pk）と異なり、かつ他の人員がその番号を使っている場合は更新不可
      if data.get('employee_no') != pk and
member.objects.filter(employee_no=data.get('employee_no')).exists():
        return Response({'status': 'error', 'message': '入力した従業員番号はすでに登録されていま
す。'},status=status.HTTP_400_BAD_REQUEST)
      # データベースに保存する。従業員番号を変えた時は、その番号で持っている工数・問い合わせ・班のデータも
      # 新しい番号にそろえる（そろえないと、その人の工数などが見えなくなる）。途中で失敗したら全部元に戻す
      old_no = member_instance.employee_no
      with transaction.atomic():
        serializer.save()
        new_no = member_instance.employee_no
        if new_no != old_no:
          Business_Time_graph.objects.filter(employee_no3=old_no).update(employee_no3=new_no)
          inquiry_data.objects.filter(employee_no2=old_no).update(employee_no2=new_no)
          team_member.objects.filter(employee_no5=old_no).update(employee_no5=new_no)
          # 班員1〜15 は文字列の項目なので、文字列で探して文字列で書き換える
          for i in range(1, 16):
            team_member.objects.filter(**{f'member{i}': str(old_no)}).update(**{f'member{i}': str(new_no)})
      return Response(serializer.data, status=status.HTTP_200_OK)
```

▶ ここで初めて出てくるコード

- `.atomic()`（Django）：`with transaction.atomic():` の中のデータベースの変更を、「全部成功か、全部取り消し」にする書き方。

場所: `kosu/views/member_views.py` の 328〜330 行目（MemberDelete.delete の確かめ）

今:

コピー

```
    # 該当する人員がいない場合は 404 を返す
    if member_instance is None:
      return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 該当する人員がいない場合は 404 を返す
if member_instance is None:
  return Response({'status': 'error', 'message': 'レコードが見つかりません。'}, status=status.HTTP_404_NOT_FOUND)

# 管理者の人員データを消せるのは管理者だけ
if member_instance.administrator and not request.user.administrator:
  return Response({'status': 'error', 'message': '管理者の人員データは、管理者だけが消せます。'}, status=status.HTTP_403_FORBIDDEN)
```

直ったかの確かめ方: T-2 のテストのうち `TestOthersData` の 6 個が通ること。✅ 直した写しでは、上の表の ①〜③ が 404、④ が 403（書き換わらない・消えない）、⑤ が 403（管理者は増えない・消えない）になりました。

5.4 S-4 ダウンロードの API が、渡された場所のファイルを、そのまま返して消そうとする

🔴 すぐ ／ 一部だけ直った（管理者だけになった） ／ ✅ 手元で再現

どこ: `kosu/views/asynchronous_views.py` 589〜645 行目（`download_file`）、539〜546 行目（`check_task_status`）

今:

- `file_path` に渡された文字を、そのまま `open()` で開いて返します（595 行目。コメントにも「パスの妥当性チェックは行わずにそのまま開く」とある）。バックアップの保存場所（`media` フォルダ）の外のファイルでも返します。
- 返した 3 秒後に、そのファイルを **消そうとします**（610〜622 行目）。
- `response.close` を自分の関数に差し替えているので、元の `close()`（開いたファイルを閉じる）が呼ばれません（642 行目のコメントにもある）。ファイルが開いたままになります。
- `check_task_status` が、サーバーの中のファイルの場所（フォルダの名前まで）を、そのまま画面に返しています（544 行目）。
- `file_path` を付けずに呼ぶと、`open(None)` で 500 になります。

✅ **手元で試した結果:** 管理者で、`media` の外に作ったファイルの場所を渡すと、200 で中身が返り、3 秒後に消そうとしました（Windows の手元では、ファイルが開いたままのため消せず、ログに「Cleanup failed … 別のプロセスが使用中です」と出た。本番の Linux では、開いたままでも消えます）。`file_path` 無しは 500 でした。

何が困るか: 管理者のアカウントを使えば（または、管理者に変な URL を開かせれば）、サーバーの中の設定ファイルなどを読まれ、消されます。ダウンロードは GET で動くので、セッションの Cookie が別のサイトからの読み込みにも付く今の設定（S-5）では、管理者が別のサイトを開いただけで呼ばれるおそれもあります。

直し方:

① `Path` を使えるようにします。

場所: `kosu/views/asynchronous_views.py` の 39 行目の前

足すコード（この行の前に入れる）：

コピー

```
from pathlib import Path  # ファイルの場所を、フォルダの外に出ていないか確かめられる形で扱う
```

② `download_file` を全部置き換えます。受け取るのは **ファイル名だけ** にし、`media` フォルダの直下にあるファイルしか開きません。送り終わったら、元の `close()` でファイルを閉じてから消します（待つための別スレッドも要らなくなります）。

場所: `kosu/views/asynchronous_views.py` の 589〜645 行目（download_file 全体）

今:

コピー

```
@api_view(['GET'])  # GET のみ受け付ける DRF のAPIにする
@permission_classes([IsAdministrator])  # 管理者のみ利用可能
def download_file(request):
    # ダウンロード対象のファイルパスをクエリパラメータから取得する
    file_path = request.GET.get('file_path')
    # ファイルをバイナリの読み込み専用モード（'rb'）で開く（パスの妥当性チェックは行わずにそのまま開く）
    file_handle = open(file_path, 'rb')
    # ファイルをダウンロードさせるレスポンスを作成する
    #   as_attachment=True: ブラウザで表示せず、ファイルとして保存させる
    #   filename          : 保存時のファイル名（パスからファイル名の部分だけを取り出す）
    response = FileResponse(
        file_handle,
        as_attachment=True,
        filename=os.path.basename(file_path),
    )
    # ダウンロード後にサーバー上のファイルを削除する（下の cleanup_on_close から別スレッドで呼ばれる）。
    #
    # 引数:
    #   なし（外側の file_path を使う）
    # 戻り値:
    #   なし
    def delayed_file_cleanup():
        # ファイルがまだ送信中・使用中の可能性があるため、3秒待ってから削除する
        time.sleep(3)
        # ファイルが存在する場合だけ削除する
        if os.path.exists(file_path):
            try:
                os.remove(file_path)
            # 削除に失敗してもダウンロード自体には影響しないため、標準出力にログを出すだけにする
            except Exception as e:
                print(
                    f'Cleanup failed after delay for '
                    f'{file_path}: {e}'
                )
    # レスポンスの送信が終わったとき（close() 呼び出し時）に実行する処理。
    #
    # 3秒の待機中にサーバーの処理を止めないよう、削除処理を別スレッドで開始する。
    #
    # 引数:
    #   なし
    # 戻り値:
    #   なし
    def cleanup_on_close():
        # ファイル削除処理を別スレッドで実行する準備
        thread = threading.Thread(
            target=delayed_file_cleanup,
        )
        # デーモンスレッドにする（サーバー終了時にスレッドが残り続けないようにする）
        thread.daemon = True
        # ファイル削除処理を開始する
        thread.start()
    # レスポンスの close() を上の cleanup_on_close に差し替える
    #   Django はレスポンスの送信完了後に close() を呼ぶため、そのタイミングで削除処理が始まる
    #   ※差し替えにより、元の close() の処理（開いたファイルを閉じる等）は実行されない
    response.close = cleanup_on_close
    # ファイルのダウンロードレスポンスを返す
    return response
```

▶ ここで初めて出てくるコード

- **`permission_classes()`**（Django REST framework）：`from rest_framework.decorators import permission_classes` で読み込む、関数の API に確かめを付ける道具。`@permission_classes([…])` の形で使います。クラスの API では、`permission_classes = [...]` という変数に書きます。
- **`@permission_classes`**（Django REST framework）：`@api_view` で書いた関数の API に、「だれが使えるか」の確かめを付ける印。`@permission_classes([IsAdministrator])` で、管理者だけにします。`@api_view` のすぐ下に書きます。

直した後（上の行を、全部これに置き換える）：

コピー

```
@api_view(['GET'])  # GET のみ受け付ける DRF のAPIにする
@permission_classes([IsAdministrator])  # 管理者のみ利用可能
def download_file(request):
    # バックアップの保存先（media フォルダ）の絶対パス
    backup_dir = Path(settings.MEDIA_ROOT).resolve()
    # 受け取るのはファイル名だけにする（フォルダの部分は捨てる。例: "../../.env" → ".env"）
    file_name = os.path.basename(request.GET.get('file_path') or '')
    file_path = (backup_dir / file_name).resolve()
    # 名前が空・media フォルダの直下でない・ファイルが無い場合は 404（ほかの場所のファイルは決して開かない）
    if not file_name or file_path.parent != backup_dir or not file_path.is_file():
        return JsonResponse(
            {
                'status': 'error',
                'message': 'ファイルが見つかりません。',
            },
            status=status.HTTP_404_NOT_FOUND,
        )
    # ファイルをダウンロードさせるレスポンスを作成する
    #   as_attachment=True: ブラウザで表示せず、ファイルとして保存させる
    response = FileResponse(
        open(file_path, 'rb'),
        as_attachment=True,
        filename=file_name,
    )
    # 送り終わった時（Django が close() を呼ぶ時）に、まず元の close()（開いたファイルを閉じる）を呼び、
    # そのあとファイルを消す（閉じてから消すので、Windows でも消せる。待つための別スレッドも要らない）
    original_close = response.close

    def close_then_delete():
        original_close()
        try:
            file_path.unlink()
        except OSError:
            pass  # 消せなくてもダウンロードには影響しない

    response.close = close_then_delete
    return response
```

▶ ここで初めて出てくるコード

- `.is_file()`（Python）：`Path` の `.is_file()` は、その場所にファイルがあれば `True` を返すメソッド。フォルダや、何も無い時は `False` です。
- `.unlink()`（Python）：`Path` の `.unlink()` は、そのファイルを消すメソッド。消せない時は `OSError` になります。
- `original_close()`（Python）：この巻のダウンロードの直し方で、`FileResponse` が元から持っていた `close`（開いたファイルを閉じる処理）を、変数 `original_close` に取っておいた物。先にこれを呼んでファイルを閉じてから、ファイルを消します。
- `OSError`（Python）：ファイルを開く・消すなど、OS とのやり取りで起きるエラーのまとめ役。`except OSError:` で、ファイルが無い・使用中で消せない、などをまとめて受け止めます。

③ `check_task_status` は、ファイル名だけを返します。

場所: `kosu/views/asynchronous_views.py` の 539〜546 行目（check_task_status の成功の所）

今:

コピー

```
        # タスクが正常終了している場合: 結果（ファイルパス等）を file_path として返す
        if task.status == 'success':
            return JsonResponse(
                {
                    'status': 'success',
                    'file_path': task.result,
                }
            )
```

直した後（上の行を、全部これに置き換える）:

コピー

```
        # タスクが正常終了している場合: 結果のファイル名だけを file_path として返す
        #   （サーバーの中のフォルダの場所は画面に知らせない。ダウンロードAPIもファイル名だけで探す）
        if task.status == 'success':
            return JsonResponse(
                {
                    'status': 'success',
                    'file_path': os.path.basename(task.result) if task.result else None,
                }
            )
```

④ 画面のダウンロードの URL を、相対の URL にします（B-2 の直しと同じ物です）。

場所: `frontend/src/AdministratorPage/AdministratorLoading.tsx` の 117〜119 行目（handleSuccess の中）

今:

コピー

```
    // GET /api/download_backup?file_path=... : バックアップファイルをダウンロード
    //   （axios を使わず、ブラウザ標準のリンク遷移でダウンロードさせるため、APIのURLを直接組み立てる）
    const endpoint = `${process.env.REACT_APP_API_BASE_URL}/api/download_backup`;
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    // GET /api/download_backup?file_path=... : バックアップファイルをダウンロード
    //   画面と同じサーバーへの相対URLにする（Vite のビルドでは process.env が {} に置き換わり、
    //   process.env.REACT_APP_API_BASE_URL は undefined になるため、使わない）
    const endpoint = "/api/download_backup";
```

直ったかの確かめ方: T-2 の `TestDownload` の 2 個と、T-4 で直す `test_download_file`・`test_check_task_status_success` が通ること。✅ 直した写しでは、`media` の外のファイルは 404（ファイルは消えない）、`file_path` 無しも 404、`media` の中のファイルは 200 で返り、送り終わると消えました。手元で、データ管理画面の「バックアップ開始」→ Excel がダウンロードされ、Excel で開けることも確かめてください。

5.5 S-5 セッションの Cookie が、別のサイトからの頼みにも付く

🔴 すぐ ／ 一部だけ直った（CORS は直った） ／ 📖 コードから

どこ: `hozen_another/settings.py` 164〜167 行目

今: `SESSION_COOKIE_SAMESITE = 'None'` です。ブラウザは、別のサイトのページの中からこのアプリへ送られる頼み（画像の読み込み・フォームの送信など）にも、セッションの Cookie を付けます。CORS（前の版の S-5 の前半）は直ったので、別のサイトがこのアプリの返事を **読む** ことはできなくなりましたが、**送らせる** ことはできます。書き込みは CSRF の確かめで止まります（S-1 を直した後）が、GET で何かが変わる API（S-4 のダウンロード）や、これから CSRF の確かめを外してしまった時に、守りがありません。

直し方: `'Lax'` にします。画面と API は同じサーバーから出しているので、`Lax` で困ることはありません（`'None'` が要るのは、画面と API を別々のサイトに置く時だけ）。

場所: `hozen_another/settings.py` の 164〜167 行目（セッション Cookie）

今:

コピー

```
# 別サイト（別オリジン）からのリクエストでもセッションCookieを送らせる設定
#   SameSite=None にする場合、ブラウザの仕様上 Secure（HTTPS 通信のときだけ送る）が必須
SESSION_COOKIE_SAMESITE = 'None'
SESSION_COOKIE_SECURE = True
```

直した後（上の行を、全部これに置き換える）:

コピー

```
# セッションCookieは、別のサイトからのリクエストには付けさせない（Lax）
#   画面と API は同じサーバー（同じオリジン）から出しているので、Lax にしても困ることはない。
#   別のサイトに置いた罠のページから、ログイン中の人として操作されるのを防ぐ
SESSION_COOKIE_SAMESITE = 'Lax'
# HTTPS 通信のときだけセッションCookieを送る（手元の http://localhost でも、ブラウザは送る）
SESSION_COOKIE_SECURE = True
```

直ったかの確かめ方: 手元で動かし、開発者ツールの「Application」→「Cookies」で、`sessionid` の「SameSite」が `Lax` になっていること。ログイン・工数の保存ができること。

5.6 S-6 ログインにパスワードが無い（従業員番号だけ）

🔴 すぐ（ただし、やり方は決める必要がある） ／ 前から残っている ／ 📖 コードから

どこ: `kosu/views/main_views.py` 182〜222 行目（ `Login` ）、`hozen_another/settings.py` 160〜162 行目（ `SESSION_COOKIE_AGE = 315360000` ＝ 10 年）、`main_views.py` 295〜325 行目（ヘルプの初期化で作る従業員番号 `12345` の管理者）

今: 人員の表にある従業員番号を入れれば、その人としてログインできます（000 巻 §2.2）。管理者の番号を知っていれば、管理者として入れます。一度ログインすると、10 年間ログインしたままです。

何が困るか: 従業員番号は、名札・書類・メールなどで、ほかの人も知っている物です。共用の PC では、前の人のログインがそのまま残ります。

直し方（どれにするかは、前任者・会社の IT 担当と決める）:

案	中身	手間
A（おすすめ）	**会社の Microsoft アカウント（Entra ID）でサインインする。** Azure App Service の「認証」という機能を有効にすると、アプリの手前で Microsoft のサインインを求めるようになる。アプリは、サインインした人の名前（メールアドレスなど）を、`X-MS-CLIENT-PRINCIPAL-NAME` という見出しで受け取れるので、人員の表にその列を足して結び付ける	会社の Entra ID の管理者の協力が要る。アプリの直しは中くらい
B	**パスワードを足す。** 人員の表に `password` の列を足し（Django の `make_password` で、元に戻せない形にして保存）、ログインで `check_password` で比べる。初めのパスワードの配り方と、変える画面を作る	中くらい。migrate が要る
C（すぐできる最低限）	ログインのたびにセッションの番号を新しくする（S-1 の④に入れた）。セッションを短くする（下）。ヘルプの初期化で作る `12345` を、すぐ消す決まりにする	小さい

C は、A・B のどちらにするにしても先にやっておけます。セッションの長さは、勤務の長さに合わせて、使う人と決めてください（例は 12 時間。毎日ログインし直すことになります）。

場所: `hozen_another/settings.py` の 160〜162 行目（セッションの有効期間）

今:

コピー

```
# セッションCookieの有効期間（秒）
#   315,360,000秒 = 3,650日（約10年）。実質的に「ログアウトするまでログイン状態を保つ」設定
SESSION_COOKIE_AGE = 315360000
```

直した後（上の行を、全部これに置き換える）:

コピー

```
# セッションCookieの有効期間（秒）
#   60秒 × 60分 × 12時間 = 12時間。勤務が終わる頃には切れ、次の勤務でログインし直す
#   （前は 10 年＝ログアウトしない限りずっとログインしたままで、共用の PC では他人に使われやすかった）
SESSION_COOKIE_AGE = 60 * 60 * 12
```

5.7 S-7 ヘルプ画面の初期化 API（総当たり・途中の失敗・パスワード未設定）

🔴 すぐ ／ 一部だけ直った（パスワードを確かめるようになった） ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 235〜245 行目（`is_valid_help_password`）・257〜356 行目（`Help`）・366〜381 行目（`HelpPass`）

今:

- ヘルプの API は、ログインしていなくても呼べます（ログインできなくなった時に使うため）。パスワードを **何回でも** 試せます。
- `password == env('HELP_PATH')` で比べています（245 行目）。比べるのにかかる時間が、合っている文字数で変わる比べ方です。
- `HELP_PATH` が設定されていないと、`env('HELP_PATH')` がエラーを出して 500 になります。
- 呼ばれるたびに `kosu/.env`（無いファイル）を読みに行きます（239〜243 行目）。
- 人員・定義・設定を消して作り直す途中で失敗すると、それまでに消した分は消えたままになります（トランザクションが無い）。

✅ **手元で試した結果:** 違うパスワードを 10 回続けて送っても、10 回とも 403 で、止められませんでした。`HELP_PATH` を空にすると 500 になりました。

直し方:

① パスワードの比べ方を直します（時間が中身で変わらない `hmac.compare_digest`。未設定・文字以外は必ず不一致にする）。

場所: `kosu/views/main_views.py` の 235〜245 行目（is_valid_help_password）

今:

コピー

```
def is_valid_help_password(password):
  # このファイルの2つ上のフォルダを取得
  #   __file__ = kosu/views/main_views.py → parent = kosu/views → parent.parent = kosu/
  #   ※プロジェクトのルートではなく kosu/ フォルダを指す
  BASE_DIR = Path(__file__).resolve().parent.parent
  # 環境変数ファイルを読み込む（kosu/.env が対象）
  #   kosu/.env が無い場合も、settings.py がルートの .env から読み込んだ環境変数を env() で参照できる
  env = environ.Env()
  env.read_env(os.path.join(BASE_DIR, '.env'))
  # 入力値と環境変数 HELP_PATH を比較する（未送信の None は必ず不一致になる）
  return password == env('HELP_PATH')
```

直した後（上の行を、全部これに置き換える）：

コピー

```
def is_valid_help_password(password):
  # ヘルプ用パスワード（.env または Azure の構成の HELP_PATH。settings.py が起動時に読み込み済み）
  expected = os.environ.get('HELP_PATH', '')
  # パスワードが設定されていない・文字列が送られていない場合は、必ず不一致にする（500 にしない）
  if not expected or not isinstance(password, str):
    return False
  # 比べるのにかかる時間が中身によって変わらない比べ方（hmac.compare_digest）を使う。
  # 日本語のパスワードも比べられるよう、両方をバイト列（UTF-8）にしてから比べる
  return hmac.compare_digest(password.encode('utf-8'), expected.encode('utf-8'))
```

> **▶ ここで初めて出てくるコード**
>
> - `.compare_digest()`（Python）：`hmac.compare_digest(a, b)` は、2 つの値が同じかを、かかる時間から中身を推測されない方法で比べる関数です。合言葉やハッシュを比べる時に使います。
> - `.encode()`（Python）：文字列を、バイトの並びにするメソッド（UTF-8）。`token.encode()` のように、ハッシュの計算など、バイトを求める関数に渡す前に使います。

② `Help`・`HelpPass` に、呼び出しの回数の上限を付けます（同じ所から 1 時間に 5 回まで。超えると 429）。上限は `settings.py` に書きます。

場所: `kosu/views/main_views.py` の 257〜259 行目（Help の頭）

今:

コピー

```
class Help(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
```

直した後（上の行を、全部これに置き換える）：

コピー

```
class Help(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
  # パスワードの総当たりを防ぐため、同じ所からの呼び出し回数を制限する（settings.py の DEFAULT_THROTTLE_RATES の 'help'）
  throttle_classes = [ScopedRateThrottle]
  throttle_scope = 'help'
```

場所: `kosu/views/main_views.py` の 366〜368 行目（HelpPass の頭）

今:

コピー

```
class HelpPass(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
```

直した後（上の行を、全部これに置き換える）:

コピー

```
class HelpPass(APIView):
  # ログインしていなくても呼び出せるようにする
  permission_classes = [AllowAny]
  # パスワードの総当たりを防ぐため、Help と同じ枠（'help'）で呼び出し回数を制限する
  throttle_classes = [ScopedRateThrottle]
  throttle_scope = 'help'
```

場所: `hozen_another/settings.py` の 148〜151 行目（REST_FRAMEWORK の最後）

今:

コピー

```
    'DEFAULT_AUTHENTICATION_CLASSES': ['kosu.permissions.LoginNoAuthentication'],  # だれがログインしているか
    'DEFAULT_PERMISSION_CLASSES': ['kosu.permissions.IsLoggedIn'],                 # 最初から全部の API でログインが要る
    'UNAUTHENTICATED_USER': None,                                                  # ログインしていない時の request.user は None
}
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    'DEFAULT_AUTHENTICATION_CLASSES': ['kosu.permissions.LoginNoAuthentication'],  # だれがログインしているか
    'DEFAULT_PERMISSION_CLASSES': ['kosu.permissions.IsLoggedIn'],                 # 最初から全部の API でログインが要る
    'UNAUTHENTICATED_USER': None,                                                  # ログインしていない時の request.user は None
    # 呼び出し回数の上限（throttle_scope を付けた API だけにかかる。1 時間に 5 回まで）
    'DEFAULT_THROTTLE_RATES': {
        'help': '5/hour',
    },
}
```

③ テストの中では、上限をかけないようにします（同じテストの中で何度もヘルプを呼ぶため）。

場所: `hozen_another/test_settings.py` の 59 行目（ファイルの最後）

今:

コピー

```
MIGRATION_MODULES = DisableMigrations()
```

直した後（上の行を、全部これに置き換える）:

コピー

```
MIGRATION_MODULES = DisableMigrations()
# テストでは呼び出し回数の制限をかけない（同じテストの中で何度もヘルプを呼ぶため）
#   制限そのもののテストは、test_security.py で上限を書き換えて確かめる
REST_FRAMEWORK = {**REST_FRAMEWORK, 'DEFAULT_THROTTLE_RATES': {'help': None}}
```

④ 初期化を「全部する / 全部しない」にします。`@transaction.atomic` を付け、例外を捕まえて 400 を返す所では、取り消しを明示します。

場所: `kosu/views/main_views.py` の 274 行目（Help.post の頭）

今:

コピー

```
  def post(self, request):
```

直した後（上の行を、全部これに置き換える）:

コピー

```
  @transaction.atomic  # 途中で失敗したら、それまでに消したデータも元に戻す
  def post(self, request):
```

> **▶ ここで初めて出てくるコード**
>
> - **`@transaction`**（Django）: `@transaction.atomic` を関数の上に付けると、その関数の中のデータベースの変更が「全部成功か、全部取り消し」になります。

場所: `kosu/views/main_views.py` の 354〜356 行目（Help.post の最後）

今:

コピー

```
    # 削除・作成中に何らかの例外が発生した場合は内容を返さず 400 を返す
    except Exception as e:
      return Response({'status': 'error', 'message': 'データの初期化中にエラーが発生しました。'}, status=status.HTTP_400_BAD_REQUEST)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 削除・作成中に何らかの例外が発生した場合は内容を返さず 400 を返す
    except Exception as e:
      # 例外をここで捕まえて普通に返すので、transaction.atomic は自動では取り消さない。取り消しを明示する
      transaction.set_rollback(True)
      return Response({'status': 'error', 'message': 'データの初期化中にエラーが発生しました。'}, status=status.HTTP_400_BAD_REQUEST)
```

> **▶ ここで初めて出てくるコード**
>
> - `.set_rollback()`（Django）：`transaction.set_rollback(True)` は、今いる `transaction.atomic` の中の書き込みを、終わる時に全部取り消すよう印を付ける関数。`except` で例外を捕まえて普通に返すと、`atomic` は「うまくいった」とみなして確定してしまうので、取り消したい時はこれを呼びます。

直ったかの確かめ方: T-2 の `TestHelp` の 2 個が通ること。✅ 直した写しでは、違うパスワードの 6 回目が 429、`HELP_PATH` が空でも 403、正しいパスワードは 200 でした。

> 本番の `HELP_PATH` は、だれが知っているかを前任者に確かめ、引き継いだら変えてください（000 巻 §10）。

5.8 S-8 サーバーのログ（直った）

直った ／ ✅ 手元で確かめた

サーバーのログの API（`/api/web_console_log/`）は、管理者だけが読めるようになりました（`main_views.py` 1331 行目）。✅ 一般の人は 403、ログインしていないと 401 でした。ログのファイルの場所のずれは D-9、ログのファイルがリポジトリに入っていることは D-3 に書きました。

5.9 S-9 リポジトリに残っている Excel

🔴 すぐ ／ 一部だけ直った ／ 📖 コードから（ファイルの見出しと行数だけ確かめた）

今:

- 前の版で書いた、氏名入りのバックアップの Excel 5 つ（リポジトリの直下・2026 年 4 月 17 日の物）は、9 月 3 日のコミット `ea674d4` で消されました。**ただし、昔のコミット（履歴）には残っています。**履歴を見られる人は、だれでも取り出せます。

- `media/` に、Excel が 9 つ入っています（11 月 9 日のコミット）。中を見ると、従業員番号 `11111`・氏名が空など、**アプリのテストが作ったデータ** でした。テストを流すと、バックアップのテストが `media/` に Excel を作り、それがコミットに入ってしまいます（T-4 で直す）。
- `data.xlsx`（リポジトリの直下）は、作業詳細の選択肢（108 行）で、氏名は入っていません。どこからも使われていません（D-11）。

直し方:

1. **GitHub で、リポジトリを見られる人の範囲を確かめます。** リポジトリの Settings → General のいちばん下（Danger Zone）の「Change repository visibility」に、今の範囲が出ます。だれでも見られる（Public）なら、組織の管理者と相談して、Private にします。
2. 会社の決まり（個人情報の扱い）に従って、担当の人に知らせます。
3. **履歴からも消すか** を決めます。消すなら、GitHub の説明「Removing sensitive data from a repository」の手順（`git filter-repo` という道具を使う）に従います。履歴を書き換えると、全員が clone し直す必要があります。**必ず前任者・組織の管理者と相談し、時間を決めてから** やります。
4. `media/` をリポジトリから外し（D-3）、テストが `media/` に書かないようにします（T-4）。
5. これからは、データ管理画面でダウンロードしたバックアップを、**リポジトリのフォルダの中に保存しない** でください。

---

6. 不具合（B）

6.1 B-1 ヘルプ画面の初期化の URL（直った）

直った ／ 📖 コードから

`Help.tsx` 151 行目が `api.post("/api/help/", …)` になり、パスワードも一緒に送るようになりました。

6.2 B-2 バックアップのダウンロードで、Excel ではなく画面の HTML が落ちてくる

🔴 早めに ／ 新しく見つかった（前の B-2 が変わった） ／ ✅ 手元で再現（URL の行き先）・📖（Vite の置き換え）

どこ: `frontend/src/AdministratorPage/AdministratorLoading.tsx` 117〜124 行目

今: ダウンロードの URL を `` `${process.env.REACT_APP_API_BASE_URL}/api/download_backup` `` で作っています（119 行目）。画面のビルドが Vite に変わり、**Vite はビルドの時に `process.env` を空の `{}` に置き換えます**（手元に入っている Vite 7.3 の `dist/node/chunks/config.js` で確かめた。`axios.ts` 31〜33 行目のコメントにも同じことが書いてある）。そのため、この値は `undefined` になり、URL は `undefined/api/download_backup?file_path=…` になります。

- 頭に `/` が無いので、ブラウザは今のページからの相対の URL として、`/undefined/api/download_backup` を開きます。

- ✅ 手元で確かめると、この URL は API ではなく `react_view`（画面を返す関数）に当たりました。**落ちてくるのは画面の HTML** で、バックアップの Excel ではありません。
- ダウンロードの API が呼ばれないので、サーバーの `media` に、作ったバックアップの Excel がたまっていきます。

直し方: S-4 の④と同じです（相対の URL `"/api/download_backup"` にする）。

場所: `frontend/src/AdministratorPage/AdministratorLoading.tsx` の 117〜119 行目（handleSuccess の中）

今:

コピー

```
// GET /api/download_backup?file_path=... : バックアップファイルをダウンロード
//   （axios を使わず、ブラウザ標準のリンク遷移でダウンロードさせるため、APIのURLを直接組み立てる）
const endpoint = `${process.env.REACT_APP_API_BASE_URL}/api/download_backup`;
```

直した後（上の行を、全部これに置き換える）:

コピー

```
// GET /api/download_backup?file_path=... : バックアップファイルをダウンロード
//   画面と同じサーバーへの相対URLにする（Vite のビルドでは process.env が {} に置き換わり、
//   process.env.REACT_APP_API_BASE_URL は undefined になるため、使わない）
const endpoint = "/api/download_backup";
```

直ったかの確かめ方: 手元で動かし（D-4）、データ管理画面で「バックアップ開始」→ ダウンロードされたファイルが Excel で開けること。開発者ツールの「Network」で、`/api/download_backup?file_path=…` が 200 になっていること。

6.3 B-3 定義の Ver 名を変える・消すと、その Ver で入れた工数が開けなくなる

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/views/def_views.py` 293〜315 行目（`DefUpdate.put`）・349〜361 行目（`DefDelete.delete`）

今: 工数データは、どの定義で入れたかを、定義の **名前**（`def_ver2` に Ver 名の文字）で覚えています。定義の名前を変えたり、定義を消したりしても、工数データの `def_ver2` はそのままです。

✅ **手元で試した結果:** 管理者が定義の Ver 名を `v2` → `v2-renamed` に変えた後、`v2` で入れた工数を工数の編集画面で開くと、401「工数区分データが存在しません。」になり、画面はログイン画面へ飛びました（B-10）。工数データが残っている定義も、そのまま消せました（204）。

直し方: 名前を変えた時は、工数データの `def_ver2` も同じトランザクションの中でそろえます。工数データが残っている定義は、消させません（409 を返す）。

場所: `kosu/views/def_views.py` の 31〜32 行目（import）

今:

コピー

```
from django.db.models import Case, When, Value, IntegerField
from ..models import kosu_division, member, def_choice
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from django.db import transaction  # 複数の書き込みを「全部する / 全部しない」にまとめる
from django.db.models import Case, When, Value, IntegerField
from ..models import kosu_division, member, def_choice, Business_Time_graph
```

場所: `kosu/views/def_views.py` の 303〜313 行目（DefUpdate.put の中）

今:

コピー

```
    if serializer.is_valid():
      # Ver名の一意性確認:
      #   送信されたVer名が変更前と異なり、かつ他の工数区分定義がそのVer名を使っている場合は更新不可
      if data.get('kosu_name') != def_instance.kosu_name and kosu_division.objects.filter(kosu_name=data.get('kosu_name')).exists():
        return Response(
          {'status': 'error', 'message': '入力した工数区分定義Ver名はすでに登録されています。'},
          status=status.HTTP_400_BAD_REQUEST
        )
      # 問題なければデータベースに保存し、更新後の内容を 200（成功）で返す
      serializer.save()
      return Response(serializer.data, status=status.HTTP_200_OK)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    if serializer.is_valid():
      old_name = def_instance.kosu_name
      new_name = serializer.validated_data.get('kosu_name')
      # Ver名の一意性確認:
      #   送信されたVer名が変更前と異なり、かつ他の工数区分定義がそのVer名を使っている場合は更新不可
      if new_name != old_name and kosu_division.objects.filter(kosu_name=new_name).exists():
        return Response(
          {'status': 'error', 'message': '入力した工数区分定義Ver名はすでに登録されています。'},
          status=status.HTTP_400_BAD_REQUEST
        )
      # 保存する。Ver名を変えた時は、そのVerで入力した工数データ（def_ver2）も新しい名前にそろえる
      #   （そろえないと、その工数データの編集・集計の画面が「工数区分データが存在しません」になる）
      with transaction.atomic():
        serializer.save()
        if new_name != old_name:
          Business_Time_graph.objects.filter(def_ver2=old_name).update(def_ver2=new_name)
      return Response(serializer.data, status=status.HTTP_200_OK)
```

場所: `kosu/views/def_views.py` の 358〜360 行目（DefDelete.delete の最後）

今:

コピー

```
    # データベースから削除し、204（成功・返すデータなし）を返す
    #   ※工数データ側は Ver名を文字列（def_ver2）で持っているだけなので、工数データは削除されずに残る
    def_instance.delete()
```

直した後（上の行を、全部これに置き換える）：

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
    # このVerで入力した工数データが残っている間は消さない
    #   （工数データは Ver名を文字列（def_ver2）で持っているだけなので、消すとその工数データが開けなくなる）
    used = Business_Time_graph.objects.filter(def_ver2=def_instance.kosu_name).count()
    if used:
      return Response({'status': 'error', 'message': f'この工数区分定義で入力した工数データが {used} 件あるため、削除できません。'}, status=status.HTTP_409_CONFLICT)
    # データベースから削除し、204（成功・返すデータなし）を返す
    def_instance.delete()
```

> 名前を変えた時、ほかの人のセッションに残っている「使用中の Ver（ `input_def` ）」は古い名前のままです。その人の工数入力の画面は「工数区分データが存在しません。」（B-10 を直すと 404 の表示）になるので、工数区分定義の選択（定義 MENU）で選び直してもらいます。名前を変える前に、みんなに知らせてください。

直ったかの確かめ方: ✅ 直した写しでは、名前を変えた後も工数の編集画面が開け（200。工数データの `def_ver2` が `v2-renamed` に変わった）、工数データが残っている定義の削除は 409 で止まりました。

6.4 B-4 班員に入れた人を人員から消すと、班長の画面が 500

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/views/team_views.py` 421 行目（ `TeamCalendar` ）・633 行目（ `TeamOverTime` ）・726 行目（ `TeamView` ）

今: 班員の番号から人員を `member.objects.get(...)` で探しています。`get` は、見つからないと例外を出すので、班員に入れた人を人員から消すと 500 になります（418 行目のコメントにも書いてある）。

✅ **手元で試した結果:** 班員に、人員に無い番号を入れると、班員の工数カレンダー（ `team_calendar` ）と残業の画面（ `team_overtime` ）が 500 になりました。

直し方: `filter(...).first()` にして、見つからない人は飛ばします。`TeamView` は、同じ番号の人が 2 人いる時（B-6）に 500 にならないよう、同じく `first()` にします。

場所: `kosu/views/team_views.py` の 417〜422 行目（TeamCalendar.get）

今:

コピー

```
    # 画面の行見出し用に［従業員番号，氏名］のリストを班員の登録順に作る
    #   ※班員に登録された従業員番号の人員が削除されていると member.DoesNotExist が発生する
    member_name_list = []
    for m in valid_member_numbers:
      member_obj = member.objects.get(employee_no=m)
      member_name_list.append([m, member_obj.name])
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 画面の行見出し用に［従業員番号，氏名］のリストを班員の登録順に作る
    #   人員から消された人は飛ばす（get だと「無い」時に例外で 500 になるので filter().first() にする）
    member_name_list = []
    for m in valid_member_numbers:
      member_obj = member.objects.filter(employee_no=m).first()
      if member_obj is not None:
        member_name_list.append([m, member_obj.name])
```

場所: `kosu/views/team_views.py` の 629〜634 行目（TeamOverTime.get）

今:

コピー

```
    # 画面の行見出し用に［従業員番号，氏名］のリストを班員の登録順に作る
    #   ※班員に登録された従業員番号の人員が削除されていると member.DoesNotExist が発生する
    member_name_list = []
    for m in valid_member_numbers:
      member_obj = member.objects.get(employee_no=m)
      member_name_list.append([m, member_obj.name])
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 画面の行見出し用に［従業員番号，氏名］のリストを班員の登録順に作る
    #   人員から消された人は飛ばす（get だと「無い」時に例外で 500 になるので filter().first() にする）
    member_name_list = []
    for m in valid_member_numbers:
      member_obj = member.objects.filter(employee_no=m).first()
      if member_obj is not None:
        member_name_list.append([m, member_obj.name])
```

場所: `kosu/views/team_views.py` の 723〜727 行目（TeamView.get）

今:

コピー

```
# 画面の行見出し用に［従業員番号，氏名］のリストを作る
member_name_list = []
for m in team_shop:
  member_obj = member.objects.get(employee_no=m)
  member_name_list.append([m, member_obj.name])
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 画面の行見出し用に［従業員番号，氏名］のリストを作る
#   （同じ従業員番号が2人いても 500 にならないよう、get ではなく filter().first() にする）
member_name_list = []
for m in team_shop:
  member_obj = member.objects.filter(employee_no=m).first()
  member_name_list.append([m, member_obj.name])
```

直ったかの確かめ方: ✅ 直した写しでは、2 つの画面が 200 になり、消した人は一覧に出ませんでした。

6.5 B-5 人員から消した人の問い合わせが、開けない・直せない・消せない

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/views/inquiry_views.py` 247〜250 行目（`InquirDetail.get`）・470〜473 行目（`InquirUpdate.put`）・536〜539 行目（`InquirUpdate.delete`）

今: 問い合わせた人を `member.objects.get(...)` で探し、見つからないと 401「質問者の人員情報が見つかりません。」を返します。画面はログイン画面へ飛びます。

✅ **手元で試した結果:** 人員から消した人の問い合わせを開くと、401 になりました。

直し方: 見つからなくても続けます（お知らせの片付けだけ飛ばす）。`put`・`delete` の直しは S-3 の④に入っているので、ここでは `InquirDetail.get` を直します（先に B-7 の①を入れてください）。

場所: `kosu/views/inquiry_views.py` の 245〜260 行目（InquirDetail.get の中ほど）

今:

コピー

```
# 問い合わせ者（従業員番号 employee_no2）の人員データを取得
#   問い合わせ者の人員データが削除されている場合は 401 を返す
try:
  inquir_member = member.objects.get(employee_no=inquir_instance.employee_no2)
except member.DoesNotExist:
  return Response({'status': 'error', 'message': '質問者の人員情報が見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

# 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の判定とお知らせ削除に使う）
#   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
try:
  admin_data = administrator_data.objects.order_by("id").last()
except administrator_data.DoesNotExist:
  return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

# ログイン者が問い合わせ者本人の場合：本人向けお知らせ（回答通知）を既読にする
if inquir_member.employee_no == member_data.employee_no:
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 問い合わせ者（従業員番号 employee_no2）の人員データを取得
#   人員から消された人の問い合わせも開けるよう、見つからなければ None のまま続ける
inquir_member = member.objects.filter(employee_no=inquir_instance.employee_no2).first()

# 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の判定とお知らせ削除に使う。無ければ初期値で作る）
admin_data = get_settings()

# ログイン者が問い合わせ者本人の場合：本人向けお知らせ（回答通知）を既読にする
if inquir_member is not None and inquir_member.employee_no == member_data.employee_no:
```

直ったかの確かめ方: ✅ 直した写しでは、200 で開けました（問い合わせた人の欄は空で返る）。

6.6 B-6 同じ従業員番号の人が 2 人いると、その人の画面が 500

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/models.py`（`member`・`Business_Time_graph`・`team_member`・`kosu_division`）

今:「同じ従業員番号の人員は 1 人だけ」「1 人 1 日 1 件の工数」などの決まりを、画面の API の中だけで確かめていて、データベースの決まり（一意の制約）がありません。ロード（B-8）や同時の操作で、重なったデータが入ることがあります。

✅ **手元で試した結果:** 同じ番号の人を 2 人入れると、その人の工数の保存（`kosu_new` の POST）と、人員の編集画面（`member_update` の GET）が 500 になりました。

直し方:

1. **本番のデータに、重なりが無いかを先に調べます**（決まりを足す前に重なりがあると、migrate が失敗します）。Azure の SSH（000 巻 §10）で `python manage.py shell` を開き、次を打ちます。読むだけで、何も変えません。

コピー

```
from django.db.models import Count
from kosu.models import member, Business_Time_graph, team_member, kosu_division
print(member.objects.values('employee_no').annotate(n=Count('id')).filter(n__gt=1))
print(Business_Time_graph.objects.values('employee_no3', 'work_day2').annotate(n=Count('id')).filter(n__gt=1))
print(team_member.objects.values('employee_no5').annotate(n=Count('id')).filter(n__gt=1))
print(kosu_division.objects.values('kosu_name').annotate(n=Count('id')).filter(n__gt=1))
```

▶ ここで初めて出てくるコード

- `.values()`（Django）：行を、モデルではなく、決めた列だけの辞書で受け取るメソッド。`member.objects.values('employee_no').annotate(n=Count('id'))` は、「従業員番号ごとに、何行あるか」を数える時の書き方です。
- `Count`（Django）：Django の `Count("id")` は、ORM の集計で、数を数えます。`.annotate(n=Count("id"))` で、グループごとの件数が付きます。

4 行とも `<QuerySet []>`（空）なら、重なりはありません。何か出たら、どちらを残すかを使う人と決めて、画面から直してから次へ進みます。

2. モデルに、データベースの決まりを足します（4 つのモデルの、項目の並びの後ろ）。

場所: `kosu/models.py` の 121 行目の前（**str** の説明のコメントの前）

足すコード（この行の前に入れる）：

コピー

```
# データベースの決まり：同じ従業員番号の人員は1人だけ（2人いると、その人の画面がほとんど 500 になる）
class Meta:
  constraints = [
    models.UniqueConstraint(fields=['employee_no'], name='unique_member_employee_no'),
  ]
```

▶ ここで初めて出てくるコード

- `.UniqueConstraint()`（Django）：「この組み合わせの値は、表の中で重ならない」という決まりを表す物。モデルの `class Meta:` の `constraints` に書きます。`fields=['employee_no3', 'work_day2']` は「同じ人の、同じ就業日の行は、2 つ作れない」。`name=` は、決まりの名前（表の中で重ならない名前）です。

場所: `kosu/models.py` の 168 行目の前（**str** の説明のコメントの前）

足すコード（この行の前に入れる）：

コピー

```
# データベースの決まり: 1人1日1件（2件あると、その日の工数入力の画面が開けない）
class Meta:
  constraints = [
    models.UniqueConstraint(fields=['employee_no3', 'work_day2'], name='unique_kosu_per_day'),
  ]
```

場所: `kosu/models.py` の 204 行目の前（**str** の説明のコメントの前）

足すコード（この行の前に入れる）：

コピー

```
# データベースの決まり: 班員データは班を管理する人1人につき1件
class Meta:
  constraints = [
    models.UniqueConstraint(fields=['employee_no5'], name='unique_team_leader'),
  ]
```

場所: `kosu/models.py` の 435 行目の前（**str** の説明のコメントの前）

足すコード（この行の前に入れる）：

コピー

```
# データベースの決まり: 工数区分定義の Ver 名は重ならない（工数データは Ver 名で定義を探すため）
class Meta:
  constraints = [
    models.UniqueConstraint(fields=['kosu_name'], name='unique_kosu_name'),
  ]
```

3. マイグレーションのファイルを作ります。先に D-2 の `0013` を入れてから、`python manage.py makemigrations kosu -n unique_constraints` を打つと、次のファイルができます。

新しいファイル: `kosu/migrations/0014_unique_constraints.py`（python manage.py makemigrations kosu -n unique_constraints で作られる物）

中身（全部）：

コピー

```
# Generated by Django 4.2.7 on 2026-11-09 15:29

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ('kosu', '0013_sync_models'),
    ]

    operations = [
        migrations.AddConstraint(
            model_name='business_time_graph',
            constraint=models.UniqueConstraint(fields=('employee_no3', 'work_day2'), name='unique_kosu_per_day'),
        ),
        migrations.AddConstraint(
            model_name='kosu_division',
            constraint=models.UniqueConstraint(fields=('kosu_name',), name='unique_kosu_name'),
        ),
        migrations.AddConstraint(
            model_name='member',
            constraint=models.UniqueConstraint(fields=('employee_no',), name='unique_member_employee_no'),
        ),
        migrations.AddConstraint(
            model_name='team_member',
            constraint=models.UniqueConstraint(fields=('employee_no5',), name='unique_team_leader'),
        ),
    ]
```

> **▶ ここで初めて出てくるコード**
>
> - **`.AddConstraint()`**（Django）：マイグレーションの中で、表に決まり（`UniqueConstraint` など）を足す操作。モデルの `class Meta:` の `constraints` を見て、`makemigrations` が書きます。

4. 本番に出す時に、`migrate` をします（D-2）。

直ったかの確かめ方: ✅ 直した写しでは、同じ番号の 2 人目を作ろうとすると、データベースが `IntegrityError`（決まり違反）で止めました。アプリのテスト 339 個は、この決まりがあっても全部通りました（テストは重なったデータを作っていない）。

6.7 B-7 設定データが無いと、いくつかの画面が 500

🟠 早めに ／ 一部だけ直った ／ ✅ 手元で再現

どこ: 設定データ（`administrator_data`）を取る 10 か所。`main_views.py` 436〜439・672〜675・712〜715・742〜745 行目、`team_views.py` 699〜702 行目、`inquiry_views.py` 172〜175・254〜257・401〜404・464〜467・543〜546 行目

今: どこも次の形です。

コピー

```
try:
  admin_data = administrator_data.objects.order_by("id").last()
except administrator_data.DoesNotExist:
  return Response({...'設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

`.last()` は、データが無い時に例外ではなく `None` を返すので、`except` には決して入りません（今のコメントにも「この except に入ることはない」と書いてある）。そのまま `None` の項目を読もうとして 500 になります。

✅ **手元で試した結果:** 設定データを全部消すと、メインメニューと一覧は動きました（設定の代わりに空の値が返る）が、ショップ単位の工数入力状況（`team_view`）と問い合わせの登録（`inquir_new` の POST）は 500 でした。

直し方:

① `main_utils.py` に、設定データを 1 件返す関数 `get_settings` を足します（無ければ初めの値で作る。ヘルプの初期化と同じ「一覧の件数 20」）。

場所: `kosu/utils/main_utils.py` の 169 行目（ファイルの最後）

今:

コピー

```
  return model_names
```

直した後（上の行を、全部これに置き換える）:

コピー

```
  return model_names



# 管理者設定（administrator_data）を1件返す。
#
# 設定は1件だけを使う決まり。最新（IDが最大）の1件を返し、1件も無ければ初期値（一覧表示項目数 20）で作ってから返す。
#   ※ .last() は「無い」時に例外ではなく None を返すので、try / except administrator_data.DoesNotExist では防げない
#
# 戻り値:
#   administrator_data: 管理者設定
def get_settings():
  admin_data = administrator_data.objects.order_by("id").last()
  if admin_data is None:
    admin_data = administrator_data.objects.create(menu_row='20')
  return admin_data
```

② `main_views.py` の 4 か所と、`team_views.py` の 1 か所を、`get_settings()` に置き換えます。

場所: `kosu/views/main_views.py` の 64 行目（import）

今:

コピー

```
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp, get_settings
```

場所: `kosu/views/main_views.py` の 434〜439 行目（Menu.get）

今:

コピー

```
    # 最新（IDが最大）の管理者設定を取得
    #   ※ last() はデータが無いとき例外ではなく None を返すため、この except には入ることはない
    try:
      admin_data = administrator_data.objects.order_by("id").last()
    except administrator_data.DoesNotExist:
      return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 最新（IDが最大）の管理者設定を取得（無ければ初期値で作る。utils/main_utils.py の get_settings）
    admin_data = get_settings()
```

場所: `kosu/views/main_views.py` の 670〜675 行目（AdministratorMenu.get）

今:

コピー

```
    # 最新（IDが最大）の管理者設定を取得
    #   ※ last() はデータが無いとき例外ではなく None を返すため、この except には入ることはない
    try:
      admin_data = administrator_data.objects.order_by("id").last()
    except administrator_data.DoesNotExist:
      return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 最新（IDが最大）の管理者設定を取得（無ければ初期値で作る。utils/main_utils.py の get_settings）
    admin_data = get_settings()
```

場所: `kosu/views/main_views.py` の 710〜715 行目（AdministratorUpdate.get）

今:

コピー

```
    # 最新（IDが最大）の管理者設定を取得
    #   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
    try:
      admin_data = administrator_data.objects.order_by("id").last()
    except administrator_data.DoesNotExist:
      return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 最新（IDが最大）の管理者設定を取得（無ければ初期値で作る。utils/main_utils.py の get_settings）
    admin_data = get_settings()
```

場所: `kosu/views/main_views.py` の 740〜745 行目（AdministratorUpdate.put）

今:

コピー

```
    # 最新（IDが最大）の管理者設定を取得（更新対象）
    #   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
    try:
      admin_data = administrator_data.objects.order_by("id").last()
    except administrator_data.DoesNotExist:
      return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 最新（IDが最大）の管理者設定を取得（無ければ初期値で作る。utils/main_utils.py の get_settings）
    admin_data = get_settings()
```

場所: `kosu/views/team_views.py` の 45 行目（import）

今:

コピー

```
from ..utils.main_utils import CustomPagination
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from ..utils.main_utils import CustomPagination, get_settings, parse_year_month
```

場所: `kosu/views/team_views.py` の 697〜702 行目（TeamView.get）

今:

コピー

```
    # 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の従業員番号を知るため）
    #   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
    try:
      admin_data = administrator_data.objects.order_by("id").last()
    except administrator_data.DoesNotExist:
      return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 最新（IDが最大）の管理者設定を取得（問い合わせ担当者の従業員番号を知るため。無ければ初期値で作る）
    admin_data = get_settings()
```

`inquiry_views.py` の 5 か所は、S-3 の④・B-5・B-18 の直しの中で、`get_settings()` に置き換えています。`B-7-team-import` で一緒に読み込んでいる `parse_year_month` は B-15 で足す関数です。

直ったかの確かめ方: ✅ 直した写しでは、設定データが無くても、上の画面が全部 200 になり、設定データが 1 件作られました。

6.8 B-8 データのロードが途中で失敗すると、データが半分だけ消える

🔴 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/tasks.py` のロード関数 7 つ（`load_kosu_file` 268 行目・`load_member_file` 459 行目・`load_team_file` 656 行目・`load_def_file` 817 行目・`load_choice_file` 952 行目・`load_inquiry_file` 1085 行目・`load_setting_file` 1253 行目）

今: ロードは、古いデータを消してから、Excel の行を 1 行ずつ入れます（たとえば `load_choice_file` は 973〜974 行目で全部消してから入れる）。1 行ごとにデータベースに確定するので、途中の行で失敗すると、**古いデータは消えたのに、新しいデータは途中まで** になります。

✅ **手元で試した結果:** 作業詳細の選択肢が 2 件ある所へ、2 行目が空の Excel をロードすると、`master` では元の 2 件が消えてしまいました（T-2 の `TestLoadIsAtomic` が `master` では失敗する）。

直し方: ロード関数を「全部入れるか、何も変えないか」にするデコレーター `atomic_load` を足し、7 つの関数に付けます。1 行ずつ「整合性エラーの行は飛ばす」ようにしている所（5 か所）は、その行だけを取り消す印（セーブポイント）を付けます。

場所: `kosu/tasks.py` の 50〜51 行目（import と、新しいデコレーター）

今:

コピー

```
from django.db import IntegrityError  # DBの整合性エラー（NOT NULL 制約違反など）
from django.core.exceptions import ObjectDoesNotExist  # 該当するレコードが無いときの例外
```

直した後（上の行を、全部これに置き換える）：

コピー

```
from django.db import IntegrityError, transaction  # DBの整合性エラー / 複数の書き込みを「全部する・全部しない」にまとめる
from django.core.exceptions import ObjectDoesNotExist  # 該当するレコードが無いときの例外
import functools  # デコレーターを作る時に、元の関数の名前などを引き継ぐ



# ロード関数を「全部入れるか、何も変えないか」にするデコレーター。
#
# ロード関数の中身をデータベースのトランザクション（まとめて確定・まとめて取り消し）の中で動かし、
# 途中で失敗して {'status': 'error'} を返した時は、それまでの削除・登録を全部取り消す。
# （前は 1 行ずつ確定していたので、途中で失敗すると「古いデータは消えたのに新しいデータは半分」になった）
#
# 引数:
#   func: ロード関数（load_kosu_file など）
# 戻り値:
#   トランザクションの中で func を呼ぶ関数
def atomic_load(func):
  @functools.wraps(func)
  def wrapper(*args, **kwargs):
    with transaction.atomic():
      result = func(*args, **kwargs)
      if isinstance(result, tuple) and result and isinstance(result[0], dict) and result[0].get('status') == 'error':
        transaction.set_rollback(True)
      return result
  return wrapper
```

▶ ここで初めて出てくるコード

- **`.wraps()`**（Python）：`@functools.wraps(func)` は、デコレーターの中で作った関数に、元の関数（`func`）の名前や説明を引き継がせる印。付けないと、エラーの表示などで、元の関数の名前が分からなくなります。
- **`func()`**（Python）：デコレーター `atomic_load` に渡された、元の関数（ロード関数）を呼んでいる所。`func(*args, **kwargs)` の `*args`・`**kwargs` は、「受け取った引数を、全部そのまま渡す」書き方です。
- **`@functools`**（Python）：`@functools.wraps(func)` の頭の部分。`functools`（関数を扱う道具を集めた、Python に最初から入っている部品）の `wraps` を、デコレーターとして使うという意味です。

7 つの関数の頭（`def` の行）に、`@atomic_load` を足します。

場所: `kosu/tasks.py` の 268 行目（load_kosu_file の頭）

今の行（`def load_kosu_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_kosu_file(file_path):
```

▶ ここで初めて出てくるコード

- `@atomic_load`（Python）：この巻で `kosu/tasks.py` に足す、自分で作ったデコレーター。ロード関数の上に付けると、その関数の中の書き込みを 1 つのトランザクションにまとめ、関数がエラーを返した時は、全部取り消します。

場所: `kosu/tasks.py` の 459 行目（load_member_file の頭）

今の行（`def load_member_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_member_file(file_path):
```

場所: `kosu/tasks.py` の 656 行目（load_team_file の頭）

今の行（`def load_team_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_team_file(file_path):
```

場所: `kosu/tasks.py` の 817 行目（load_def_file の頭）

今の行（`def load_def_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_def_file(file_path):
```

場所: `kosu/tasks.py` の 952 行目（load_choice_file の頭）

今の行（`def load_choice_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_choice_file(file_path):
```

場所: `kosu/tasks.py` の 1085 行目（load_inquiry_file の頭）

今の行（`def load_inquiry_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_inquiry_file(file_path):
```

場所: `kosu/tasks.py` の 1253 行目（load_setting_file の頭）

今の行（`def load_setting_file(file_path):`）を、次に置き換える:

コピー

```
@atomic_load  # 途中で失敗したら全部取り消す
def load_setting_file(file_path):
```

1 行ずつの登録の `try:` と `except IntegrityError` に、セーブポイントを足します（5 つの関数）。

場所: `kosu/tasks.py` の 318 行目（load_kosu_file の 1 行ずつの登録）

今の行（`try:`）を、次に置き換える:

コピー

```
    try:
      sid = transaction.savepoint()  # この行だけを取り消せるよう、印を付けておく
```

▶ ここで初めて出てくるコード

- `.savepoint()`（Django）：`transaction.savepoint()` は、トランザクションの途中に「ここまで戻れる印（セーブポイント）」を付けて、その印を返す関数。後で `transaction.savepoint_rollback(印)` を呼ぶと、印より後の書き込みだけを取り消せます。

場所: `kosu/tasks.py` の 337 行目（load_kosu_file の 1 行ずつの登録）

今の行（`except IntegrityError as create_err:`）を、次に置き換える:

コピー

```
    except IntegrityError as create_err:
      transaction.savepoint_rollback(sid)  # この行の分だけ取り消して、次の行へ進む
```

▶ ここで初めて出てくるコード

- `.savepoint_rollback()`（Django）：`transaction.savepoint_rollback(sid)` は、`savepoint()` で付けた印 `sid` より後の書き込みだけを取り消す関数。ロードで「この 1 行だけ飛ばして、次の行へ進む」のに使います。

場所: `kosu/tasks.py` の 504 行目（load_member_file の 1 行ずつの登録）

今の行（`try:`）を、次に置き換える:

コピー

```
    try:
      sid = transaction.savepoint()  # この行だけを取り消せるよう、印を付けておく
```

場所: `kosu/tasks.py` の 548 行目（load_member_file の 1 行ずつの登録）

今の行（`except IntegrityError as create_e:`）を、次に置き換える:

コピー

```
    except IntegrityError as create_e:
      transaction.savepoint_rollback(sid)  # この行の分だけ取り消して、次の行へ進む
```

場所: `kosu/tasks.py` の 693 行目（load_team_file の 1 行ずつの登録）

今の行（`try:`）を、次に置き換える:

コピー

```
    try:
      sid = transaction.savepoint()  # この行だけを取り消せるよう、印を付けておく
```

場所: `kosu/tasks.py` の 714 行目（load_team_file の 1 行ずつの登録）

今の行（`except IntegrityError as create_e:`）を、次に置き換える:

コピー

```
    except IntegrityError as create_e:
      transaction.savepoint_rollback(sid)  # この行の分だけ取り消して、次の行へ進む
```

場所: `kosu/tasks.py` の 1110 行目（load_inquiry_file の 1 行ずつの登録）

今の行（`try:`）を、次に置き換える:

コピー

```
    try:
      sid = transaction.savepoint()  # この行だけを取り消せるよう、印を付けておく
```

場所: `kosu/tasks.py` の 1127 行目（load_inquiry_file の 1 行ずつの登録）

今の行（`except IntegrityError as create_e:`）を、次に置き換える:

コピー

```
    except IntegrityError as create_e:
      transaction.savepoint_rollback(sid)  # この行の分だけ取り消して、次の行へ進む
```

場所: `kosu/tasks.py` の 1293 行目（load_setting_file の 1 行ずつの登録）

今の行（`try:`）を、次に置き換える:

コピー

```
    try:
      sid = transaction.savepoint()  # この行だけを取り消せるよう、印を付けておく
```

場所: `kosu/tasks.py` の 1314 行目（load_setting_file の 1 行ずつの登録）

今の行（`except IntegrityError as create_e:`）を、次に置き換える:

コピー

```
    except IntegrityError as create_e:
      transaction.savepoint_rollback(sid)  # この行の分だけ取り消して、次の行へ進む
```

直ったかの確かめ方: T-2 の `TestLoadIsAtomic` が通ること。✅ 直した写しでは、元の 2 件が残りました。

6.9 B-9 ログインが切れている時の 500（直った）

直った ／ ✅ 手元で確かめた

ログインしていない時は、どの API も、共通の確かめ（`IsLoggedIn`）で先に 401 になるようになりました。

6.10 B-10 ログイン以外の理由でも 401 を返し、画面がログイン画面へ飛ぶ

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

今: 画面は、どこでも「401 が返ったらログイン画面へ飛ぶ」作りです（49 ファイル・90 か所）。ところがサーバーは、ログインとは関係ない理由（データが無い・定義が見つからない・設定が無い）でも 401 を返しています。使う人には、理由が分からないまま、ログイン画面に戻されたように見えます。

401 を返している所は 46 か所あります。

理由	か所	どうするか
「使用する工数区分定義情報が確認できません。」（セッションに `input_def` が無い）	17	**このままでよい。**ログインし直せば `input_def` が入るので、ログイン画面へ戻すのが正しい
「設定データが見つかりません。」	10	B-7 で消える（決して通らない所だった）
「質問者の人員情報が見つかりません。」	3	B-5・S-3 の④で消える
「工数区分データが存在しません。」など、残りの理由	16	**下の表のとおり、404 か 409 に変える**

✅ **手元で試した結果:** 定義の Ver で、無い名前を選べてしまい（`def_ver` の POST が 200）、その後の工数入力の画面が 401「工数区分データが存在しません。」になりました。

直し方:

① 定義の Ver を選ぶ API で、実際にある名前だけを受け付けます。

場所: `kosu/views/def_views.py` の 92〜94 行目（DefVer.post）

今:

コピー

```
    # 未選択（空・未送信）の場合は 400 を返す
    if not selected_version:
      return Response({'status': 'error', 'message': 'Versionchoice is required.'}, status=status.HTTP_400_BAD_REQUEST)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 未選択（空・未送信）の場合は 400 を返す
    if not selected_version:
      return Response({'status': 'error', 'message': 'Versionchoice is required.'}, status=status.HTTP_400_BAD_REQUEST)
    # 実際にある工数区分定義のVer名だけを受け付ける（無い名前をセッションに入れると、ほかの画面が開けなくなる）
    if not kosu_division.objects.filter(kosu_name=selected_version).exists():
      return Response({'status': 'error', 'message': '選んだ工数区分定義がありません。'}, status=status.HTTP_400_BAD_REQUEST)
```

② 残りの 16 か所は、その行の `status=status.HTTP_401_UNAUTHORIZED` だけを、次のように変えます（行のほかの所は変えません）。

ファイル	行	返している理由（ message ）	今	直した後
def_views.py	460	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
def_views.py	552	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	190	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	924	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	926	複数の工数区分データが存在します。	HTTP_401_UNAUTHORIZED	HTTP_409_CONFLICT
kosu_views.py	933	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	935	複数の工数区分データが存在します。	HTTP_401_UNAUTHORIZED	HTTP_409_CONFLICT
kosu_views.py	1177	工数データが見つかりません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	1180	工数データが見つかりません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	1668	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
kosu_views.py	1764	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
main_views.py	965	工数データのユーザーが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
team_views.py	83	複数の班員データが存在します。	HTTP_401_UNAUTHORIZED	HTTP_409_CONFLICT
team_views.py	318	メンバーが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
team_views.py	330	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND
team_views.py	339	工数区分データが存在しません。	HTTP_401_UNAUTHORIZED	HTTP_404_NOT_FOUND

画面は、401 以外のエラーでは、サーバーから届いた `message` を画面に出す作りなので（例: `KosuNew.tsx` の `setErrorMessage(err.response?.data?.message …)`）、画面の直しは要りません。

直ったかの確かめ方: ✅ 直した写しでは、無い Ver 名を選ぶと 400「選んだ工数区分定義がありません。」になり、工数入力の画面は開いたままでした。T-4 の時間帯削除の 2 つのテスト（404 を確かめる）が通ること。

6.11 B-11 作業詳細選択肢の「ロード開始」が、定義のロード中に「実行中...」になる

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `frontend/src/AdministratorPage/AdministratorLoading.tsx` 844〜849 行目

今: 作業詳細選択肢の「ロード開始」ボタンの文字が、作業詳細選択肢ではなく、工数区分定義のロードの実行中かどうかで変わります（844 行目のコメントにも書いてある）。

場所: `frontend/src/AdministratorPage/AdministratorLoading.tsx` の 844〜849 行目（作業詳細選択肢の「ロード開始」ボタン）

今:

コピー

```
{/* ※「実行中...」表示の判定には ChoiceLoad ではなく DefLoad の実行中フラグを参照している */}
<input
  id="start-choice-load"
  name="start-choice-load"
  type="button"
  value={runningStates.DefLoad ? "実行中..." : "ロード開始"}
```

▶ ここで初めて出てくるコード

- **`name=`**（React）: 部品（`<input>` など）の名前。フォームを送る時の項目の名前や、テストで部品を探す目印に使います。`id=` と同じ名前にしておくことが多いです。

直した後（上の行を、全部これに置き換える）:

コピー

```
{/* ロード開始ボタン（ファイル未選択の間も押せない）。「実行中...」は作業詳細選択肢のロードの実行中フラグで出す */}
<input
  id="start-choice-load"
  name="start-choice-load"
  type="button"
  value={runningStates.ChoiceLoad ? "実行中..." : "ロード開始"}
```

6.12 B-12 ファイル選びで CSV・XLS も選べるが、サーバーは xlsx しか読めない

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `frontend/src/AdministratorPage/AdministratorLoading.tsx` の 7 か所

サーバーのロード（`openpyxl.load_workbook`）は `.xlsx` しか読めません。ファイル選びの窓で `.xlsx` だけを出すようにします。7 か所とも、行の中の `accept=".csv, .xlsx, .xls"` を `accept=".xlsx"` に変えます。

ファイル	行	今	直した後
`AdministratorLoading.tsx`	737	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	783	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	830	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	877	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	923	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	969	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`
`AdministratorLoading.tsx`	1015	`accept=".csv, .xlsx, .xls"`	`accept=".xlsx"`

画面の「ファイルを選択 (CSV/XLSX)」の文字も、近くを直す時に「ファイルを選択 (XLSX)」にそろえてください。

6.13 B-13 バックアップ・ロードが、サーバーの再起動で止まり、「待ち」のまま残る

🟠 早めに ／ 前から残っている ／ 📖 コードから

どこ: `kosu/views/asynchronous_views.py` 115〜123 行目・412 行目（`backup`）・555〜562 行目（`check_task_status`）

今:

- バックアップ・ロードは、サーバーの中の別のスレッドで動かしています（422〜429 行目）。App Service が再起動すると、途中のスレッドは消え、タスクの記録は「pending（処理中）」のまま残ります。画面は、1 秒ごとに状態を聞き続けます。
- タスクの記録を、入力の確かめより前に作っています（120〜123 行目。119 行目のコメントにも書いてある）。日付の誤りなどで返した時も、「pending」の記録が残ります。

直し方（今すぐできる分）: タスクの記録は、入力の確かめが全部通ってから作ります。30 分（`settings.py` の `Q_CLUSTER` の `timeout` と同じ）を過ぎても「pending」の物は、止まったとみなして「error」にします。

場所: `kosu/views/asynchronous_views.py` の 40 行目（import）

今:

コピー

```
from django.urls import resolve  # アクセスされたURLから urls.py のURL名を調べる
```

直した後（上の行を、全部これに置き換える）:

コピー

```
from django.urls import resolve  # アクセスされたURLから urls.py のURL名を調べる
from django.utils import timezone  # 今の日時（タイムゾーン付き）
```

場所: `kosu/views/asynchronous_views.py` の 115〜123 行目（backup の最初）

今:

コピー

```
    # タスクIDを生成する
    #   uuid4: 重複しないランダムなID（例: "3f2b9c1e-..."）。画面はこのIDで進み具合を問い合わせる
    task_id = str(uuid.uuid4())
    # 非同期タスクの状態を管理するレコードを「処理中（pending）」で作成する
    #   ※この時点で作成するため、以下の入力チェックでエラーを返した場合も pending のレコードが残る
    AsyncTask.objects.create(
        task_id=task_id,
        status='pending',
    )
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # タスクIDを生成する
    #   uuid4: 重複しないランダムなID（例: "3f2b9c1e-..."）。画面はこのIDで進み具合を問い合わせる
    #   （AsyncTask のレコードは、入力チェックがすべて通った後に作る。下の「タスクを開始する」の所）
    task_id = str(uuid.uuid4())
```

場所: `kosu/views/asynchronous_views.py` の 412 行目（タスクを開始する所）

今:

コピー

```
    if getattr(settings, 'TESTING', False):
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 入力チェックがすべて通ったので、非同期タスクの状態を管理するレコードを「処理中（pending）」で作成する
    #   （前は関数の最初で作っていたので、入力エラーで返した時も pending のレコードが残り続けた）
    AsyncTask.objects.create(
        task_id=task_id,
        status='pending',
    )
    if getattr(settings, 'TESTING', False):
```

場所: `kosu/views/asynchronous_views.py` の 555〜562 行目（check_task_status の処理中の所）

今:

コピー

```
        # それ以外（pending）は処理中として 202（受付済み・処理中）を返す
        else:
            return JsonResponse(
                {
                    'status': 'pending',
                },
                status=status.HTTP_202_ACCEPTED,
            )
```

直した後（上の行を、全部これに置き換える）：

コピー

```
        # それ以外（pending）は処理中。ただし作ってから 30 分（settings.py の Q_CLUSTER の timeout と同じ）を過ぎても
        # pending のままなら、サーバーの再起動などで別スレッドが止まったとみなして error にする
        else:
            if task.created_at < timezone.now() - datetime.timedelta(minutes=30):
                task.status = 'error'
                task.result = '処理が途中で止まりました（サーバーの再起動など）。もう一度実行してください。'
                task.save()
                return JsonResponse({'status': 'error', 'message': task.result})
            return JsonResponse(
                {
                    'status': 'pending',
                },
                status=status.HTTP_202_ACCEPTED,
            )
```

▶ ここで初めて出てくるコード

- **`.now()`**（Django）：`timezone.now()` は、今の日時を、時差付きで返します（Django の設定に合わせた形）。Python の `datetime.now()` は時差が付かないので、Django ではこちらを使います。

長い目では、別のスレッドではなく、仕事を順に受け持つ仕組み（`settings.py` に設定だけある Django-Q。今は開発が止まっているので、後を継いだ `django-q2` ）で動かすと、再起動しても続きから動きます（D-8）。

6.14 B-14 月で絞り込むと、月初めの 0〜9 時の記録が前の月に入る

🟡 余裕のある時に ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 1076〜1083 行目（タスクの一覧）・1213〜1219 行目（操作履歴の一覧）

今:「月」で絞り込む時に、日時を **文字** にして「`2026-02` で始まるか」（`__startswith`）で比べています。データベースの日時は世界の標準時（UTC）で入っていて、文字にすると UTC のままなので、日本時間の 2 月 1 日 0〜9 時の記録は「`2026-01-31 …`」になり、1 月に入ります。

✅ **手元で試した結果:** 日本時間 2026 年 2 月 1 日 5 時に作ったタスクは、2 月で絞り込むと出ず、1 月で絞り込むと出ました（手元の SQLite で確かめた。本番の PostgreSQL でも、Django はデータベースとのやり取りを UTC にするので、同じになる）。

直し方: `__year`・`__month` で比べます（こちらは `settings.py` の `TIME_ZONE` = 日本時間で判定される）。

場所: `kosu/views/main_views.py` の 1076〜1083 行目（AdministratorTaskList.get）

今:

コピー

```
        # 作成日での絞り込み（検索日がある場合のみ）
        #   月単位: 作成日時が検索日の先頭7文字（"YYYY-MM"）で始まるもの → その月の全件
        #   日単位: 作成日時の日付部分（created_at__date）が検索日と一致するもの
        if search_day:
          if mode == 'month':
            tasks = tasks.filter(created_at__startswith=search_day[:7])
          else:
            tasks = tasks = tasks.filter(created_at__date=search_day)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
        # 作成日での絞り込み（検索日がある場合のみ）
        #   月単位: 作成日時を日本時間（settings.TIME_ZONE）で見て、検索日と同じ年・月のもの
        #           （__year / __month は TIME_ZONE で判定する。__startswith は UTC の文字で比べるので、
        #            月初めの 0〜9 時の記録が前の月に入ってしまう）
        #   日単位: 作成日時の日付部分（created_at__date）が検索日と一致するもの
        if search_day:
          if mode == 'month':
            tasks = tasks.filter(created_at__year=int(search_day[:4]), created_at__month=int(search_day[5:7]))
          else:
            tasks = tasks.filter(created_at__date=search_day)
```

場所: `kosu/views/main_views.py` の 1213〜1219 行目（AdministratorHistoryList.get）

今:

コピー

```
        # 操作日での絞り込み（検索日がある場合のみ）
        #   月単位: 記録日時が検索日の先頭7文字（"YYYY-MM"）で始まるもの / 日単位: 記録日時の日付部分が一致するもの
        if search_day:
          if mode == 'month':
            historys = historys.filter(timestamp__startswith=search_day[:7])
          else:
            historys = historys.filter(timestamp__date=search_day)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 操作日での絞り込み（検索日がある場合のみ）
    #   月単位：記録日時を日本時間で見て、検索日と同じ年・月のもの（__year / __month は TIME_ZONE で判定する）
    #   日単位：記録日時の日付部分が一致するもの
    if search_day:
      if mode == 'month':
        historys = historys.filter(timestamp__year=int(search_day[:4]), timestamp__month=int(search_day[5:7]))
      else:
        historys = historys.filter(timestamp__date=search_day)
```

直ったかの確かめ方: ✅ 直した写しでは、2 月で絞り込むと出て、1 月では出ませんでした。工数データの絞り込み（`work_day2__startswith`）は、時刻の無い「日付」なので、このずれは起きません。

6.15 B-15 思っていない値が送られると 500 になる

🟡 余裕のある時に ／ 前から残っている ／ ✅ 手元で再現

今: 画面からは正しい値しか送らない前提で書かれていて、値が無い・形が違う時に、500 になる所があります。✅ 手元で、次の 11 か所が 500 になることを確かめました（ログインの API と就業日の変更は、S-1・S-3 の直しで 400 にしています）。

API	500 になる時	場所
工数の保存（`kosu_new` の POST）	時刻（`time1`・`time2`）が無い	`kosu_views.py` 289〜290 行目
工数の編集（`kosu_update` の PUT）	何行目かの時刻が無い	`kosu_views.py` 1029〜1030 行目
時間帯の削除（`item_delete`）	時刻が無い	`kosu_views.py` 1186〜1187 行目
勤務のカレンダー（`kosu_calendar_change` → `kosu_calendar`）	年・月が数字でない（保存した後、カレンダーを開くと 500）	`kosu_views.py` 1336〜1337 行目
工数の集計（`kosu_total` の POST）	期間が「日間・月間・年間」でない・日付が読めない	`kosu_views.py` 1712・1719〜1745 行目
班員の週の移動（`team_calendar_week_jump`）	`week` が `A`・`B` でない	`team_views.py` 482〜488 行目
工数入力状況の Excel（`team_export`）	年・月が数字でない	`team_views.py` 517〜529 行目
残業の画面・ショップの画面の年月（`team_overtime`・`team_view` の POST）	年・月が数字でない（保存した後、開くと 500）	`team_views.py` 663〜664・757〜758 行目
ロード（`*_load`）	ファイルが付いていない	`asynchronous_views.py` 152 行目など 7 か所

直し方:

① 時刻と年月を読む関数を `main_utils.py` に足します（先に B-7 の①を入れてから、その後ろに足します）。

場所: `kosu/utils/main_utils.py` の 22 行目（import）

今:

コピー

```
import inspect
```

直した後（上の行を、全部これに置き換える）:

コピー

```
import datetime
import inspect
```

場所: `kosu/utils/main_utils.py` の 170 行目の前（ファイルの最後（get_settings の後ろ））

足すコード（この行の前に入れる）:

コピー

```
# 画面から届いた UTC の日時の文字列（例: "2026-11-08T23:30:00.000Z"）を、日本時間の "HH:MM" にする。
#
# 引数:
#   value: 日時の文字列（届いていない場合は None）
# 戻り値:
#   str: 日本時間の "HH:MM"（例: "08:30"）
# 例外:
#   ValueError: 届いていない・形が違う場合（呼ぶ側で捕まえて 400 を返す）
def utc_to_jst_hhmm(value):
  if not isinstance(value, str):
    raise ValueError('時刻がありません')
  utc_time = datetime.datetime.strptime(value, "%Y-%m-%dT%H:%M:%S.%fZ").replace(tzinfo=datetime.timezone.utc)
  return utc_time.astimezone(datetime.timezone(datetime.timedelta(hours=9))).strftime("%H:%M")



# 画面から届いた年・月を確かめて、数値の（年, 月）にする。
#
# 引数:
#   data: request.data（year・month を持つ）
# 戻り値:
#   (int, int): (年, 月)
# 例外:
#   ValueError: 数字でない・月が 1〜12 でない場合（呼ぶ側で捕まえて 400 を返す）
def parse_year_month(data):
  try:
    year, month = int(data.get('year')), int(data.get('month'))
  except (TypeError, ValueError):
    raise ValueError('年・月が数字ではありません')
  if not (2000 <= year <= 2100 and 1 <= month <= 12):
    raise ValueError('年・月の範囲が正しくありません')
  return year, month
```

② `kosu_views.py`

場所: `kosu/views/kosu_views.py` の 57 行目（import）

今:

コピー

```
from ..utils.main_utils import CustomPagination
```

直した後（上の行を、全部これに置き換える）:

コピー

```
from ..utils.main_utils import CustomPagination, utc_to_jst_hhmm, parse_year_month
```

場所: `kosu/views/kosu_views.py` の 285〜294 行目（KosuNew.post の中ほど）

今:

コピー

```
    # 作業時間から時間のみ抽出
    #   画面からは UTC の ISO形式（例: "2026-11-08T23:30:00.000Z"）で届くため、
    #   日本時間（UTC+9時間）に変換してから "HH:MM" を取り出す（例: → "08:30"）
    jst = datetime.timezone(datetime.timedelta(hours=9))
    start_time = datetime.datetime.strptime(post_data.get('time1'), "%Y-%m-%dT%H:%M:%S.%fZ")
    end_time = datetime.datetime.strptime(post_data.get('time2'), "%Y-%m-%dT%H:%M:%S.%fZ")
    start_time = start_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
    end_time = end_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
    start_time = start_time.strftime("%H:%M")
    end_time = end_time.strftime("%H:%M")
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 作業時間から時間のみ抽出
    #   画面からは UTC の ISO形式（例: "2026-11-08T23:30:00.000Z"）で届くため、
    #   日本時間（UTC+9時間）に変換してから "HH:MM" を取り出す（例: → "08:30"）
    #   届いていない・形が違う場合は 500 にせず 400 を返す
    try:
      start_time = utc_to_jst_hhmm(post_data.get('time1'))
      end_time = utc_to_jst_hhmm(post_data.get('time2'))
    except ValueError:
      return Response({'status': 'error', 'message': '作業時間を入力してください。'}, status=status.HTTP_400_BAD_REQUEST)
```

場所: `kosu/views/kosu_views.py` の 1023〜1034 行目（KosuUpdate.put の中ほど）

今:

コピー

```
    # 工数書き込み
    #   JST（日本時間 = UTC+9時間）のタイムゾーンを用意し、1行ずつ書き込む
    jst = datetime.timezone(datetime.timedelta(hours=9))
    if max_num:
      for n in range(max_num):
        # n+1 行目の開始・終了日時（UTC の ISO形式）を日本時間の "HH:MM" に変換する
        start_time = datetime.datetime.strptime(request.data.get(f'time1_{n + 1}'), "%Y-%m-%dT%H:%M:%S.%fZ")
        end_time = datetime.datetime.strptime(request.data.get(f'time2_{n + 1}'), "%Y-%m-%dT%H:%M:%S.%fZ")
        start_time = start_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
        end_time = end_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
        start_time = start_time.strftime("%H:%M")
        end_time = end_time.strftime("%H:%M")
```

直した後（上の行を、全部これに置き換える）：

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →
コピー

```
    # 工数書き込み（1行ずつ書き込む）
    if max_num:
      for n in range(max_num):
        # n+1 行目の開始・終了日時（UTC の ISO形式）を日本時間の "HH:MM" に変換する。形が違えば 400
        try:
          start_time = utc_to_jst_hhmm(request.data.get(f'time1_{n + 1}'))
          end_time = utc_to_jst_hhmm(request.data.get(f'time2_{n + 1}'))
        except ValueError:
          return Response({'status': 'error', 'message': f'{n + 1}行目の作業時間が正しくありません。'},
status=status.HTTP_400_BAD_REQUEST)
```

場所: `kosu/views/kosu_views.py` の 1182〜1191 行目（ItemDlete.post の中ほど）

今:

コピー

```
    # 項目番号から削除時間取得
    #   削除する時間帯の開始・終了日時（UTC の ISO形式）を日本時間の "HH:MM" にし、
    #   インデックス（時×12 + 分÷5）に変換する
    jst = datetime.timezone(datetime.timedelta(hours=9))
    start_time = datetime.datetime.strptime(request.data.get(f'time1'), "%Y-%m-%dT%H:%M:%S.%fZ")
    end_time = datetime.datetime.strptime(request.data.get(f'time2'), "%Y-%m-%dT%H:%M:%S.%fZ")
    start_time = start_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
    end_time = end_time.replace(tzinfo=datetime.timezone.utc).astimezone(jst)
    start_time = start_time.strftime("%H:%M")
    end_time = end_time.strftime("%H:%M")
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 項目番号から削除時間取得
    #   削除する時間帯の開始・終了日時（UTC の ISO形式）を日本時間の "HH:MM" にする。形が違えば 400
    try:
      start_time = utc_to_jst_hhmm(request.data.get('time1'))
      end_time = utc_to_jst_hhmm(request.data.get('time2'))
    except ValueError:
      return Response({'status': 'error', 'message': '削除する時間が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
```

場所: `kosu/views/kosu_views.py` の 1334〜1338 行目（KosuCalendarChange.post）

今:

コピー

```
def post(self, request):
  # セッション値取得（送信された年・月をそのままセッションに保存する）
  request.session['year'] = request.data.get('year')
  request.session['month'] = request.data.get('month')
  return Response({'status': 'success', 'message': 'カレンダーが更新されました。'})
```

直した後（上の行を、全部これに置き換える）:

コピー

```
def post(self, request):
  # 送信された年・月を確かめてからセッションに保存する（数字でない値を入れると、次の GET が 500 になるため）
  try:
    year, month = parse_year_month(request.data)
  except ValueError:
    return Response({'status': 'error', 'message': '年・月が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
  request.session['year'] = year
  request.session['month'] = month
  return Response({'status': 'success', 'message': 'カレンダーが更新されました。'})
```

場所: `kosu/views/kosu_views.py` の 1705〜1712 行目（KosuTotal.post の最初）

今:

コピー

```
  # 就業日取得
  #   送信された基準日を使い、空なら今日の日付を使ってセッションにも保存する
  #   day_object: 年・月での絞り込みに使うため、日付文字列を日時データに変換したもの
  day = request.data.get('date')
  if not day:
    day = str(datetime.date.today())
    request.session['day'] = day
  day_object = datetime.datetime.strptime(day, '%Y-%m-%d')
```

直した後（上の行を、全部これに置き換える）:

コピー

```
# 就業日取得
#   送信された基準日を使い、空なら今日の日付を使ってセッションにも保存する
#   day_object: 年・月での絞り込みに使うため、日付文字列を日時データに変換したもの
day = request.data.get('date')
if not day:
  day = str(datetime.date.today())
  request.session['day'] = day
# 集計期間が「日間・月間・年間」のどれでもない、または日付が読めない場合は 400 を返す
if request.data.get('period') not in ['日間', '月間', '年間']:
  return Response({'status': 'error', 'message': '集計期間を選んでください。'}, status=status.HTTP_400_BAD_REQUEST)
try:
  day_object = datetime.datetime.strptime(day, '%Y-%m-%d')
except (TypeError, ValueError):
  return Response({'status': 'error', 'message': '日付が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
```

③ `team_views.py` （ `parse_year_month` の `import` は、B-7 の②の `B-7-team-import` に入っています）

場所: `kosu/views/team_views.py` の 479〜488 行目（TeamCalendarWeekJump.post）

今:

コピー

```
# 表示日を文字列から日付に変換し、7日前 / 7日後にずらす
datetime_object = datetime.datetime.strptime(day, '%Y-%m-%d')
# "B"（Before）：1週間前へ
if request.data.get('week') == 'B':
  date_object = datetime_object.date() - datetime.timedelta(days=7)
# "A"（After）：1週間後へ
elif request.data.get('week') == 'A':
  date_object = datetime_object.date() + datetime.timedelta(days=7)
# ずらした日付を "YYYY-MM-DD" 文字列にしてセッションの表示日に保存する
request.session['day'] = str(date_object)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 表示日を文字列から日付に変換し、7日前 / 7日後にずらす
datetime_object = datetime.datetime.strptime(day, '%Y-%m-%d')
# "B"（Before）：1週間前へ
if request.data.get('week') == 'B':
  date_object = datetime_object.date() - datetime.timedelta(days=7)
# "A"（After）：1週間後へ
elif request.data.get('week') == 'A':
  date_object = datetime_object.date() + datetime.timedelta(days=7)
# "A" / "B" 以外は 400（前は date_object が無いまま進んで 500 になった）
else:
  return Response({'status': 'error', 'message': 'week は A か B を指定してください。'}, status=status.HTTP_400_BAD_REQUEST)
# ずらした日付を "YYYY-MM-DD" 文字列にしてセッションの表示日に保存する
request.session['day'] = str(date_object)
```

場所: `kosu/views/team_views.py` の 515〜519 行目（TeamExport.post の最初）

今:

コピー

```
    # 送信された対象年・月・ショップを取得
    post_data = request.data
    year = post_data.get('year')
    month = post_data.get('month')
    shop = post_data.get('shop2')
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 送信された対象年・月・ショップを取得（年・月は数字か確かめる。数字でなければ 400）
    post_data = request.data
    try:
      year, month = parse_year_month(post_data)
    except ValueError:
      return Response({'status': 'error', 'message': '年・月が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
    shop = post_data.get('shop2')
```

場所: `kosu/views/team_views.py` の 661〜665 行目（TeamOverTime.post）

今:

コピー

```
  def post(self, request):
    # 送信された年・月をセッションに保存する（次回の GET でその月が表示される）
    request.session['year'] = request.data.get('year')
    request.session['month'] = request.data.get('month')
    return Response({'status': 'success', 'message': 'カレンダーが更新されました。'})
```

直した後（上の行を、全部これに置き換える）：

コピー

```
def post(self, request):
  # 送信された年・月を確かめてからセッションに保存する（次回の GET でその月が表示される）
  try:
    year, month = parse_year_month(request.data)
  except ValueError:
    return Response({'status': 'error', 'message': '年・月が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
  request.session['year'] = year
  request.session['month'] = month
  return Response({'status': 'success', 'message': 'カレンダーが更新されました。'})
```

場所: `kosu/views/team_views.py` の 755〜759 行目（TeamView.post）

今:

コピー

```
def post(self, request):
  # 送信された年・月をセッションに保存する（TeamOverTime と同じキーのため表示年月は共通になる）
  request.session['year'] = request.data.get('year')
  request.session['month'] = request.data.get('month')
  return Response({'status': 'success', 'message': 'カレンダーが更新されました。'}, status=status.HTTP_200_OK)
```

直した後（上の行を、全部これに置き換える）：

コピー

```
def post(self, request):
  # 送信された年・月を確かめてからセッションに保存する（TeamOverTime と同じキーのため表示年月は共通になる）
  try:
    year, month = parse_year_month(request.data)
  except ValueError:
    return Response({'status': 'error', 'message': '年・月が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)
  request.session['year'] = year
  request.session['month'] = month
  return Response({'status': 'success', 'message': 'カレンダーが更新されました。'}, status=status.HTTP_200_OK)
```

④ `asynchronous_views.py`（ロードでファイルが付いていない時）

場所: `kosu/views/asynchronous_views.py` の 132 行目（backup の URL 名を調べた直後）

今:

コピー

```
    url_name = match.url_name
```

直した後（上の行を、全部これに置き換える）：

コピー

```
  url_name = match.url_name
  # ロード（*_load）なのにファイルが添付されていない場合は 400（前は None.chunks() で 500 になった）
  if url_name.endswith('_load') and not request.FILES.get('file'):
      return JsonResponse({'status': 'error', 'message': 'ファイルを選択してください。'}, status=status.HTTP_400_BAD_REQUEST)
```

> **▶ ここで初めて出てくるコード**
>
> - **`.endswith()`**（Python）：文字列が、その文字で終わっていれば `True` を返すメソッド。

直ったかの確かめ方: ✅ 直した写しでは、上の 11 か所が全部 400 になり、画面には理由の文が出ます。

6.16 B-16 月間・年間の集計で、最初の 1 件の定義 Ver で全部を数える

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `kosu/views/kosu_views.py` 1755〜1760 行目（`KosuTotal.post`）

今: 月間・年間の集計では、期間の中の工数を、最初の 1 件の定義 Ver の工数区分名で数えます。途中で定義の Ver を変えた月は、区分の記号（A・B…）が同じでも中身の違う物を、同じ区分として足してしまいます。

直し方（まず、気付けるようにする）: 期間の中に違う Ver が混ざっている時は、サーバーが注意の文（`warning`）を返し、集計の画面に出します。Ver ごとに分けて数えるかどうかは、使う人と決めてください。

場所: `kosu/views/kosu_views.py` の 1715 行目（KosuTotal.post）

今:

コピー

```
    member_data = request.user
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    member_data = request.user
    # 画面に出す注意（違う Ver が混ざっている時だけ入れる）
    warning = None
```

場所: `kosu/views/kosu_views.py` の 1755〜1760 行目（KosuTotal.post の中ほど）

今:

コピー

```
else:
  first_kosu_with_def = kosu_data.exclude(def_ver2__isnull=True).exclude(def_ver2__exact='').order_by('id').first()
  if first_kosu_with_def:
    def_ver_to_filter = first_kosu_with_def.def_ver2
  else:
    def_ver_to_filter = def_ver_session
```

> **▶ ここで初めて出てくるコード**
>
> - `.exclude()`（Django）: 問い合わせから、条件に合う行を除くメソッド（`filter` の逆）。

直した後（上の行を、全部これに置き換える）:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →

```
    else:
      first_kosu_with_def =
kosu_data.exclude(def_ver2__isnull=True).exclude(def_ver2__exact='').order_by('id').first()
      if first_kosu_with_def:
        def_ver_to_filter = first_kosu_with_def.def_ver2
      else:
        def_ver_to_filter = def_ver_session
      # 期間の中に、違う工数区分定義Verで入力した日が混ざっている場合は、画面に注意を出す
      #   （集計は最初の1件のVerの工数区分名で数えるため、混ざっていると区分名と中身がずれる）
      ver_count =
kosu_data.exclude(def_ver2__isnull=True).exclude(def_ver2__exact='').values('def_ver2').distinct().count()
      if ver_count > 1:
        warning = f'この期間には {ver_count} 種類の工数区分定義Verの工数が混ざっています。区分ごとの合計は
「{def_ver_to_filter}」の区分名で数えています。'
```

▶ ここで初めて出てくるコード

- `.distinct()`（Django）：問い合わせの結果から、重なった値を 1 つにするメソッド。`.values_list('def_ver2', flat=True).distinct()` は、使われている定義の Ver 名を、重ならないように並べます。

場所: `kosu/views/kosu_views.py` の 1779〜1785 行目（KosuTotal.post の最後）

今:

```
    response_data = {
      'member_data': member_serializer.data,
      'kosu_data': kosu_serializer.data,
      'def_data': def_serializer.data,
      'session_day': day,
    }
    return Response(response_data)
```

直した後（上の行を、全部これに置き換える）：

```
    response_data = {
      'member_data': member_serializer.data,
      'kosu_data': kosu_serializer.data,
      'def_data': def_serializer.data,
      'session_day': day,
    }
    if warning:
      response_data['warning'] = warning
    return Response(response_data)
```

画面（`KosuTotal.tsx`）にも、注意を出す所を足します。

場所: `frontend/src/KosuPage/KosuTotal.tsx` の 79〜80 行目（ApiResponseData）

今:

コピー

```
  session_day: string; // 基準となる就業日（"YYYY-MM-DD"）
}
```

直した後（上の行を、全部これに置き換える）:

コピー

```
  session_day: string; // 基準となる就業日（"YYYY-MM-DD"）
  warning?: string; // 注意（期間の中に違う工数区分定義Verの工数が混ざっている時だけ届く）
}
```

場所: `frontend/src/KosuPage/KosuTotal.tsx` の 461 行目（工数MENU のリンクの後ろ）

今:

コピー

```
      </nav>
```

直した後（上の行を、全部これに置き換える）:

コピー

```
      </nav>
      {/* 注意: 期間の中に違う工数区分定義Verの工数が混ざっている時だけ、サーバーから届いた文を出す */}
      {apiData.warning && (
        <div role="alert">{apiData.warning}</div>
      )}
```

6.17 B-17 操作履歴を 1 件保存するたびに、全件を数えている

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `kosu/models.py` 649〜661 行目（`History.save`）

今: 操作履歴（`History`。データの作成・変更・削除のたびに 1 件書かれる）を保存するたびに、表の全件数を数えています（上限は 50 万件）。記録が増えるほど、どの画面の保存も遅くなります。

直し方: 数えるのを 1,000 件に 1 回にします。

場所: `kosu/models.py` の 649〜656 行目（History.save）

今:

コピー

```
def save(self, *args, **kwargs):
  # まず通常どおりデータベースに保存する（super() は親クラス models.Model を指す）
  super().save(*args, **kwargs)
  # 保存後のこのテーブルの全件数を数える
  current_count = self.__class__.objects.count()

  # 件数が上限（MAX_RECORDS）を超えた場合の処理
  if current_count > self.MAX_RECORDS:
```

直した後（上の行を、全部これに置き換える）:

コピー

```
def save(self, *args, **kwargs):
  # まず通常どおりデータベースに保存する（super() は親クラス models.Model を指す）
  super().save(*args, **kwargs)
  # 全件を数えるのは 1,000 件に 1 回だけにする
  #   （毎回数えると、記録が増えるほど、どの画面の保存も遅くなる。上限を少し超える間があっても困らない）
  if self.pk % 1000 != 0:
    return
  # 保存後のこのテーブルの全件数を数える
  current_count = self.__class__.objects.count()

  # 件数が上限（MAX_RECORDS）を超えた場合の処理
  if current_count > self.MAX_RECORDS:
```

▶ ここで初めて出てくるコード

- `%`（Python）: Python の `7 % 2` は、割った余り（`1`）です。

6.18 B-18 問い合わせのお知らせが、別の人の問い合わせを指すことがある

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `kosu/views/inquiry_views.py` 170〜192 行目（`InquirNew.post`）

今: 担当者向けのお知らせに入れる問い合わせの ID を、「ID が最大の問い合わせ」（178 行目）で探しています。2 人がほぼ同時に登録すると、別の人の問い合わせを指します。すぐ上で作った `new_inquiry` の ID を使えば済みます（先に B-7 の①を入れてください）。

場所: `kosu/views/inquiry_views.py` の 170〜192 行目（InquirNew.post の後半）

今:

コピー

```
# 最新（IDが最大）の管理者設定を取得
#   ※ last() はデータが無いとき例外ではなく None を返すため、この except に入ることはない
try:
  admin_data = administrator_data.objects.order_by("id").last()
except administrator_data.DoesNotExist:
  return Response({'status': 'error', 'message': '設定データが見つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)

# IDが最大の問い合わせ（＝直前に登録した問い合わせ）を取得し、お知らせのリンク先IDに使う
inquiry_data_id = inquiry_data.objects.order_by('id').last()
# 問い合わせ担当者向けお知らせの書き込み:
#   pop_up1 → pop_up5 の順に確認し、最初に見つかった空き枠に文言と問い合わせIDを書き込んで終了する
for i in range(1, 6):
  pop_up_field = f'pop_up{i}'
  pop_up_id_field = f'pop_up_id{i}'
  if getattr(admin_data, pop_up_field) in ["", None]:
    # 管理者設定（同じID）のレコードを更新する
    administrator_data.objects.update_or_create(
      id=admin_data.id,
      defaults={
        pop_up_id_field: inquiry_data_id.id,
        pop_up_field: f'{member_data.name}さんからの新しい問い合わせがあります。'
      }
    )
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 最新（IDが最大）の管理者設定を取得（無ければ初期値で作る）
admin_data = get_settings()

# 問い合わせ担当者向けお知らせの書き込み:
#   pop_up1 → pop_up5 の順に確認し、最初に見つかった空き枠に、文言と「いま登録した問い合わせのID」を書き込んで終了する
#   （前は「IDが最大の問い合わせ」を探していたので、同時に登録した別の人の問い合わせを指すことがあった）
for i in range(1, 6):
  pop_up_field = f'pop_up{i}'
  pop_up_id_field = f'pop_up_id{i}'
  if getattr(admin_data, pop_up_field) in ["", None]:
    # 管理者設定（同じID）のレコードを更新する
    administrator_data.objects.update_or_create(
      id=admin_data.id,
      defaults={
        pop_up_id_field: new_inquiry.id,
        pop_up_field: f'{member_data.name}さんからの新しい問い合わせがあります。'
      }
    )
```

6.19 B-19 従業員番号を変えると、その人の工数・班・問い合わせが切り離される

🟠 早めに ／ 前から残っている ／ 📖 コードから（直した後は ✅）

どこ: `kosu/views/member_views.py` 272〜291 行目（`MemberUpdate.put`）

今: 工数（`employee_no3`）・問い合わせ（`employee_no2`）・班（`employee_no5`・`member1`〜`member15`）は、人を **従業員番号の数字** で覚えています。人員の編集で従業員番号を変えても、それらは古い番号のままなので、その人の工数などが見えなくなります。

直し方: S-3 の⑤の `MemberUpdate` の直しに入っています（番号を変えた時は、同じトランザクションの中で、工数・問い合わせ・班の番号もそろえる）。気を付けること:

- `.update()` でそろえるので、操作履歴（`History`）には、そろえた分は記録されません。

- 管理者の設定の「問い合わせ担当者の従業員番号」（管理者 MENU ⇒ 設定）は、手で直してください。
- その人がログイン中なら、セッションの番号が古いままなので、ログインし直してもらいます。

6.20 B-20 `.gitattributes` のせいで、PDF・Excel などのファイルが壊れる

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

どこ: `.gitattributes`（4 行）

今: `* text eol=crlf` で、**全部のファイルを文字のファイル（テキスト）として扱い**、行末の記号を書き換えます。`binary`（そのまま保存）にしているのは `*.png` と `*.ico` だけです。PDF・Excel・圧縮ファイル・フォントの中に、行末の記号と同じ並びのバイトがあると、コミットの時に書き換わって壊れます。

✅ **手元で確かめた結果:** 11 月 9 日のコミットで足された `docs/pdf/README.pdf` を、GitHub に記録されている中身（`git show`）で取り出すと、PDF を読む道具（PyMuPDF）が「壊れているので直しながら読む」と言い、中の圧縮の誤り（zlib error）が多数出ました。手元（この PC）の作業フォルダにある元のファイルは正常でした（コミットの時に、行末の記号が 9 か所書き換わっていた）。`git check-attr` でも、`.xlsx`・`.pdf`・`.gz`・`.coverage`・`.vs/slnx.sqlite` が `text` 扱いでした。

直し方:

① `.gitattributes` を、Git が中身を見て判定する形（`text=auto`）にし、壊れやすい種類を `binary` にします。

場所: `.gitattributes` の 1〜4 行目（.gitattributes 全体）

今:

コピー

```
# 行末記号をWindows(CRLF)で明示
* text eol=crlf
*.png binary
*.ico binary
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# 行末記号: 文字のファイル（テキスト）は Git が判定し、作業フォルダでは Windows の CRLF にそろえる
#   （「* text」で全部をテキスト扱いにすると、Excel・PDF・画像・フォントなどの中身が書き換わって壊れる）
* text=auto eol=crlf
# 中身をそのまま保存するファイル（行末記号を変えない・差分を文字で出さない）
*.png binary
*.jpg binary
*.ico binary
*.gif binary
*.svg binary
*.pdf binary
*.xlsx binary
*.xls binary
*.gz binary
*.zip binary
*.woff binary
*.woff2 binary
*.ttf binary
*.eot binary
*.sqlite binary
*.sqlite3 binary
```

② 直した後、記録の中身を、新しい決まりで付け直します。

コピー

```
git add --renormalize .
git status
git commit -m "B-20: .gitattributes を text=auto にし、バイナリを書き換えないようにする"
```

▶ ここで初めて出てくるコード

- **`git add --renormalize`**（Git）：`git add --renormalize .` は、記録されているファイルを、今の `.gitattributes` の決まり（改行の扱いなど）で記録し直すコマンド。`.gitattributes` を変えた後に使います。

③ すでに壊れて記録された PDF などは、元のファイル（壊れていない物）から入れ直します。`docs/pdf/` の PDF は作り直せる物なので、作り直してから `git add` します（D-14 で、そもそもリポジトリに入れるかを決める）。

6.21 B-21 Excel の見出し「ポップアップID6」の中身が ID5

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `kosu/tasks.py` 398・1188 行目（バックアップの見出し）、479・1277 行目（ロードの時に確かめる見出し）

今: 人員と設定のバックアップの Excel で、5 番目のお知らせの ID の列の見出しが「ポップアップID6」になっています（中身は ID5。386・1172 行目のコメントにも書いてある）。ロードも同じ見出しを確かめているので、動きは合っていますが、見た人が迷います。

直し方: 見出しを「ポップアップID5」にし、ロードでは、古い見出しの Excel も読めるように読み替えます（今までに取ったバックアップが、ロードできなくならないように）。

場所: `kosu/tasks.py` の 398 行目（generate_member_backup の見出し）

今の行（`'ポップアップ5', 'ポップアップID6', '休憩エラー有効チェック',`）を、次に置き換える:

コピー

```
    'ポップアップ5', 'ポップアップID5', '休憩エラー有効チェック',
```

場所: `kosu/tasks.py` の 479 行目（load_member_file の見出し）

今の行（`'ポップアップ5', 'ポップアップID6', '休憩エラー有効チェック',`）を、次に置き換える:

コピー

```
    'ポップアップ5', 'ポップアップID5', '休憩エラー有効チェック',
```

場所: `kosu/tasks.py` の 483 行目（load_member_file）

今の行（`actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]`）を、次に置き換える:

コピー

```
    actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]
    # 前の見出し「ポップアップID6」で作った古いバックアップも読めるよう、ID5 に読み替えてから比べる
    actual_headers = ['ポップアップID5' if h == 'ポップアップID6' else h for h in actual_headers]
```

場所: `kosu/tasks.py` の 1188 行目（generate_setting_backup の見出し）

今の行（`'ポップアップID6',`）を、次に置き換える:

コピー

```
    'ポップアップID5',
```

場所: `kosu/tasks.py` の 1277 行目（load_setting_file の見出し）

今の行（`'ポップアップID6',`）を、次に置き換える:

コピー

```
    'ポップアップID5',
```

場所: `kosu/tasks.py` の 1281 行目（load_setting_file）

今の行（`actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]`）を、次に置き換える:

コピー

```
    actual_headers = [ws.cell(1, col).value for col in range(1, len(expected_headers) + 1)]
    # 前の見出し「ポップアップID6」で作った古いバックアップも読めるよう、ID5 に読み替えてから比べる
    actual_headers = ['ポップアップID5' if h == 'ポップアップID6' else h for h in actual_headers]
```

---

7. 開発・運用の整理（D）

7.1 D-1 GitHub Actions が、テストを流さずに本番へ出す作りのまま

🟠 早めに ／ 前から残っている ／ 📖 コードから（止めてあることは GitHub で確かめた）

どこ: `.github/workflows/master_hozen-kosu-react.yml`（55 行）

今:

- `master` に push すると、テストを流さずに、ビルドして本番に出します。
- Node.js が 20（2026 年 4 月でサポートが終わった。D-8）、`npm install`（`package-lock.json` と違う版が入ることがある）です。
- データベースのパスワードなどの秘密の値を、ジョブ全体（11〜17 行目）に渡しています。ビルドと `collectstatic` は settings.py を読むだけで、本物の値は要りません（本番の値は、Azure の構成から読まれる）。
- **今は、GitHub の画面で手動で止めてあります**（2026 年 11 月 9 日に GitHub の API で確かめた。状態は `disabled_manually`）。最後に動いたのは 9 月 3 日です。

直し方: ワークフローを、「テスト（`test`）→ テストが通った時だけ本番へ出す（`build-and-deploy`）」の 2 段にします。Pull Request の時は、テストだけ流します。ファイルを全部、次に置き換えます。

場所: `.github/workflows/master_hozen-kosu-react.yml` の 1〜55 行目（ファイル全体）

今:

コピー

```
name: Deploy Django + React to Azure
on:
  push:
    branches:
      - master
  workflow_dispatch:
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    env:
      SECRET_KEY: ${{ secrets.SECRET_KEY }}
      DEBUG: ${{ secrets.DEBUG }}
      DB_NAME: ${{ secrets.DB_NAME }}
      DB_USER: ${{ secrets.DB_USER }}
      DB_PASSWORD: ${{ secrets.DB_PASSWORD }}
      DB_HOST: ${{ secrets.DB_HOST }}
      HELP_PATH: ${{ secrets.HELP_PATH }}
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "20"
      - name: Install React dependencies
        run: |
          cd frontend
          npm install
      - name: Build React
        run: |
          cd frontend
          npm run build
      - name: Install Python dependencies
        run: |
          python -m venv antenv
          source antenv/bin/activate
          pip install --upgrade pip
          pip install -r requirements.txt
      - name: Collect static files
        run: |
          source antenv/bin/activate
          python manage.py collectstatic --noinput
      - name: Create deployment zip
        run: |
          zip -r release.zip . -x ".git/*" "node_modules/*" "frontend/node_modules/*"
      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v3
        with:
```

```
          app-name: hozen-kosu-React
          publish-profile: ${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_<今の名前のまま> }}
          package: release.zip
```

▶ ここで初めて出てくるコード

- **`name:`**（GitHub Actions）：`name:` は、名前です。いちばん上の `name: Deploy Django + React to Azure` は、ワークフロー（自動の手順）の名前、`- name: Build React` は、手順 1 つの名前で、GitHub の Actions の画面に表示されます。
- **`on:`**（GitHub Actions）：`on:` は、「いつ動かすか」です。`push:`（送った時）・`pull_request:`（プルリクエストの時）・`schedule:`（決まった時刻）などを並べます。
- **`push:`**（GitHub Actions）：`push:` は、「リポジトリに送った（push）時に動かす」指定です。
- **`branches:`**（GitHub Actions）：`branches:` の下に、ブランチの名前を並べて、「そのブランチに送った時だけ」としぼります。このアプリは `- master` なので、`master` に push した時だけ動きます。
- **`workflow_dispatch:`**（GitHub Actions）：`on:` の下の `workflow_dispatch:` は、「GitHub の Actions の画面の『Run workflow』ボタンで、手で動かせるようにする」指定です。
- **`jobs:`**（GitHub Actions）：`jobs:` の下に、動かす仕事（ジョブ）を並べます。ジョブは、ふつう同時に動きます。
- **`build-and-deploy:`**（GitHub Actions）：`jobs:` の下の `build-and-deploy:` は、仕事（ジョブ）の名前です。名前は自分で決めます。このアプリは、build・テスト・本番に出す、を 1 つの仕事でやっています。
- **`runs-on:`**（GitHub Actions）：`runs-on: ubuntu-latest` は、その仕事を、どの OS の機械で動かすかです。GitHub が用意した Linux（Ubuntu）の機械を、その時だけ借ります。
- **`env:`**（GitHub Actions）：`env:` の下に `名前: 値` を並べると、その手順（または仕事全体）の中だけで使える環境変数になります。手順の下に書けば、その手順の中だけで見えます。
- **`SECRET_KEY:`**（GitHub Actions）：`env:` の下の `SECRET_KEY: …` は、`SECRET_KEY` という環境変数を作る行です。Django の `settings.py` は、起動の時にこの値を読みます（無いと動きません）。
- **`${{ … }}`**（GitHub Actions）：`${{ }}` の中に書いた物は、GitHub Actions が、動かす時に値に置き換えます。`${{ secrets.SECRET_KEY }}` は、リポジトリの Secrets に入れた `SECRET_KEY` の値、`${{ github.event_name }}` は、何をきっかけに動いたか（`push`・`pull_request` など）です。
- **`secrets.名前`**（GitHub Actions）：`secrets.SECRET_KEY` のように書くと、リポジトリの Settings → Secrets and variables → Actions に入れた、秘密の値を読めます。値は画面やログには出ません（`***` になります）。
- **`DEBUG:`**（GitHub Actions）：`env:` の下の `DEBUG: "False"` は、`DEBUG` という環境変数を作る行です。`"False"` と引用符で囲むのは、YAML が `False` を「文字」ではなく「真偽の値」と読まないようにするためです。
- **`DB_NAME:`**（GitHub Actions）：`env:` の下の `DB_NAME: ${{ secrets.DB_NAME }}` は、`DB_NAME` という環境変数に、GitHub に登録した秘密の値（`secrets`）を入れる行です。`DB_USER`・`DB_PASSWORD`・`DB_HOST` も同じ形です。
- **`DB_USER:`**（GitHub Actions）：`env:` の下の `DB_USER: ${{ secrets.DB_USER }}` は、データベースにつなぐ人の名前を、GitHub に登録した秘密の値から入れる行です。
- **`DB_PASSWORD:`**（GitHub Actions）：`env:` の下の `DB_PASSWORD: ${{ secrets.DB_PASSWORD }}` は、データベースのパスワードを、GitHub に登録した秘密の値から入れる行です。

- **`DB_HOST:`**（GitHub Actions）：`env:` の下の `DB_HOST: ${{ secrets.DB_HOST }}` は、データベースのサーバーの住所を、GitHub に登録した秘密の値から入れる行です。
- **`HELP_PATH:`**（GitHub Actions）：`env:` の下の `HELP_PATH: …` は、ヘルプ画面のパスワードを入れる環境変数を作る行です。テストの手順では、テストのためだけの値を入れます。
- **`steps:`**（GitHub Actions）：`steps:` の下に、その仕事の手順を、上から順に `-` で並べます。
- **`-`**（GitHub Actions）：YAML の行の頭の `-` は、「並びの 1 つ」を表します。`steps:` の下の `- name: …` は、手順の 1 つ目・2 つ目…です。字下げ（空白の数）で、どこに入るかが決まるので、そろえて書きます。
- **`uses:`**（GitHub Actions）：`uses: actions/checkout@v4` は、ほかの人が作った決まった手順（アクション）を使う指定です。`actions/checkout` は、リポジトリのファイルを取ってくる手順、`@v4` はその版です。`actions/setup-python` は Python を、`actions/setup-node` は Node.js を入れる手順です。
- **`with:`**（GitHub Actions）：`with:` は、`uses:` で使うアクションに渡す設定です。
- **`python-version:`**（GitHub Actions）：`actions/setup-python` に渡す設定で、入れる Python の版です。`"3.12"` と引用符で囲むのは、`3.10` が数の `3.1` と読まれるような間違いを防ぐためです。
- **`node-version:`**（GitHub Actions）：`actions/setup-node` に渡す設定で、入れる Node.js の版です。`"22"` なら、22 系の新しい版が入ります。
- **`run:`**（GitHub Actions）：`run:` は、その手順で動かすコマンドです。`run: |` と書くと、次の行から何行も書けます（`cd frontend` の次の行に `npm ci` 、のように）。
- **`|`**（GitHub Actions）：`run: |` の `|` は、「次の行から、字下げした所までを、改行も含めてそのまま 1 つの文字にする」という印です。コマンドを何行も書く時に使います。
- **`app-name:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、送り先の Azure App Service の名前です（このアプリは `hozen-kosu-React` ）。
- **`publish-profile:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、Azure に送るための「通行証」（発行プロファイル）です。値は秘密なので、Secrets から読みます。
- **`package:`**（GitHub Actions）：`azure/webapps-deploy` に渡す設定で、送るファイルです。このアプリは、全部を 1 つにまとめた `release.zip` を送ります。
- **`source`**（WSL・bash）：`source antenv/bin/activate` は、Python の仮想環境（ `antenv` ）に入るコマンド（Linux・Mac の書き方）。入ると、`python` ・ `pip` が、その仮想環境の物になります。
- **`pip install --upgrade`**（Python の道具）：`pip install --upgrade pip` は、部品をいちばん新しい版にするコマンド。ここでは `pip` 自身を新しくしています。
- **`python --noinput`**（Django の道具）：`python manage.py collectstatic --noinput` の `--noinput` は、「上書きしてよいか」などの質問をせずに進める、という指定です。人が答えられない GitHub Actions の中で使います。
- **`zip`**（WSL・bash）：ファイルやフォルダを、1 つの zip ファイルにまとめるコマンド。`zip -r release.zip .` は、今いるフォルダの中を全部、`release.zip` にまとめます。
- **`zip -r`**（WSL・bash）：`zip` の `-r` は、フォルダの中のフォルダまで、全部まとめるという指定です（recursive）。

- **`zip -x`**（WSL・bash）：`zip` の `-x` の後ろに書いた物は、まとめる物から外します。`-x ".git/*" "node_modules/*"` は、Git の記録と部品のフォルダを入れない、という意味です。

直した後（上の行を、全部これに置き換える）：

コピー

```
name: Deploy Django + React to Azure
on:
  push:
    branches:
      - master
  pull_request:            # PR を出した時は、テストだけ流す（本番には出さない）
    branches:
      - master
  workflow_dispatch:
jobs:
  # ---- 1. テスト（失敗したら、ここで止まり、本番には出ない）
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"   # Node 20 はサポートが終わったので 22（LTS）にする
      - name: Backend tests
        env:                   # テスト用の値（本物の秘密ではない）
          SECRET_KEY: test-only-not-a-secret
          DEBUG: "False"
          HELP_PATH: test-only
        run: |
          pip install -r requirements.txt
          python manage.py test kosu --settings=hozen_another.test_settings
      - name: Frontend tests and build
        run: |
          cd frontend
          npm ci
          npm run test:run
          npm run build
  # ---- 2. 本番へ出す（テストが全部通った時だけ。PR の時は出さない）
  build-and-deploy:
    needs: test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: "22"
      - name: Build React
        run: |
          cd frontend
          npm ci                # package-lock.json のとおりの版を入れる（npm install だと版が変わることがある）
          npm run build
      - name: Install Python dependencies
        run: |
          python -m venv antenv
          source antenv/bin/activate
          pip install --upgrade pip
```

```
          pip install -r requirements.txt
      - name: Collect static files
        env:                    # collectstatic は settings.py を読むだけなので、秘密の値は要らない
          SECRET_KEY: build-only-not-a-secret
          DEBUG: "False"
        run: |
          source antenv/bin/activate
          python manage.py collectstatic --noinput
      - name: Create deployment zip
        run: |
          zip -r release.zip . -x ".git/*" "node_modules/*" "frontend/node_modules/*"
      - name: Deploy to Azure Web App
        uses: azure/webapps-deploy@v3
        with:
          app-name: hozen-kosu-React
          publish-profile: ${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_<今の名前のまま> }}
          package: release.zip
```

▶ ここで初めて出てくるコード

- **`pull_request:`**（GitHub Actions）：`pull_request:` は、「プルリクエスト（変更を取り込んでほしいというお願い）が作られた・更新された時に動かす」指定です。
- **`#`**（GitHub Actions）：YAML（GitHub Actions の設定などの書き方）では、`#` から行の終わりまでがコメントです。
- **`test:`**（GitHub Actions）：`jobs:` の下の `test:` は、仕事（ジョブ）に付けた名前です（自分で決めます）。
- **`needs:`**（GitHub Actions）：`needs: test` は、「`test` の仕事が成功してから動かす」という順番の指定です。
- **`if:`**（GitHub Actions）：`if:` に条件を書くと、その条件が合う時だけ、その手順を動かします。`if: github.event_name != 'pull_request'` は「Pull Request がきっかけでない時だけ」（＝ master に push した時だけ本番に出す）。
- **`npm ci`**（Node.js）：`package-lock.json` に書かれたとおりの版で、部品を入れ直すコマンド（`node_modules` を一度消してから入れる）。`npm install` と違い、`package-lock.json` を書きかえないので、いつ・どこで入れても同じになります。

`publish-profile` の行（いちばん下から 2 行目）の `secrets.` の後ろの名前は、今のファイルに書いてある名前のまま使ってください（この資料では一部を伏せています）。

置き換えたら:

1. Azure の構成（App Service → 設定 → 環境変数）に、`SECRET_KEY`・`DEBUG`・`DB_NAME`・`DB_USER`・`DB_PASSWORD`・`DB_HOST`・`HELP_PATH` が入っていることを、前任者と確かめます（今のワークフローは、これらを本番に送っていません。本番はここから読んでいるはずです）。
2. ブランチで直して Pull Request を出し、Actions のタブで `test` が緑になることを確かめます。

3. S-1 などの段 1（§9）が `master` に入ってから、Actions のタブで、ワークフローを「Enable workflow」で動かします。

7.2 D-2 migrate の決まりが無い。モデルとマイグレーションが 26 か所食い違っている

🟠 早めに ／ 前から残っている ／ ✅ 手元で再現

今:

- 本番のデータベースに `migrate` をする手順が、どこにもありません（Actions でもしていない）。
- ✅ 手元で `python manage.py makemigrations kosu --check` を打つと、「マイグレーションに書かれていない変更がある」と言われました。中身は 26 か所の `AlterField`（`administrator_data`・`member` のお知らせの項目に `max_length=255` を付けた、など。`models.py` には 4 月より前から入っていた）です。本番の表の形が、`models.py` と違っている恐れがあります。

> `makemigrations` の結果は、ふつうは画面に出ますが、このアプリでは出ません。`main_views.py` が `print` の行き先をログのファイルに差し替えているためです（D-9）。結果は `web_console.log` に入ります。

直し方:

① 食い違いを、マイグレーションのファイルにします。`python manage.py makemigrations kosu -n sync_models` を打つと、次のファイルができます（作った物を、そのまま載せています）。

新しいファイル: `kosu/migrations/0013_sync_models.py`（python manage.py makemigrations kosu -n sync_models で作られる物）

中身（全部）:

コピー

```
# Generated by Django 4.2.7 on 2026-11-09 15:29

from django.db import migrations, models


class Migration(migrations.Migration):

    dependencies = [
        ('kosu', '0012_def_choice'),
    ]

    operations = [
        migrations.AlterField(
            model_name='administrator_data',
            name='administrator_employee_no1',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='問い合わせ担当者従業員番号1'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='administrator_employee_no2',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='問い合わせ担当者従業員番号2'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='administrator_employee_no3',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='問い合わせ担当者従業員番号3'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up1',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ1'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up2',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ2'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up3',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ3'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up4',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ4'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up5',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ5'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up_id1',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID1'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up_id2',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID2'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up_id3',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID3'),
        ),
```

```
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up_id4',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID4'),
        ),
        migrations.AlterField(
            model_name='administrator_data',
            name='pop_up_id5',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID5'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up1',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ1'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up2',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ2'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up3',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ3'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up4',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ4'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up5',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップ5'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up_id1',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID1'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up_id2',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID2'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up_id3',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID3'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up_id4',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID4'),
        ),
        migrations.AlterField(
            model_name='member',
            name='pop_up_id5',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ポップアップID5'),
        ),
        migrations.AlterField(
            model_name='operation_history',
            name='operation_models',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='編集したモデル'),
        ),
        migrations.AlterField(
            model_name='operation_history',
            name='post_page',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='ページ'),
```

```
        ),
        migrations.AlterField(
            model_name='operation_history',
            name='status',
            field=models.CharField(blank=True, max_length=255, null=True, verbose_name='結果'),
        ),
    ]
```

> **▶ ここで初めて出てくるコード**
>
> - `.AlterField()`（Django）：マイグレーションの中で、表の列の形（文字数の上限・空を許すか、など）を変える操作。`models.py` の項目の書き方を変えると、`makemigrations` が書きます。

② 本番への `migrate` は、だれが・いつ・どこで打つかを、前任者と決めます。おすすめは、本番に出した直後に、Azure の SSH（000 巻 §10）で、次を打つことです。

コピー

```
python manage.py showmigrations kosu   # どこまで当たっているかを見る（[X] が当たっている）
python manage.py migrate kosu          # 残りを当てる
```

打つ前に、本番のデータベースのバックアップを取ります（§4.2）。B-6 の `0014` は、重なりを調べてから当てます。

7.3 D-3 リポジトリに、入れなくてよい物が 1,291 ファイル（212 MB）ある

🟡 余裕のある時に ／ 前から残っている ／ ✅ 数えた

今: `ec2cedc` のリポジトリは 1,581 ファイル・220 MB です。そのうち、次の物は、作り直せる・動かすと増える・人の PC の物で、リポジトリに入れなくてよい物です。

物	ファイル数	大きさ	何か
`staticfiles/`	1,035	124.2 MB	`collectstatic` の結果。Actions が毎回作り直す
`frontend/node-v22.17.1-linux-x64.tar.xz`	1	29.1 MB	Linux 用の Node.js の一式。どこからも使っていない
`docs/pdf/`	15	25.0 MB	`docs/` の資料の PDF（B-20 で壊れている。D-14）
`frontend/build/`	25	15.5 MB	画面のビルドの結果（8 月の Create React App の物のまま）。Actions が毎回作り直す
`.vs/`	13	8.6 MB	Visual Studio の作業用のファイル（コードの索引のデータベースも入っている）
`node_modules/`（リポジトリの直下）	171	6.2 MB	直下で `npm install` した時の物（`chart.js` など。画面の部品は `frontend/` にある）
`frontend/dist/`	16	3.7 MB	9 月に Vite が出力先を決める前のビルドの結果
`media/`	9	0.04 MB	テストが作ったバックアップの Excel（S-9）
`.coverage`・`web_console.log`・`data.xlsx`・`frontend/package.json.save`・直下の `package.json`・`package-lock.json`	6	0.1 MB	テストの記録・ログ・使っていない Excel・エディターの控え・直下の部品の一覧

`.vscode/` も `.gitignore` に書いてありますが、書く前に記録された `settings.json`・`tasks.json` が残っています（チームで同じ設定を使うなら、残してよい）。

直し方:

① `.gitignore` を、次に置き換えます。

場所: `.gitignore` の 1〜9 行目（.gitignore 全体）

今:

コピー

```
.env
**/__pycache__/
*.py[cod]
*$py.class
.vscode/
.claude

node_modules/
frontend/node_modules/
```

直した後（上の行を、全部これに置き換える）：

コピー

```
.env
**/__pycache__/
*.py[cod]
*$py.class
.vscode/
.claude
# Visual Studio が作る作業用のファイル・テストの結果
.vs/
.coverage
node_modules/
frontend/node_modules/
# ビルドの結果（GitHub Actions が毎回作り直すので、リポジトリには入れない）
frontend/build/
frontend/dist/
staticfiles/
# 動かすと増えるファイル（ログ・バックアップの Excel）
web_console.log*
media/
# エディターが残す控えのファイル
*.save
```

② 記録から外します（手元のファイルは消えません）。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
git rm -r --cached staticfiles frontend/build frontend/dist node_modules .vs media
git rm --cached .coverage web_console.log data.xlsx package.json package-lock.json
frontend/package.json.save "frontend/node-v22.17.1-linux-x64.tar.xz"
git commit -m "D-3: ビルドの結果・作業用のファイルをリポジトリから外す"
```

▶ ここで初めて出てくるコード

- **`git rm -r`**（Git）：`git rm` の `-r` は、フォルダの中を全部、という指定です。`git rm -r --cached staticfiles` は、`staticfiles` フォルダの中を全部、Git の記録から外します（手元のファイルは残る）。

- `frontend/build/` を外すと、手元で `python manage.py runserver` だけで画面を見る時は、先に `cd frontend` → `npm run build` が要ります（ふだんの開発は D-4 の `npm start` を使う）。
- 履歴の中の大きなファイルは、外しても残ります（リポジトリの大きさは減りません）。減らしたい時は、S-9 の 3 と一緒に考えます。

7.4 D-4 手元で `npm start` すると、画面から API に届かない

🟡 余裕のある時に ／ 新しく見つかった（前の D-4 が変わった） ／ 📖 コードから

どこ: `frontend/vite.config.mts` 33〜34 行目、`frontend/src/api/axios.ts` 30〜34 行目、`frontend/.env.production`、`CLAUDE.md` 13・30 行目

今:

- 画面の開発用のサーバー（`npm start` ＝ `vite`）は `http://localhost:5173` で動きます。axios の `baseURL` は `process.env.REACT_APP_API_BASE_URL` ですが、Vite では `process.env` が使えない（B-2）ので、API は **画面と同じ** `localhost:5173` へ送られます。Vite には、それを Django（`localhost:8000`）へ中継する設定がありません。
- `frontend/.env.production` の `REACT_APP_API_BASE_URL` は、Vite では読まれません（Vite が読むのは `VITE_` で始まる名前だけ）。000 巻 §8.1・01 巻で「書き換える」と書いた所は、今は意味がありません。
- `CLAUDE.md` に、開発用のサーバーが `localhost:3000`、`REACT_APP_API_BASE_URL` を使う、と古いことが書いてあります。

直し方: Vite に `/api` の中継を足し、axios の `baseURL` をやめます。本番と同じく、画面と API が同じ所に見えるようになります（CORS も要らない）。

場所: `frontend/vite.config.mts` の 33〜34 行目（defineConfig の最初）

今:

コピー

```
export default defineConfig({
  plugins: [react()], // 使用するプラグインの一覧（React 対応）
```

直した後（上の行を、全部これに置き換える）：

コピー

```
export default defineConfig({
  plugins: [react()], // 使用するプラグインの一覧（React 対応）
  // 開発サーバー（npm start）の設定: /api/ で始まるリクエストを、手元の Django（http://localhost:8000）へ中継する。
  //   画面（http://localhost:5173）と API が同じオリジンに見えるので、本番と同じく Cookie がそのまま使える
  server: {
    proxy: {
      '/api': 'http://localhost:8000',
    },
  },
```

場所: `frontend/src/api/axios.ts` の 30〜34 行目（axios.create の最初）

今:

コピー

```
    // 接続先の基準URL。api.get("/api/〜") は「baseURL ＋ /api/〜」へ送られる
    //   コード上は環境変数 REACT_APP_API_BASE_URL（frontend/.env.production に同名の設定あり）を読む。
    //   ただし Vite のビルド（npm run build）では process.env が空のオブジェクト {} に置き換えられるため
    //   この値は undefined になり、baseURL なし＝画面を表示しているのと同じサーバーへ送信される
    baseURL: process.env.REACT_APP_API_BASE_URL,
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    // 接続先の基準URL（baseURL）は付けない。api.get("/api/〜") は、画面を表示しているのと同じサーバーへ送られる
    //   本番: Django が画面と API の両方を出している
    //   手元: Vite の開発サーバー（http://localhost:5173）が /api を Django（http://localhost:8000）へ中継する（vite.config.mts）
```

`frontend/.env.production` は消します（`git rm frontend/.env.production`）。`CLAUDE.md` も直します。

場所: `CLAUDE.md` の 13 行目

今:

コピー

```
npm start          # Start dev server at http://localhost:3000
```

直した後（上の行を、全部これに置き換える）:

コピー

```
npm start         # Start Vite dev server at http://localhost:5173 (proxies /api to http://localhost:8000)
```

場所: `CLAUDE.md` の 30 行目

今:

コピー

```
- Frontend uses `REACT_APP_API_BASE_URL` for API endpoint configuration
```

直した後（上の行を、全部これに置き換える）：

コピー

```
- Frontend calls the API with relative URLs (`/api/...`); in development Vite proxies `/api` to Django (see `frontend/vite.config.mts`)
```

手元で動かす手順（直した後）：

1. 1 つ目の PowerShell で、`python manage.py runserver`（`http://localhost:8000`）
2. 2 つ目の PowerShell で、`cd frontend` → `npm start`（`http://localhost:5173`）
3. ブラウザで `http://localhost:5173` を開く

7.5 D-5 住所（ホスト名）がコードに直接書いてある

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `hozen_another/settings.py` 56〜62 行目（`ALLOWED_HOSTS`）・306〜312 行目（`CSRF_TRUSTED_ORIGINS`）

今: 本番のホスト名が、コードに直接書いてあります。ステージング（試しの環境）などに出す時は、コードを書き換える必要があります。

直し方: 環境変数（`.env` や Azure の構成）にあればそれを、無ければ今の値を使うようにします（今の動きは変わりません）。

場所: `hozen_another/settings.py` の 56〜62 行目（ALLOWED_HOSTS）

今:

コピー

```
# このサーバーへのアクセスとして受け付けるホスト名の一覧（ここに無いホスト名宛てのリクエストは 400 エラー）
#   本番（Azure 上のサーバー）のホスト名と、開発用のローカル環境
ALLOWED_HOSTS = [
    '<本番のホスト名>',
    'localhost',
    '127.0.0.1'
    ]
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# このサーバーへのアクセスとして受け付けるホスト名の一覧（ここに無いホスト名宛てのリクエストは 400 エラー）
#   .env や Azure の構成に ALLOWED_HOSTS（カンマ区切り）があればそれを使い、無ければ下の値（本番と手元）を使う
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS', default=[
    '<本番のホスト名>',
    'localhost',
    '127.0.0.1',
    ])
```

▶ ここで初めて出てくるコード

- `.list()`（Django）：`env.list('ALLOWED_HOSTS', default=[...])` は、`django-environ` で、`a,b,c` のようにカンマで区切った環境変数を、並び（リスト）として読むメソッド。

場所: `hozen_another/settings.py` の 306〜312 行目（CSRF_TRUSTED_ORIGINS）

今:

コピー

```
# CSRF の確認で「信頼できる送信元」とみなすオリジンの一覧
#   POST/PUT/DELETE のリクエストは、送信元（Origin ヘッダー）がここに含まれるか、同じホストである必要がある
CSRF_TRUSTED_ORIGINS = [
  'https://<本番のホスト名>',
  'http://localhost:8000',
  'http://127.0.0.1:8000',
]
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# CSRF の確認で「信頼できる送信元」とみなすオリジンの一覧
#   POST/PUT/DELETE のリクエストは、送信元（Origin ヘッダー）がここに含まれるか、同じホストである必要がある
#   .env や Azure の構成に CSRF_TRUSTED_ORIGINS（カンマ区切り）があればそれを使う
CSRF_TRUSTED_ORIGINS = env.list('CSRF_TRUSTED_ORIGINS', default=[
  'https://<本番のホスト名>',
  'http://localhost:8000',
  'http://127.0.0.1:8000',
])
```

7.6 D-6 HTTPS まわりの設定が足りない

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

今: Azure App Service は、HTTPS で受けた頼みを、中のアプリへは HTTP で渡します。アプリは「HTTPS で来た」と分からないので、`request.is_secure()` が `False` になります。今は `CSRF_TRUSTED_ORIGINS` に本番の `https://…` を書いてあるので CSRF の確かめは通りますが、HTTPS を前提にした Django の守り（Referer の確かめなど）が働きません。

直し方: `CSRF_TRUSTED_ORIGINS` のすぐ後ろに足します。

場所: `hozen_another/settings.py` の 313 行目の前（CSRF_TRUSTED_ORIGINS のすぐ後ろ）

足すコード（この行の前に入れる）：

コピー

```
# Azure App Service は、HTTPS で受けたリクエストを、中のアプリへは HTTP で渡す。
# その時に付く X-Forwarded-Proto ヘッダーを見て「本当は HTTPS だった」と分かるようにする
#   （request.is_secure() が正しくなり、CSRF の確かめも HTTPS として行われる）
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
# CSRF トークンの Cookie も、本番（DEBUG=False）では HTTPS の時だけ送る
CSRF_COOKIE_SECURE = not DEBUG
```

HTTPS だけで使うと決めたら、ブラウザに「この住所は HTTPS だけで開く」と覚えさせる `SECURE_HSTS_SECONDS` も足せます。大きくすると、その間 http で開けなくなるので、小さい値（例: `3600` ＝ 1 時間）から始めます。

7.7 D-7 使っていない部品

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから（2026 年 11 月 10 日に、コードの中に名前が出てこないことを `grep` で確かめた）

サーバー側（`requirements.txt`）：

部品	何をする物か	今
`xlwings`	Windows・Mac の Excel を動かす	使っていない（Linux の本番では使えない）
`python-Levenshtein`・`beautifulsoup4`	文字の近さを測る・HTML を読む	使っていない
`django-crispy-forms`・`django-widget-tweaks`・`django-widgets-improved`・`django-filter`	Django の画面（テンプレート）のフォームを作る	使っていない（今の画面は React）
`python-dotenv`・`python-json-logger`・`python-dateutil`	`.env` を読む・ログを JSON で書く・日付の計算	使っていない（`.env` は `django-environ` が読む）
`django-q`	仕事を順に受け持つ仕組み	`settings.py` に設定だけある。どこからも使っていない（B-13）
`django-bootstrap4`・`django-bootstrap-datepicker-plus`	Django の画面の見た目	`settings.py` の `INSTALLED_APPS`（76〜77 行目）・`TEMPLATES` の `builtins`（194〜196 行目）・`BOOTSTRAP4`（217〜219 行目）にだけ出てくる。昔の Django の画面の名残
`pandas`	表の計算	使っていない関数（D-11 の `generate_prediction`）でだけ使っている

画面側（`frontend/package.json`）：

部品	今
`@fullcalendar/daygrid`・`@fullcalendar/interaction`・`@fullcalendar/react`・`@fullcalendar/timegrid`	使っていない
`react-calendar`・`dayjs`	使っていない（日付の部品は `date-fns` を使っている）
`web-vitals`	`src/reportWebVitals.ts` だけが使い、そのファイルはどこからも読み込まれていない
`@types/jest`	Jest の型。テストは Vitest なので要らない
`@testing-library/dom`	テストでだけ使う物なのに、`dependencies`（本番用）に入っている。`devDependencies` へ移す
`eslintConfig`（`"react-app"`）・`browserslist`	Create React App の時の設定の名残。今は使っていない

直し方: 1 つずつ外して、`pip install -r requirements.txt`（または `npm install`）→ テスト → 手元で動かす、を繰り返します。`django-bootstrap4`・`django-bootstrap-datepicker-plus` は、`settings.py` の 3 か所も一緒に消します。

7.8 D-8 サポート（安全の直し）が終わった部品・版のずれ

🟠 早めに ／ 一部だけ直った（Create React App → Vite） ／ 📖 コードから

部品	今の版	状況（2026 年 11 月）	上げる先
Django	4.2.7	4.2 系のサポートは 2026 年 4 月で終わった。安全の直しが出ない	まず 4.2 系の最後の版に上げ、次に 5.2（LTS。2028 年 4 月まで）
djangorestframework	3.14.0	Django 5.2 に対応していない	Django 5.2 に対応した版（3.16 以降）
django-q	1.3.9	開発が止まっている（後を継いだのは `django-q2`）	B-13 と合わせて決める
Node.js（Actions）	20	2026 年 4 月でサポート終了	22（D-1）
画面の部品（Vite・TypeScript など）	記録（`package-lock.json`）は Vite 8.2.2・TypeScript 7.0.2・`@vitejs/plugin-react` 6.1.1。この PC の `frontend/node_modules` に入っているのは Vite 7.3.0・TypeScript 4.9.5・`@vitejs/plugin-react` 5.1.2	手元と Actions で、違う版で動いている	手元でも `npm ci` で入れ直し、`npm run build`（`tsc --noEmit` と `vite build`）が通ることを確かめる

Django を 5.1 以上にする時は、`settings.py` 317 行目の `STATICFILES_STORAGE` が使えなくなるので、次のように書き換えます。

コピー

```
STORAGES = {
    "default": {"BACKEND": "django.core.files.storage.FileSystemStorage"},
    "staticfiles": {"BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage"},
}
```

進め方: 一度に全部上げません。1 つずつ、テスト → 手元で動かす → （あれば）ステージング → 本番、の順にします。上げる前に、その部品の「リリースノート」（版ごとの変更点）の、変わった所・消えた所を読みます。

> この巻で手元の写しで確かめた画面のテストとビルドは、この PC の `frontend/node_modules` に入っている版（Vite 7.3.0 など）で流した物です。記録どおりの版（Vite 8.2.2・TypeScript 7.0.2）では流していません。

7.9 D-9 `print` の行き先のすり替えと、ログのファイルの場所

🟡 余裕のある時に ／ 前から残っている ／ ✅ 手元で再現

どこ: `kosu/views/main_views.py` 70〜126 行目（`PrintToLogger`・`get_logs`）、`hozen_another/settings.py` 349 行目、`kosu/tasks.py` の `print` 8 か所、`asynchronous_views.py` 712〜714 行目

今:

- `sys.stdout = PrintToLogger()`（109 行目）で、`main_views.py` が読み込まれた瞬間から、**サーバー全体の `print`** がログのファイルに書かれます。✅ 手元で、`python manage.py makemigrations --check` の結果も画面に出ず、ログのファイルに入りました（D-2）。
- ログのファイルは、書く側（`settings.py` 349 行目）が「起動した場所からの `web_console.log`」、読む側（`WebConsoleLogView`、1342 行目）が「プロジェクトの直下の `web_console.log`」です。起動の場所が違うと、書く所と読む所がずれます。
- `web_console.log` がリポジトリに記録されているので、デプロイのたびに上書きされます（D-3）。

直し方:

① `PrintToLogger` と、使っていない `get_logs` を消します（`views_logger` は残す）。

場所: `kosu/views/main_views.py` の 70〜126 行目（PrintToLogger と get_logs）

今:

コピー

```
# ============================================================
# ログ出力の設定（print の出力を web_console.log に記録する）
# ============================================================
# 専用ロガー取得('views_logger'という名前のカスタムロガーを使用しログメッセージ記録)
#   views_logger の出力先は settings.py の LOGGING で設定されている
#   （プロジェクトルートの web_console.log。1MBを超えたら1世代だけ残して新しいファイルに切り替わる）
views_logger = logging.getLogger('views_logger')

# ------------------------------------------------------------
# PrintToLogger: 標準出力の代わりになるクラス
#   print 関数は sys.stdout の write() を呼び出して文字を出力する。
#   sys.stdout をこのクラスに差し替えることで、print の出力を views_logger に流し、
#   管理者画面のログ表示（WebConsoleLogView）で確認できるようにする。
# ------------------------------------------------------------
class PrintToLogger:
  # print から渡された文字列をログに記録する。
  #
  # 引数:
  #   message: 出力しようとした文字列（print は本文と末尾の改行を別々に渡してくることがある）
  # 戻り値:
  #   なし
  def write(self, message):
    # 受け取った標準出力のメッセージ処理(空白および改行のみのメッセージは無視)
    if message.strip():
      views_logger.info(message)

  # バッファ（出力待ちの文字）を書き出す。
  #
  # sys.stdout として使うには flush() が必要なため用意しているが、
  # ログへは write() の時点で書き込み済みなので何もしない。
  #
  # 戻り値:
  #   なし
  def flush(self):
    # 処理不要のため空処理
    pass

# 標準出力をロガーへリダイレクト(sys.stdoutをPrintToLoggerのインスタンスに設定)
#   このファイルが読み込まれた時点で実行され、以降はサーバープロセス全体の print 出力が対象になる
sys.stdout = PrintToLogger()

# 記録されたログを JSON で返す（関数ビュー）。
#
# ※現在 urls.py には登録されておらず、どこからも呼ばれていない。
#   管理者画面のログ表示には WebConsoleLogView（/api/web_console_log/）が使われている。
#
# 引数:
#   request: HTTPリクエスト
# 戻り値:
#   JsonResponse: {'logs': ログファイルの各行を要素とするリスト}
def get_logs(request):
  # 'web_console.log' ファイルを読み込み専用モードで開く（サーバー起動時のカレントフォルダからの相対パス）
  with open('web_console.log', 'r') as log_file:
    # ファイルの内容を行ごとにリストとして格納
    logs = log_file.readlines()
  # JSONレスポンスとして、ログを返す
  return JsonResponse({'logs': logs})
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# ============================================================
# ログ出力の設定
# ============================================================
# 専用ロガー取得('views_logger'という名前のカスタムロガーを使用しログメッセージ記録)
#   views_logger の出力先は settings.py の LOGGING で設定されている
#   （プロジェクトルートの web_console.log。1MBを超えたら1世代だけ残して新しいファイルに切り替わる）
#   ログに残したいことは、print ではなく views_logger.info(...) で書く
#   （前は sys.stdout を差し替えて、サーバー全体の print をログに流していた。ほかの部品の出力も混ざり、
#     テストの出力も見えなくなるため、やめた。使われていなかった get_logs も消した）
views_logger = logging.getLogger('views_logger')
```

② ログのファイルの場所を、プロジェクトの直下にそろえます。

場所: `hozen_another/settings.py` の 349 行目（LOGGING の出力先）

今:

コピー

```
            'filename': 'web_console.log',  # 出力先ファイル（サーバーを起動したフォルダからの相対パス）
```

直した後（上の行を、全部これに置き換える）:

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
            'filename': os.path.join(BASE_DIR, 'web_console.log'),  # 出力先ファイル（プロジェクトルート。
WebConsoleLogView が読む場所と同じにする）
```

③ `tasks.py` の `print` を、ロガー（`views_logger.info`）に変えます。

場所: `kosu/tasks.py` の 52 行目の前（import の後ろ）

足すコード（この行の前に入れる）:

コピー

```
import logging  # ログ（web_console.log）に書く
# ロード中の警告などを書くロガー（出力先は settings.py の LOGGING。管理者のログ画面で見られる）
views_logger = logging.getLogger('views_logger')
```

8 か所とも、行の中の `print(` を `views_logger.info(` に変えます（行のほかの所は変えません）。

ファイル	行	今の行（頭の所）	直した後
`tasks.py`	304	`print(f"警告: 従業員番号 {employee_no} (行 {i}) のメンバーが見つ…`	`print(` → `views_logger.info(`
`tasks.py`	338	`print(f"エラー: 行 {i} のデータ作成中に整合性エラーが発生しました: {creat…`	`print(` → `views_logger.info(`
`tasks.py`	497	`print(f"従業員番号 {employee_no_value} は既に存在するためスキップし…`	`print(` → `views_logger.info(`
`tasks.py`	549	`print(f"従業員番号 {employee_no_value} の書き込み中にIntegri…`	`print(` → `views_logger.info(`
`tasks.py`	688	`print(f"従業員番号 {employee_no_value} は既に存在するためスキップし…`	`print(` → `views_logger.info(`
`tasks.py`	715	`print(f"従業員番号 {employee_no_value} の書き込み中にIntegri…`	`print(` → `views_logger.info(`
`tasks.py`	1128	`print(f"書き込み中にIntegrityErrorが発生しました: {create_e}"…`	`print(` → `views_logger.info(`
`tasks.py`	1315	`print(f"書き込み中にIntegrityErrorが発生しました: {create_e}"…`	`print(` → `views_logger.info(`

④ `asynchronous_views.py` の `print` も変えます。

場所: `kosu/views/asynchronous_views.py` の 38 行目の前（import）

足すコード（この行の前に入れる）：

コピー

```
import logging  # ログ（web_console.log）に書く
```

場所: `kosu/views/asynchronous_views.py` の 712〜714 行目（handle_task の最後）

今:

コピー

```
            print(
                f'Error: AsyncTask with id {task_id} not found.'
            )
```

直した後（上の行を、全部これに置き換える）：

コピー

```
            logging.getLogger('views_logger').warning(
                f'Error: AsyncTask with id {task_id} not found.'
            )
```

> **▶ ここで初めて出てくるコード**
>
> - `.warning()`（Python）：`ロガー.warning(文)` は、ロガー（ログを書く窓口）に「注意」の重さでログを書くメソッド。`info` より重い知らせに使います。

長く残したいログは、App Service のログの機能を使います（前任者・Azure の権限を持つ人と）。

7.10 D-10 古い入口 `/BA…` から、全部の API が使える

🟡 余裕のある時に ／ 前から残っている ／ ✅ 手元で再現

どこ: `hozen_another/urls.py` 34〜36 行目

今: `path('BA', include('kosu.urls'))` があるので、全部の API が、`/api/login/` のほかに `/BAlogin/` のような住所でも動きます（`/` が無いので、`BA` と API の名前がつながる）。✅ 手元で `/BAlogin/` がログインの API に当たりました。画面からは使っていません。

場所: `hozen_another/urls.py` の 34〜36 行目（urlpatterns の中）

今:

コピー

```
# 'BA' で始まるURLを kosu/urls.py へ振り分ける
#   'BA' の後ろに「/」が無いため、実際のURLは /BAlogin/ のように 'BA' と kosu 側のURLが直結した形になる
path('BA', include('kosu.urls')),  # 'BA' パターンの処理
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# （前にあった 'BA' で始まる入口は消した。画面が使うのは /api/ だけ）
```

7.11 D-11 使っていないコードが残っている

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

場所	中身	消す時に気を付けること
`main_views.py` 111〜126 行目	`get_logs`（どの URL にもつながっていない）	D-9 の①で一緒に消える
`tasks.py` 141〜212 行目	`generate_prediction`（どこからも呼ばれていない）と、それだけが使う `pandas`（43 行目）	`import pandas as pd` も消す（D-7）
`tasks.py` 41 行目	`from django.utils import timezone`（コメントにも「未使用」とある）	無い
`models.py` 594〜614 行目	`Operation_history`（どこからも使われていない表）	消すと migrate で表が消える。本番の表に中身が無いかを先に確かめる
`main_utils.py` 105〜129 行目	`validate_employee_no_logic`（テストからしか呼ばれていない）	テスト（`test_backend.py`）も一緒に消す
`settings.py` 26 行目	`from logging.handlers import RotatingFileHandler`（コメントにも「未使用」とある）	無い
`frontend/public/index.html`・`src/react-app-env.d.ts`・`src/reportWebVitals.ts`	Create React App の時のファイル（Vite は `frontend/index.html` を使う）	`react-app-env.d.ts` に、画像の `import` の型が書いてあれば、`vite-env.d.ts` へ移す
`data.xlsx`（リポジトリの直下）	作業詳細の選択肢の Excel。どこからも読まれていない	無い（D-3）

7.12 D-12 同じコードの繰り返しが多い

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

同じことを何か所にも書いていると、1 か所だけ直し忘れます（B-4 の 500 も、3 か所に同じ書き方があった）。

繰り返し	場所（ ec2cedc ）	まとめ方
班の `member1` ～ `member15` から番号を並べる	`team_views.py` 92～102・212～222・406～416・618～628 行目、`main_views.py` 579～589 行目	S-3 の③で足す `team_member_numbers` を使う
休憩時間の初めの値（24項目）	`main_views.py` 295～323 行目、`member_views.py` 148～202 行目	1 つの辞書（定数）にする
お知らせ（ポップアップ）を前に詰める	`inquiry_views.py` 264～291・301～328・549～571・573～594 行目	関数にする
5 つのメニューの API	`main_views.py` 415～647 行目	ほぼ同じ。1 つの関数にまとめられる
バックアップの 19 個の `if`	`asynchronous_views.py` 135～405 行目	「URL の名前 → 関数」の表（辞書）にする
月末の日を求める計算	`kosu_views.py` 1370～1379・1478～1487・1557～1566 行目	`calendar.monthrange` を使う

7.13 D-13 URL の名前の入れ違い

🟡 余裕のある時に ／ 前から残っている ／ 📖 コードから

どこ: `kosu/urls.py` 85～90 行目・157 行目

`day_update/` の名前が `'item_delete'`、`item_delete/` の名前が `'day_update'` と入れ違っています（86・89 行目のコメントにも書いてある）。`team_shop_select/` の名前は、最後に `/` が付いています。今は、名前（`name=`）を使っている所が無いので（テストも URL を直接書いている）、動きには関係しません。

場所: `kosu/urls.py` の 85～90 行目（day_update と item_delete）

今:

コピー

```
# PUT /api/day_update/ : 工数データの就業日を変更 → DayUpdate
#   ※name が 'item_delete' になっている（下の item_delete/ と名前が入れ替わっている）
path('day_update/', kosu_views.DayUpdate.as_view(), name='item_delete'),
# POST /api/item_delete/ : 工数編集画面で選んだ項目（時間帯）の工数を削除 → ItemDlete
#   ※name が 'day_update' になっている（上の day_update/ と名前が入れ替わっている）
path('item_delete/', kosu_views.ItemDlete.as_view(), name='day_update'),
```

直した後（上の行を、全部これに置き換える）：

コピー

```
# PUT /api/day_update/ : 工数データの就業日を変更 → DayUpdate
path('day_update/', kosu_views.DayUpdate.as_view(), name='day_update'),
# POST /api/item_delete/ : 工数編集画面で選んだ項目（時間帯）の工数を削除 → ItemDlete
path('item_delete/', kosu_views.ItemDlete.as_view(), name='item_delete'),
```

場所: `kosu/urls.py` の 157 行目（team_shop_select）

今:

コピー

```
path('team_shop_select/', team_views.TeamShopSelect.as_view(), name='team_shop_select/'),
```

直した後（上の行を、全部これに置き換える）:

コピー

```
path('team_shop_select/', team_views.TeamShopSelect.as_view(), name='team_shop_select'),
```

`check_backup_status` と `download_backup`（229・231 行目）だけ、URL の最後に `/` がありません。そろえる時は、画面（`AdministratorLoading.tsx`）の URL も同時に直します。

7.14 D-14 アプリのリポジトリの `docs/` が、この資料の古い版。`CLAUDE.md` も古い

🟡 余裕のある時に ／ 新しく見つかった ／ 📖 コードから

今:

- 11 月 9 日のコミット `ec2cedc` で、アプリのリポジトリに `docs/`（00〜12 巻の Markdown と PDF、PDF を作る道具）が入りました。中身は、この資料（`kosu/`）の、名前を変える前の古い版です。000 巻・13〜15 巻は入っていません。資料が 2 か所にあると、片方だけが直されて、食い違っていきます。
- `docs/pdf/` の PDF は、B-20 のせいで、GitHub の中では壊れています。
- `CLAUDE.md`（AI の道具に読ませる説明）は、Create React App の時のまま（D-4）です。

直し方: 資料を置く所を 1 か所に決めます。たとえば、アプリのリポジトリの `docs/` を消し、`docs/README.md` だけを残して「資料はこちら」と、置き場所を書きます。PDF をリポジトリに入れるなら、B-20 を直してからにします。

---

8. テスト（T）

8.1 T-1 CSRF を確かめる状態のテストが 1 つも無い

🔴 すぐ ／ 新しく見つかった ／ ✅ 手元で再現

今: アプリのテスト（`test_backend.py`）は、Django のテスト用の `Client`（`APIClient`）を使っています。これは、**最初から CSRF の確かめを切っています**（`enforce_csrf_checks=False`）。そのため、S-1（本物のブラウザでは書き込みが全部 403）があっても、339 個が全部通ってしまいます。

直し方: `enforce_csrf_checks=True` の `Client` で、「画面を開く → ログイン → 書き込み → ログアウト」を流すテストを足します。T-2 の `test_security.py` の `TestCsrfFlow`（3 個）がそれです。

8.2 T-2 直した所を守るテストを足す

🟠 早めに ／ 前から残っている ／ ✅ 手元で確かめた

直した所が、また壊れていないかを確かめるテストを、新しいファイルに 14 個まとめました。✅ 今の `master` では 14 個とも落ち（10 個が失敗・4 個がエラー）、この巻の直しを全部入れた写しでは 14 個とも通りました。

クラス	確かめること	番号
`TestCsrfFlow`	画面を開くと `csrftoken` がもらえる・ログイン後の書き込みとログアウトが通る・トークン無しの書き込みは 403・ログインでセッションの番号が変わる	S-1・T-1
`TestOthersData`	他人の工数を開けない・日付を変えられない・班でない人の工数を開けない・他人の問い合わせを直せない・消せない・本人は回答を書けない・権限だけの人は管理者を作れない	S-3
`TestDownload`	`media` の外のファイルは 404 で消えない・`media` の中のファイルは返して消える	S-4
`TestHelp`	6 回目は 429・`HELP_PATH` が無くても 500 にならない	S-7
`TestLoadIsAtomic`	ロードの途中で失敗すると、元のデータに戻る	B-8

新しいファイル: `kosu/tests/test_security.py`

中身（全部）：

コピー

```
# ============================================================
# ファイル名 : test_security.py
# 種別       : テストコード（Django の TestCase）
# 概要       : 直した所（ログイン・CSRF・他人のデータ・ダウンロード・ヘルプ・ロード）が、
#              また壊れていないかを確かめるテスト。
#              test_backend.py のテストは CSRF の確かめを切った状態で動く（Django のテスト用 Client の既定）ので、
#              ここでは enforce_csrf_checks=True で、ブラウザと同じ流れも確かめる。
# 実行       : python manage.py test kosu --settings=hozen_another.test_settings
# ============================================================
import json
import os
import tempfile
from pathlib import Path
from unittest import mock

import openpyxl
from django.core.cache import cache
from django.test import Client, TestCase, override_settings
from rest_framework.throttling import ScopedRateThrottle

from kosu.models import (Business_Time_graph, administrator_data, def_choice, inquiry_data, kosu_division,
                         member, team_member)
from kosu.tasks import load_choice_file

# 休憩時間の24項目（人員データを作るのに必要。値は何でもよい）
BT = {f'break_time{i}{s}': '#12001300' for i in range(1, 7) for s in ('', '_over1', '_over2', '_over3')}

# 画面（index.html）の代わり。frontend/build が無くても react_view のテストができるようにする
TEMPLATE_DIR = tempfile.mkdtemp(prefix='kosu-test-templates-')
Path(TEMPLATE_DIR, 'index.html').write_text('<!doctype html><div id="root"></div>', encoding='utf-8')
TEMPLATES_FOR_TEST = [{
  'BACKEND': 'django.template.backends.django.DjangoTemplates',
  'DIRS': [TEMPLATE_DIR],
  'APP_DIRS': True,
}]


def make_member(no, admin=False, auth=False):
  return member.objects.create(employee_no=no, name=f'm{no}', shop='P', authority=auth, administrator=admin, **BT)


def as_json(data):
  return {'data': json.dumps(data), 'content_type': 'application/json'}


class SecurityTestBase(TestCase):
  def setUp(self):
    kosu_division.objects.create(kosu_name='v1', kosu_title_1='A')
    administrator_data.objects.create(menu_row='20')
    self.admin = make_member(1001, admin=True, auth=True)   # 管理者
    self.leader = make_member(1501, auth=True)              # 権限あり・管理者でない
    self.user_a = make_member(2001)
    self.user_b = make_member(2002)
    self.rec_b = Business_Time_graph.objects.create(employee_no3=2002, name=self.user_b, work_day2='2026-01-05',
                                                    time_work='#' * 288, detail_work='$' * 287, def_ver2='v1')

  def login(self, no):
    c = Client()
    r = c.post('/api/login/', **as_json({'employee_no': no}))
    self.assertEqual(r.status_code, 200)
    return c


@override_settings(TEMPLATES=TEMPLATES_FOR_TEST)
class TestCsrfFlow(SecurityTestBase):
  """T-1: ブラウザと同じく CSRF を確かめる状態で、画面を開く → ログイン → 書き込み → ログアウト ができる。"""

  def test_page_gives_cookie_and_writes_work(self):
    c = Client(enforce_csrf_checks=True)
    c.get('/')                                              # 画面を開くと csrftoken の Cookie がもらえる
    self.assertIn('csrftoken', c.cookies)
    r = c.post('/api/login/', HTTP_X_CSRFTOKEN=c.cookies['csrftoken'].value, **as_json({'employee_no': 2001}))
    self.assertEqual(r.status_code, 200)
    token = c.cookies['csrftoken'].value                    # ログインでトークンが新しくなるので読み直す
    r = c.post('/api/def_ver/', HTTP_X_CSRFTOKEN=token, **as_json({'versionchoice': 'v1'}))
    self.assertEqual(r.status_code, 200)
    r = c.post('/api/logout/', HTTP_X_CSRFTOKEN=token, **as_json({}))
```

```
    self.assertEqual(r.status_code, 200)

  def test_write_without_token_is_403(self):
    c = Client(enforce_csrf_checks=True)
    c.get('/')
    c.post('/api/login/', HTTP_X_CSRFTOKEN=c.cookies['csrftoken'].value, **as_json({'employee_no': 2001}))
    r = c.post('/api/def_ver/', **as_json({'versionchoice': 'v1'}))     # X-CSRFToken を付けない
    self.assertEqual(r.status_code, 403)

  def test_login_changes_session_key(self):
    c = Client()
    session = c.session
    session['probe'] = 1
    session.save()
    before = session.session_key
    c.post('/api/login/', **as_json({'employee_no': 2001}))
    self.assertNotEqual(c.cookies['sessionid'].value, before)


class TestOthersData(SecurityTestBase):
  """T-2: 他人のデータを見たり、変えたり、消したりできない（S-3）。"""

  def test_kosu_update_get_of_others_is_404(self):
    c = self.login(2001)
    self.assertEqual(c.get(f'/api/kosu_update/{self.rec_b.id}/').status_code, 404)

  def test_day_update_of_others_is_404(self):
    c = self.login(2001)
    r = c.put('/api/day_update/', **as_json({'id': self.rec_b.id, 'work_day2': '2026-02-01'}))
    self.assertEqual(r.status_code, 404)
    self.rec_b.refresh_from_db()
    self.assertEqual(str(self.rec_b.work_day2), '2026-01-05')

  def test_team_detail_only_for_my_team(self):
    c = self.login(1501)
    self.assertEqual(c.get(f'/api/team_detail/{self.rec_b.id}/').status_code, 404)
    team_member.objects.create(employee_no5=1501, member1='2002')
    self.assertEqual(c.get(f'/api/team_detail/{self.rec_b.id}/').status_code, 200)

  def test_inquiry_of_others_cannot_be_changed_or_deleted(self):
    inq = inquiry_data.objects.create(employee_no2=2002, name=self.user_b, content_choice='要望', inquiry='q', answer='')
    c = self.login(2001)
    r = c.put(f'/api/inquir_update/{inq.id}/', **as_json({'content_choice': '要望', 'inquiry': 'x', 'answer': 'y'}))
    self.assertEqual(r.status_code, 403)
    self.assertEqual(c.delete(f'/api/inquir_update/{inq.id}/').status_code, 403)
    inq.refresh_from_db()
    self.assertEqual((inq.inquiry, inq.answer), ('q', ''))

  def test_owner_cannot_write_answer(self):
    inq = inquiry_data.objects.create(employee_no2=2001, name=self.user_a, content_choice='要望', inquiry='q', answer='')
    c = self.login(2001)
    r = c.put(f'/api/inquir_update/{inq.id}/', **as_json({'content_choice': '要望', 'inquiry': 'q2', 'answer': '自分で回答'}))
    self.assertEqual(r.status_code, 403)

  def test_authority_cannot_make_or_remove_administrators(self):
    c = self.login(1501)
    new = dict(BT, employee_no=3001, name='n', shop='P', authority=True, administrator=True)
    self.assertEqual(c.post('/api/member_new/', **as_json(new)).status_code, 403)
    me = dict(BT, employee_no=1501, name='m1501', shop='P', authority=True, administrator=True)
    self.assertEqual(c.put('/api/member_update/1501/', **as_json(me)).status_code, 403)
    self.assertEqual(c.delete('/api/member_delete/1001/').status_code, 403)
    self.assertTrue(member.objects.filter(employee_no=1001).exists())


class TestDownload(SecurityTestBase):
  """T-2: ダウンロードは media フォルダの中のファイルだけ（S-4）。"""

  def test_file_outside_media_is_404_and_kept(self):
    fd, outside = tempfile.mkstemp(suffix='.txt')
    os.close(fd)
    try:
      c = self.login(1001)
      self.assertEqual(c.get('/api/download_backup', {'file_path': outside}).status_code, 404)
      self.assertTrue(os.path.exists(outside))
    finally:
      os.remove(outside)

  def test_file_in_media_is_sent_then_deleted(self):
    with tempfile.TemporaryDirectory() as media, override_settings(MEDIA_ROOT=media):
      p = Path(media, 'backup.xlsx')
```

```
        p.write_bytes(b'data')
        c = self.login(1001)
        r = c.get('/api/download_backup', {'file_path': p.name})
        self.assertEqual(r.status_code, 200)
        self.assertEqual(b''.join(r.streaming_content), b'data')   # 読み終わると close() が呼ばれる
        self.assertFalse(p.exists())


class TestHelp(SecurityTestBase):
    """T-2: ヘルプのパスワード（S-7）。"""

    def test_too_many_tries_are_429(self):
        cache.clear()
        with mock.patch.object(ScopedRateThrottle, 'THROTTLE_RATES', {'help': '5/hour'}):
            c = Client()
            codes = [c.post('/api/help_pass/', **as_json({'password': f'x{i}'})).status_code for i in range(6)]
        cache.clear()
        self.assertEqual(codes, [403] * 5 + [429])

    def test_no_help_path_is_403_not_500(self):
        with mock.patch.dict(os.environ, {'HELP_PATH': ''}):
            r = Client().post('/api/help_pass/', **as_json({'password': ''}))
        self.assertEqual(r.status_code, 403)


class TestLoadIsAtomic(TestCase):
    """T-2: ロードが途中で失敗したら、元のデータに戻る（B-8）。"""

    def test_choice_load_rolls_back(self):
        def_choice.objects.create(def_symbol='A', def_select='前からある1')
        def_choice.objects.create(def_symbol='A', def_select='前からある2')
        fd, path = tempfile.mkstemp(suffix='.xlsx')
        os.close(fd)
        wb = openpyxl.Workbook()
        ws = wb.active
        ws.append(['定義記号', '作業詳細'])
        ws.append(['A', '新しい1'])
        ws.append(['A', None])            # 作業詳細が空 → 保存できない行
        wb.save(path)
        result = load_choice_file(path)
        self.assertEqual(result[0]['status'], 'error')
        self.assertEqual(sorted(def_choice.objects.values_list('def_select', flat=True)), ['前からある1', '前からある2'])
```

▶ ここで初めて出てくるコード

- **`finally`**（Python）：`try … except` のあと、成功しても失敗しても最後に必ず動く所。
- **`.mkdtemp()`**（Python）：`tempfile.mkdtemp()` は、OS の一時フォルダの中に、新しい空のフォルダを作って、その場所を返す関数。`prefix=` は名前の頭です。動かすたびに別のフォルダになるので、前のテストで作った物が混ざりません。
- **`.write_text()`**（Python）：`Path` のファイルに、文字列を書くメソッド。`encoding="utf-8"` を付けます。
- **`.assertNotEqual()`**（Python のテスト）：2 つの値が「違う」ことを確かめるメソッド。同じだと、テストが失敗します。
- **`.refresh_from_db()`**（Django）：手元にある 1 行分のデータ（モデルのインスタンス）の値を、データベースから読み直すメソッド。API でデータが書き換わったかを確かめる前に呼びます。
- **`.mkstemp()`**（Python）：`tempfile.mkstemp()` は、一時フォルダに空のファイルを作り、（開いたファイルの番号, ファイルの場所）の 2 つを返す関数。番号は `os.close()` で閉じてから、場所の方を使います。
- **`.close()`**（Python）：接続やファイルを閉じるメソッド。
- **`.TemporaryDirectory()`**（Python）：`with tempfile.TemporaryDirectory() as 名前:` で、一時フォルダを作り、`with` の中だけ使えるようにする書き方。`with` を出ると、フォルダは中身ごと消えます。
- **`.write_bytes()`**（Python）：`Path` のファイルに、バイトの並びをそのまま書くメソッド。
- **`.clear()`**（Django）：`cache.clear()` は、Django のキャッシュ（一時的に覚えておく場所）を全部空にするメソッド。呼び出し回数の制限（`ScopedRateThrottle`）は回数をキャッシュに覚えるので、テストの前後で空にします。
- **`.object()`**（Python のテスト）：`mock.patch.object(物, '名前', 値)` を `with` で使うと、`with` の中だけ、その物の属性を別の値に差し替えます（`unittest.mock`）。テストで `ScopedRateThrottle.THROTTLE_RATES` を差し替えて、上限を 5 回にしています。
- **`.dict()`**（Python のテスト）：`mock.patch.dict(os.environ, {'HELP_PATH': ''})` を `with` で使うと、`with` の中だけ、辞書（ここでは環境変数）の中身を書き換えます。`with` を出ると元に戻ります。
- **`.values_list()`**（Django）：問い合わせの結果を、タプル（値の組）の並びで返すメソッド。`flat=True` を付けると、1 つの列の値だけの並びになります。
- **`override_settings()`**（Django）：`from django.test import override_settings` で読み込む、テストの間だけ設定を変える道具。`@override_settings(…)` の形で使います。
- **`dict()`**（Python）：辞書を作る関数。`dict(BT, employee_no=3001, name='n')` は、辞書 `BT`（休憩時間の 24 項目）の中身に、`employee_no` などを足した、新しい辞書を作ります。
- **`sorted()`**（Python）：並べかえた**新しい**リストを返す関数。`sorted(def_choice.objects.values_list('def_select', flat=True))` は、作業詳細の選択肢の文字を、並べかえたリストにします（テストで、並びの順番に関係なく中身をくらべるため）。
- **`@override_settings`**（Django）：テストのメソッドの上に付けると、そのテストの間だけ、`settings.py` の値を変えられる印。`@override_settings(HELP_PATH='test-help')`。

- `b"…"`（Python）：`b'data'` は、文字ではなく「バイトの並び」（`bytes`）を作る書き方です。ファイルに書く中身や、ダウンロードで返ってきた中身（`b''.join(response.streaming_content)`）とくらべるのに使います。
- `{キー: 値 for …}`（Python）：`{f'break_time{i}{s}': '#12001300' for i in range(1, 7) for s in ('', '_over1', '_over2', '_over3')}` は、「1 つずつから、キーと値の組を作って、辞書にする」書き方です。ここでは、休憩時間の 24 項目の辞書を、1 行で作っています。
- `Client`（Django）: Django のテスト用の、ブラウザの代わり（`django.test.Client`）。`Client(enforce_csrf_checks=True)` とすると、本物のブラウザと同じく、CSRF を確かめる状態で頼みを送れます（何も付けないと、確かめを切った状態）。
- `.cookies`（Django）: テスト用のクライアントの `client.cookies` は、そのクライアントが覚えているクッキーです。`client.cookies['csrftoken'].value` で、配られた CSRF の合言葉の値が取れます。

8.3 T-3 画面のテストが 3 個落ちる

🟡 余裕のある時に ／ 新しく見つかった ／ ✅ 手元で再現

今: 画面のテスト 535 個のうち、次の 3 個が落ちます（テストのファイルの中にも、落ちる理由が書いてある）。

- `WorkSelect.test.tsx` の 2 個: 就業形態の選択肢に「(時短なし)」の 3 つが足されたのに（7 月のコミット）、テストが前の個数（7 個・13 個）のまま。今は 10 個・16 個。
- `KosuNew.test.tsx` の「toggles tomorrow checkbox」: 工数入力の画面に「翌日」のチェックボックス（`id="tomorrow_check"`）が無いのに、それを押すテストが残っている。

直し方:

場所: `frontend/src/tests/Components/WorkSelect.test.tsx` の 49〜84 行目（選択肢の個数を確かめる 2 つのテスト）

今:

コピー

```
// 【テスト】mode 省略時（LIMIT）は出勤を伴う就業形態だけが表示されること
//   前提: mode を渡さない（"LIMIT" 扱い）
//   操作: 部品を表示する
//   期待: 「---」「出勤」「シフト出」「休出」「半前年休」「半後年休」「早退・遅刻」が表示され、
//         option がちょうど7個ある（※現在の部品は「(時短なし)」3項目を含む10個のため、個数の確認で失敗する）
it("デフォルト(LIMIT)モードで限定された選択肢が表示されること", () => {
  // 操作: mode を省略して部品を表示
  render(<WorkSelect id="work" value="" onChange={mockOnChange} />);
  // 検証: 出勤を伴う就業形態の選択肢を1つずつ確認する
  const limitLabels = ["---", "出勤", "シフト出", "休出", "半前年休", "半後年休", "早退・遅刻"];
  limitLabels.forEach((label) => {
    expect(screen.getByText(label)).toBeInTheDocument();
  });
  // 検証: option 要素の個数が7個である
  expect(screen.getAllByRole("option")).toHaveLength(7);
});

// 【テスト】mode="ALL" では休日・年休なども含めた全就業形態が表示されること
//   前提: mode に "ALL" を渡す
//   操作: 部品を表示する
//   期待: LIMIT の選択肢に加えて「休日」「年休」「公休」「シフト休」「代休」「欠勤」が表示され、
//         option がちょうど13個ある（※現在の部品は「(時短なし)」3項目を含む16個のため、個数の確認で失敗する）
it("ALL モードで全ての選択肢が表示されること", () => {
  // 操作: mode="ALL" で部品を表示
  render(<WorkSelect id="work" value="" onChange={mockOnChange} mode="ALL" />);
  // 検証: 全就業形態の選択肢を1つずつ確認する
  const allLabels = [
    "---", "出勤", "シフト出", "休出", "半前年休", "半後年休", "早退・遅刻",
    "休日", "年休", "公休", "シフト休", "代休", "欠勤",
  ];
  allLabels.forEach((label) => {
    expect(screen.getByText(label)).toBeInTheDocument();
  });
  // 検証: option 要素の個数が13個である
  expect(screen.getAllByRole("option")).toHaveLength(13);
});
```

▶ ここで初めて出てくるコード

- **`.forEach()`**（TypeScript）：配列の中身を 1 つずつ処理するメソッド（新しい配列は作らない）。`(x, i) => …` で、番号 `i` も受け取れます。
- **`.getByText()`**（Vitest・Testing Library）：`screen.getByText("出勤")` は、画面に表示されている文字で、部品を探すメソッド。見つからないと、テストが失敗します。
- **`.getAllByRole()`**（Vitest・Testing Library）：画面の中から、役割（`"option"` ＝ プルダウンの選択肢、など）が同じ物を全部探して、並び（配列）で返すメソッド。
- **`.toHaveLength()`**（Vitest・Testing Library）：並び（配列）の個数が、その数であることを確かめるメソッド。`expect(並び).toHaveLength(10)` は「10 個ある」。
- **`<WorkSelect>`**（React）：就業形態を選ぶプルダウンの部品（`frontend/src/Components/WorkSelect.tsx`）。`mode` を省くと出勤を伴う物だけ、`mode="ALL"` で休日なども含めた全部の選択肢を出します。

直した後（上の行を、全部これに置き換える）：

コピー

```
// 【テスト】mode 省略時（LIMIT）は出勤を伴う就業形態だけが表示されること
//   前提: mode を渡さない（"LIMIT" 扱い）
//   操作: 部品を表示する
//   期待: 「---」「出勤」「シフト出」「休出」「半前年休」「半後年休」「早退・遅刻」と
//         「(時短なし)」の3項目が表示され、option がちょうど10個ある
it("デフォルト(LIMIT)モードで限定された選択肢が表示されること", () => {
  // 操作: mode を省略して部品を表示
  render(<WorkSelect id="work" value="" onChange={mockOnChange} />);
  // 検証: 出勤を伴う就業形態の選択肢を1つずつ確認する
  const limitLabels = [
    "---", "出勤", "シフト出", "休出", "半前年休", "半後年休", "早退・遅刻",
    "出勤(時短なし)", "半前年休(時短なし)", "半後年休(時短なし)",
  ];
  limitLabels.forEach((label) => {
    expect(screen.getByText(label)).toBeInTheDocument();
  });
  // 検証: option 要素の個数が10個である
  expect(screen.getAllByRole("option")).toHaveLength(10);
});

// 【テスト】mode="ALL" では休日・年休なども含めた全就業形態が表示されること
//   前提: mode に "ALL" を渡す
//   操作: 部品を表示する
//   期待: LIMIT の選択肢に加えて「休日」「年休」「公休」「シフト休」「代休」「欠勤」が表示され、
//         option がちょうど16個ある
it("ALL モードで全ての選択肢が表示されること", () => {
  // 操作: mode="ALL" で部品を表示
  render(<WorkSelect id="work" value="" onChange={mockOnChange} mode="ALL" />);
  // 検証: 全就業形態の選択肢を1つずつ確認する
  const allLabels = [
    "---", "出勤", "シフト出", "休出", "半前年休", "半後年休", "早退・遅刻",
    "出勤(時短なし)", "半前年休(時短なし)", "半後年休(時短なし)",
    "休日", "年休", "公休", "シフト休", "代休", "欠勤",
  ];
  allLabels.forEach((label) => {
    expect(screen.getByText(label)).toBeInTheDocument();
  });
  // 検証: option 要素の個数が16個である
  expect(screen.getAllByRole("option")).toHaveLength(16);
});
```

場所: `frontend/src/tests/KosuPage/KosuNew.test.tsx` の 445〜462 行目（「翌日」チェックボックスのテスト）

今:

コピー

```
// 【テスト】「翌日」チェックボックス（id="tomorrow_check"）をクリックするとONになること
//   前提：初期状態はチェックなし
//   操作：チェックボックスをクリック
//   期待：チェックが入る
//   ※現在の KosuNew.tsx には id="tomorrow_check" の要素が存在しない（送信データに
//     tomorrow_check: false を固定で入れているだけ）ため、要素は見つからない
it("toggles tomorrow checkbox", async () => {
  // 準備：画面を表示し、読み込み完了を待つ
  render(<MemoryRouter><KosuNew /></MemoryRouter>);
  await screen.findByText("テスト太郎の工数入力");
  // 準備：id="tomorrow_check" の要素を取得し、初期状態でチェックが外れていることを確認する
  const tomorrowCheckbox = document.getElementById("tomorrow_check") as HTMLInputElement;
  expect(tomorrowCheckbox).not.toBeChecked();
  // 操作：チェックボックスをクリック
  fireEvent.click(tomorrowCheckbox);
  // 検証：チェックが入る
  expect(tomorrowCheckbox).toBeChecked();
});
```

▶ ここで初めて出てくるコード

- `.findByText()`（Vitest・Testing Library）：画面に、その文字が出てくるまで待って、見つかった物を返すメソッド（`await` で待つ）。読み込みが終わるのを待つ時に使います。
- `.toBeChecked()`（Vitest・Testing Library）：チェックボックスに、チェックが入っていることを確かめるメソッド。`.not.toBeChecked()` は「入っていない」ことを確かめます。

直した後（上の行を、全部これに置き換える）：

コピー

```
// （「翌日」チェックボックス id="tomorrow_check" のテストは消した。KosuNew.tsx にその欄が無いため）
```

✅ 直した写しでは、画面のテストが 534 個（1 個消した）全部通りました。

8.4 T-4 テストが `media/` に Excel を作る。直した後の動きに合わせるテストが 4 個

🟡 余裕のある時に ／ 新しく見つかった ／ ✅ 手元で再現

今:

- バックアップのテストが、本物と同じ `media/` に Excel を作ります。テストを流した後にそのままコミットすると、Excel がリポジトリに入ります（S-9。11 月 9 日のコミットに入っていた 9 つがこれ）。

- この巻の直しを入れると、前の動きを確かめていた 4 個のテストが落ちます（直すのが正しい物です）。✅ 手元で確かめた 4 個: `test_item_delete_no_kosu`・`test_item_delete_no_time_work`（401 → 404。B-10）、`test_check_task_status_success`（フォルダの場所 → ファイル名だけ。S-4）、`test_download_file`（`media` の外のファイル → `media` の中のファイル。S-4）。

直し方:

① テストの時は、`media` を毎回新しい一時フォルダにします。

場所: `hozen_another/test_settings.py` の 60 行目の前（ファイルの最後）

足すコード（この行の前に入れる）:

コピー

```
# テストで作るバックアップの Excel は、毎回新しい一時フォルダに置く
#   （media/ に置くと、テストのたびに Excel が増え、リポジトリに入ってしまう）
import tempfile
MEDIA_ROOT = tempfile.mkdtemp(prefix='kosu-test-media-')
```

② 4 個のテストを、直した後の動きに合わせます。

場所: `kosu/tests/test_backend.py` の 3681〜3697 行目（test_item_delete_no_kosu）

今:

コピー

```
    # 【テスト】指定日に工数データが無いと時間帯削除ができないこと
    #   前提: 2026-05-01 の工数データは無い
    #   操作: POST /api/item_delete/ に就業日 2026-05-01 を送る
    #   期待: 401（このAPIは「工数データが見つかりません」の場合も 401 を返す）
    def test_item_delete_no_kosu(self):
        # 準備: 管理者ユーザーでログイン
        self.login_as_admin()
        # 準備: 工数データの無い日の削除内容
        data = {
            'work_day2': '2026-05-01',
            'time1': '2026-05-01T00:00:00.000Z',
            'time2': '2026-05-01T01:00:00.000Z',
        }
        # 操作: 時間帯削除APIを呼ぶ
        response = self.client.post('/api/item_delete/', data, format='json')
        # 検証: 401（工数データなし）
        self.assertEqual(response.status_code, 401)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 【テスト】指定日に工数データが無いと時間帯削除ができないこと
    #   前提: 2026-05-01 の工数データは無い
    #   操作: POST /api/item_delete/ に就業日 2026-05-01 を送る
    #   期待: 404（工数データが見つかりません。ログインが切れた時の 401 とは分ける）
    def test_item_delete_no_kosu(self):
        # 準備: 管理者ユーザーでログイン
        self.login_as_admin()
        # 準備: 工数データの無い日の削除内容
        data = {
            'work_day2': '2026-05-01',
            'time1': '2026-05-01T00:00:00.000Z',
            'time2': '2026-05-01T01:00:00.000Z',
        }
        # 操作: 時間帯削除APIを呼ぶ
        response = self.client.post('/api/item_delete/', data, format='json')
        # 検証: 404（工数データなし）
        self.assertEqual(response.status_code, 404)
```

場所: `kosu/tests/test_backend.py` の 3699〜3720 行目（test_item_delete_no_time_work）

今:

コピー

```
    # 【テスト】作業内容が未登録（None）の工数データでは時間帯削除ができないこと
    #   前提: 2026-04-08 に作業内容 None の工数データがある
    #   操作: POST /api/item_delete/ に就業日 2026-04-08 を送る
    #   期待: 401（「工数データが見つかりません」）
    def test_item_delete_no_time_work(self):
        # 準備: 管理者ユーザーでログインし、作業内容が None の工数データを作成（変数 kosu_empty は以降未使用）
        self.login_as_admin()
        kosu_empty = Business_Time_graph.objects.create(
            employee_no3=12345, name=self.member,
            work_day2=date(2026, 4, 8), def_ver2='TestVer',
            tyoku2='1', over_time=0, time_work=None
        )
        # 準備: その日の削除内容
        data = {
            'work_day2': '2026-04-08',
            'time1': '2026-04-08T00:00:00.000Z',
            'time2': '2026-04-08T01:00:00.000Z',
        }
        # 操作: 時間帯削除APIを呼ぶ
        response = self.client.post('/api/item_delete/', data, format='json')
        # 検証: 401（作業内容なし）
        self.assertEqual(response.status_code, 401)
```

直した後（上の行を、全部これに置き換える）:

コピー

```
    # 【テスト】作業内容が未登録（None）の工数データでは時間帯削除ができないこと
    #   前提: 2026-04-08 に作業内容 None の工数データがある
    #   操作: POST /api/item_delete/ に就業日 2026-04-08 を送る
    #   期待: 404（「工数データが見つかりません」）
    def test_item_delete_no_time_work(self):
        # 準備: 管理者ユーザーでログインし、作業内容が None の工数データを作成
        self.login_as_admin()
        Business_Time_graph.objects.create(
            employee_no3=12345, name=self.member,
            work_day2=date(2026, 4, 8), def_ver2='TestVer',
            tyoku2='1', over_time=0, time_work=None
        )
        # 準備: その日の削除内容
        data = {
            'work_day2': '2026-04-08',
            'time1': '2026-04-08T00:00:00.000Z',
            'time2': '2026-04-08T01:00:00.000Z',
        }
        # 操作: 時間帯削除APIを呼ぶ
        response = self.client.post('/api/item_delete/', data, format='json')
        # 検証: 404（作業内容なし）
        self.assertEqual(response.status_code, 404)
```

場所: `kosu/tests/test_backend.py` の 4803〜4817 行目（test_check_task_status_success）

今:

コピー

```
    # 【テスト】成功したタスクの状況確認で、結果ファイルのパスが返ること
    #   前提: タスクID「test-check-1」・状態 success・結果 "/path/to/file.xlsx" のタスク記録がある
    #   操作: GET /api/check_backup_status?task_id=test-check-1
    #   期待: 200 が返り、本文の status が "success"、file_path が "/path/to/file.xlsx"
    def test_check_task_status_success(self):
        # 準備: 成功済みのタスク記録を作成（result には作成したファイルのパスが入る）
        task = AsyncTask.objects.create(
            task_id='test-check-1', status='success', result='/path/to/file.xlsx')
        # 操作: タスク状況確認APIを呼ぶ
        response = self.client.get('/api/check_backup_status', {'task_id': 'test-check-1'})
        # 検証: 200 で、本文（JSON）の状態とファイルパスが正しい
        self.assertEqual(response.status_code, 200)
        data = response.json()
        self.assertEqual(data['status'], 'success')
        self.assertEqual(data['file_path'], '/path/to/file.xlsx')
```

▶ ここで初めて出てくるコード

- `.json()`（Django）：テストで受け取った答え（レスポンス）の本文を、JSON として読んで、Python のリストや辞書にするメソッド。`self.assertEqual(res.json(), [])` 。

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 【テスト】成功したタスクの状況確認で、結果ファイルの名前（フォルダの場所は除く）が返ること
    #   前提：タスクID「test-check-1」・状態 success・結果 "/path/to/file.xlsx" のタスク記録がある
    #   操作：GET /api/check_backup_status?task_id=test-check-1
    #   期待：200 が返り、本文の status が "success"、file_path が "file.xlsx"
    def test_check_task_status_success(self):
        # 準備：成功済みのタスク記録を作成（result には作成したファイルのパスが入る）
        AsyncTask.objects.create(
            task_id='test-check-1', status='success', result='/path/to/file.xlsx')
        # 操作：タスク状況確認APIを呼ぶ
        response = self.client.get('/api/check_backup_status', {'task_id': 'test-check-1'})
        # 検証：200 で、本文（JSON）の状態とファイル名が正しい
        self.assertEqual(response.status_code, 200)
        data = response.json()
        self.assertEqual(data['status'], 'success')
        self.assertEqual(data['file_path'], 'file.xlsx')
```

場所: `kosu/tests/test_backend.py` の 4868〜4883 行目（test_download_file）

今:

コピー

```
    # 【テスト】指定したパスのファイルをダウンロードできること
    #   前提：一時フォルダに .xlsx ファイルがある
    #   操作：GET /api/download_backup?file_path=<一時ファイルのパス>
    #   期待：200
    def test_download_file(self):
        # 準備：中身 "test content" の一時ファイルを作成し、そのパスを控える
        #   delete=False：with ブロックを抜けてもファイルを自動削除しない
        with tempfile.NamedTemporaryFile(suffix='.xlsx', delete=False) as f:
            f.write(b'test content')
            temp_path = f.name
        # 操作：ファイルダウンロードAPIを呼ぶ
        response = self.client.get('/api/download_backup', {'file_path': temp_path})
        # 検証：200（成功）
        self.assertEqual(response.status_code, 200)
        # ※一時ファイルはテスト側では削除していない
        #   （元のコメント：Windows のファイルロックのため後片付けを省略し、OS に任せる）
```

直した後（上の行を、全部これに置き換える）：

コピー

```
    # 【テスト】media フォルダの中のファイルを、ファイル名でダウンロードでき、送った後に消えること
    #   前提: media フォルダ（テストでは一時フォルダ。test_settings.py の MEDIA_ROOT）に .xlsx ファイルがある
    #   操作: GET /api/download_backup?file_path=<ファイル名>
    #   期待: 200 で中身が返り、読み終わるとファイルが消える
    def test_download_file(self):
        # 準備: media フォルダに中身 "test content" のファイルを作る
        from django.conf import settings
        os.makedirs(settings.MEDIA_ROOT, exist_ok=True)
        file_path = os.path.join(settings.MEDIA_ROOT, 'test_download.xlsx')
        with open(file_path, 'wb') as f:
            f.write(b'test content')
        # 操作: ファイルダウンロードAPIを呼ぶ（ファイル名だけを渡す）
        response = self.client.get('/api/download_backup', {'file_path': 'test_download.xlsx'})
        # 検証: 200（成功）で中身が返り、読み終わるとファイルが消える
        self.assertEqual(response.status_code, 200)
        self.assertEqual(b''.join(response.streaming_content), b'test content')
        self.assertFalse(os.path.exists(file_path))
```

9. 直す順番（おすすめ）

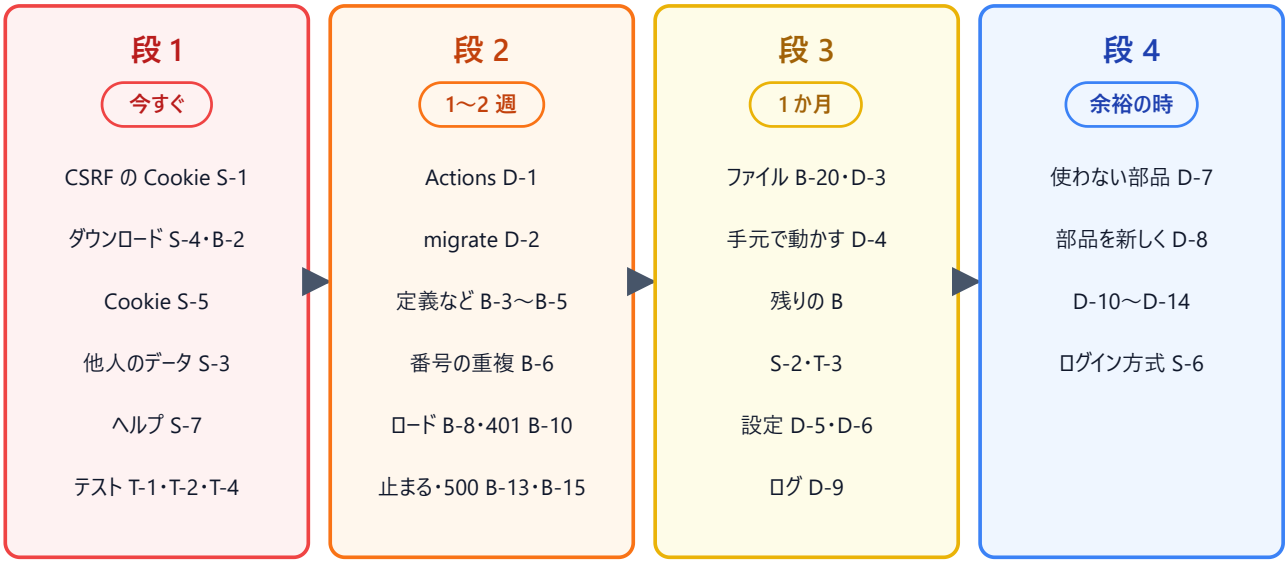


図2: 直す順番のおすすめ。左ほど急ぐ。どの段も「作業用のブランチで直す → テスト → 確かめてから master へ」（Actions を動かした後は、master に入れた物がそのまま本番に出るため）

段	直す物	目安
1（今すぐ・まとめて 1 回で出す）	S-1・T-1 → S-4（B-2 も直る）→ S-5 → B-7 の① → S-3 → S-7 → T-2・T-4。S-9 の 1・2（リポジトリの公開範囲と、会社への連絡）は、コードを直すより先に	数日
2（1〜2 週）	D-1（Actions を動かす前に）・D-2、B-3〜B-6・B-8・B-10・B-13・B-15、S-6 の C（セッションを短く）	1〜2 週
3（1 か月）	B-20・D-3・D-4、残りの B（B-11・B-12・B-14・B-16〜B-19・B-21）、S-2・T-3、D-5・D-6・D-9	1 か月
4（余裕の時）	D-7・D-8（部品を新しく）・D-10〜D-14、S-6 の A か B（ログインの方式）	数か月

- 段 1 は、全部まとめて 1 回で本番に出すのがおすすめです。どれも「ログインしている人の書き込み」に関わり、別々に出すと、そのたびに確かめが要るためです。
- どの段も、§4 の手順（作業用のブランチ → テスト → 手元で確かめる → Pull Request → master）で出します。
- 出した後は、本番で、ログイン・工数の入力・ログアウト・人員の一覧・データ管理のバックアップとダウンロードを、一通り確かめます（09 巻 §14.4）。

---

10. この巻で確かめたこと

10.1 確かめ方

- GitHub の `master`（コミット `ec2cedc`）を、`git archive` で手元の別のフォルダに書き出しました（アプリのフォルダ・本番・GitHub には、何も書き込んでいない）。
- アプリに入っているテスト用の設定（`hozen_another/test_settings.py`。データベースはメモリの中だけの SQLite）で、Django のテスト用の `Client` を使って API に頼みを送りました。「ブラウザと同じ流れ」の確かめ（S-1）は、CSRF を確かめる状態（`enforce_csrf_checks=True`）で行いました。
- この巻の直し（164 か所）を、全部、別の写しに入れて、同じことを確かめました。直し方のコードは、その写しに入れた物と同じです（同じ一覧から、この巻のコードの枠を作っている）。
- 画面のテストとビルドは、この PC の `frontend/node_modules` を借りて、写しの上で流しました（D-8 の版のずれに注意）。
- GitHub Actions の状態とリポジトリの公開範囲は、GitHub の API で読みました（何も変えていない）。
- **本番のサーバー・データベースには、一切触っていません。**本番で同じことを試さないでください。

10.2 結果

確かめたこと	今の `master`（`ec2cedc`）	この巻の直しを入れた写し
画面を開いた時の `csrftoken` の Cookie	来ない	来る
ログイン後の書き込み・ログアウト・ログインし直し（CSRF を確かめる状態）	403・403・403	200・200・200
A が B の工数を開く（`kosu_update` の GET）	200（見えた）	404
A が B の工数の就業日を変える	200（変わった）	404（変わらない）
L が班でない人の工数を開く（`team_detail`）	200	404
A が B の問い合わせを書き換える・消す	200・204	403・403
L が管理者を作る・自分を管理者にする・管理者を消す	201・200・204	403・403・403
管理者が `media` の外のファイルをダウンロード	200（中身が返り、消そうとした）	404（消えない）
`file_path` 無しのダウンロード	500	404
別のサイトを名乗った CORS の問い合わせ	許可の見出しは返らない（直っている）	同じ
ヘルプの違うパスワードを続けて送る	10 回とも 403（止まらない）	6 回目から 429（T-2 のテスト）
`HELP_PATH` が無い時のヘルプ	500	403
ダウンロードの URL `/undefined/api/download_backup` の行き先	画面を返す関数（`react_view`）	（URL を直したので使わない）
定義の Ver 名を変えた後、その Ver の工数を開く	401	200（工数の `def_ver2` もそろう）
工数の残っている定義を消す	204（消えた）	409（消えない）
班員に人員に無い番号がある時の班の画面 2 つ	500・500	200・200
人員から消した人の問い合わせを開く	401	200
同じ番号の人を 2 人入れる	入った。その人の工数の保存・人員の編集画面が 500	データベースが止める

確かめたこと	今の `master`（`ec2cedc`）	この巻の直しを入れた写し
設定データが無い時の、ショップの画面・問い合わせの登録	500・500	200・200
無い定義の Ver を選ぶ	200（その後の工数入力が 401）	400
思っていない値（B-15 の 11 か所）	500	400
日本時間 2 月 1 日 5 時のタスクを「2 月」で絞り込む	出ない（1 月に出る）	出る
`/BAlogin/` の行き先	ログインの API	画面を返す関数
モデルとマイグレーションの食い違い（`makemigrations --check`）	26 か所	無い（`0013`・`0014`）
GitHub に記録された `docs/pdf/README.pdf`	壊れている（B-20）	（直し方を書いた。写しでは確かめていない）
アプリのテスト	339 個 全部 OK	353 個（T-2 の 14 個を足した）全部 OK
T-2 のテスト（14 個）	14 個とも落ちる	14 個とも OK
画面のテスト	535 個のうち 3 個 失敗	534 個 全部 OK
画面のビルド（`vite build`）	—	通った（`frontend/build/index.html` が Vite の物になる）

11. この巻のまとめ

- 前の版で「いちばん大きな問題」と書いた **API ごとの確かめの書き忘れ** は、共通の確かめ（`permissions.py`）で片付きました。
- ただし、**CSRF トークンの Cookie を渡す所が抜けている** ため、今のまま本番に出すと、ログインした後の保存・削除・ログアウトが全部できなくなります（S-1）。Actions は止めてあるので、まだ本番には出ていません。**S-1 を直すまで、Actions を動かさないでください。**
- 次に急ぐのは、他人のデータ（S-3）、ダウンロード（S-4・B-2）、Cookie（S-5）、ヘルプ（S-7）です。リポジトリの Excel（S-9）は、コードより先に、公開範囲の確認と会社への連絡をします。
- どの直しも、**作業用のブランチで直し、テストを流し、手元で確かめてから** master に入れます。Actions を動かした後は、master に入れた物がそのまま本番に出ます。
- 直したら、この巻の該当する節に「済み（日付・コミット）」と書き足しておくと、次の人が助かります。